Computer & Network Security

Dola Mam's Lectures & Notes

CSE 4115 ? Computer & Network Security

Comprehensive Course Slide Deck with Embedded Navigation & Table of Contents

Table of Contents

1	Chapter 1: Threats to Network Security	Page 3
2	Chapter 2: Program Security & Non-Malicious Flaws	Page 29
3	Chapter 3: Format String Vulnerabilities	Page 75
4	Chapter 4: Salami Attack & Targeted Attacks	Page 80
5	Chapter 5: Protection in General-Purpose Operating Systems	Page 101
6	Chapter 6: Operating System Security & Access Control	Page 205
7	Chapter 7: Firewalls & Network Intrusion Prevention	Page 218

Threats to Network Security

Dola Das
Assistant Professor, Dept. of CSE,
KUET

Tampering

- Changing or deleting a resource without authorization.
- Usually caused for business purposes.
- One of the biggest security threats.
- Faced by web applications.
- Started in the late 1980’s as a way to sabotage data or plant a malicious or destructive program to delete data.

Types of Data Tampering

- COOKIE TAMPERING.
- HTML FORM FIELDS TAMPERING.
- URL QUERY STRINGS TAMPERING.
- HTTP HEADER TAMPERING.
- PASSWORD CRACKING TAMPERING.

Risks Involved in Data Tampering

- Hacker can eavesdrop on important conversions.
- Browser cache may contain contents of private messages.
- Data validation may allow SQL injection: SQL injection is a process whereby the hacker would insert a malicious code into strings that are later passed to a SQL Server for parsing and execution leaving it vulnerable to attack.
- Hackers or attackers may be able to read, change or alter other user’s messages.
- Authorization may fail, allowing unauthorized access leading to deletion of important files.

Prevention and Countermeasures

Prevention:

- Enforce Encryption for Data-at-Rest and Data-in-Transit.
- primary defense is to use a firewall and windows security to lock down important files, directories and other resources.
- The web application should also run with minimum privileges.
- File Integrity Monitoring (FIM).
- WORM systems (Write Once Read Many).
- Copy-on-Write File Systems.

Prevention and Countermeasures

Counter-Measures:

- ✓by using data signing and hashing,
- ✓using digital signatures,
- ✓using strong authorization,
- ✓using tamper resistant protocols across communication links, using secure communication links with protocols that provide message integrity,
- ✓by using strong and powerful firewalls, and long passwords that consist of alphanumeric characters,
- ✓by blocking IP addresses for a certain period of time which will cause repeated failed login attempts by the attacker.

Wiretapping

- Wiretapping is a way to get information about people without them knowing. This is done through, as the name suggests, wires.
- Wiretapping is a form of surveillance in which wires are used to intercept and monitor communications from a phone line.
- Wiretaps on the phone lines can be both intentional or unintentional.
- can be implemented for political, industrial, or criminal purposes.
- can be done at either the source or destination point, or both, depending on how it’s configured

How Does Wiretapping Work?

- When two people on the phone are having a conversation, sounds translate into electrical currents.
- For sound, the traveling is fast, but that does not mean that its path is short.
- Wiretapping is when another wire, a third-party wire, connects to your telephone line.

Who Does Wiretapping?

- Amateur wiretappers.
- Professional wiretappers.
- Legal Reasons (in the United States of America).
- Illegal Reasons.

What Can Be Wiretapped?

- Instant Messages.
- Smartphones.
- Telephones.

What Are the Types of Wiretapping?

- Transmit Wiretap.
- Recording Wiretap.
- Hardwired Wiretap.
- Soft Wiretap.

9 Signs Your Phone Is Wiretapped

- Unknown Sounds on Phone Calls.
- Phone Turning On and Off.
- Battery Easily Draining.
- Phone Always Warm.
- Data Usage Rise on Your Mobile Bill.
- Receiving Unwanted Text Messages.
- Secrets Revealed.
- Long Shutdown Process.
- Third-Party Apps Requesting For Personal Details.

Ways to Keep Phone Safe From Wiretapping: For Individuals

- Do Not Use Just Any Public Wi-Fi.
- Do Not Leave Your Phone Unattended.
- Do Disable Your Bluetooth.
- Do Take The Time to Study Your Phone.
- Do Double-Check Your Phone’s Settings.
- Do Install Antivirus Software.

Ways to Keep Phone Safe From Wiretapping: For Businesses

- Do Have Data Security Policies.
- Do Have a Strong PBX.
- Do Use Security Protection.
- Do Take The Time to Study Your Phone.
- Do Be Open to Investigation.

Impersonation

- The act of deceiving someone by pretending to be another person.
- Cyber criminals have been using it to gain access to networks and systems to commit fraud and identity theft and sell data to the highest bidder on the dark web.
- The practice of “pretexting” is defined as presenting oneself as someone else to manipulate a recipient into providing sensitive data.
- Online impersonation victims can experience defamation or extreme embarrassment.

Impersonation

- CEO fraud, business email compromise and whaling are specific forms of impersonation attacks.
- These attacks are designed to take advantage of human error.
- You need solutions that can automatically scan email and block any potential attack.

What is an Email Impersonation Attack?

- type of phishing attack.
- impersonates a legitimate sender in order to trick the recipient into clicking on a malicious link or attachment.
- Email impersonation attacks can be difficult to detect, as the attacker will often use a fake email address that is similar to the legitimate sender's address.

How does an Impersonation Attack Work?

- Impersonation attacks are typically malware-less attacks
- conducted through email using social engineering to gain the trust of a targeted employee
- The employee receives an email that appears to be from a legitimate source, often a high-level executive within the company, urgently requesting that money be wired to a certain account or that sensitive information be sent immediately.

What Are Common Email Impersonation Attacks?

- Urgent, short-notice requests
- Purchase requests to be completed on behalf of the CEO, often gift cards.
- Employees abruptly requesting changes to direct deposit information.
- Creating emails with slight name deviations such as writing the letter "m" as "rn".
- Using language that induces urgency and a sense of fear.

How to Recognize an Impersonation Attack?

Unlike common phishing attacks, impersonation attacks are highly targeted and well-crafted to appear realistic and authentic.

- An urgent and possibly threatening tone
- An emphasis on confidentiality
- A request to send money or share sensitive information
- A problem with email addresses or links
- Unusual requests or accounts

How to Stop an Impersonation Attack?

- Security awareness training
- DNS authentication services
- Anti-malware and anti-spam
- etc.

Hacking

- Refers to the misuse of devices like computers, smartphones, tablets, and networks to cause damage to or corrupt systems.
- Hackers are generally curious programmers who enjoy getting into the nuts and bolts of how computer systems and networks (usually TCP/IP) work.
- Hackers usually guard their identity carefully and use purely technological means to try to break into a system.

Hacking

- Examples of network hacking tools include:
  - ✓Kismet (Wireless Hacking)
  - ✓Sniffing & Monitoring
  - ✓THC-Hydra
  - ✓Infernal Twin
  - ✓Firesheep

Hacking

- There are typically four key drivers that lead to bad actors hacking websites or systems:
  - ✓ financial gain through the theft of credit card details or by defrauding financial services
  - ✓ corporate espionage
  - ✓ to gain notoriety or respect for their hacking talents
  - ✓ state-sponsored hacking that aims to steal business information and national intelligence.

  On top of that, there are politically motivated hackers—or hacktivists—who aim to raise public attention by leaking sensitive information, such as Anonymous.
- Types of Hacking/Hackers:
  - ✓ Black Hat Hackers
  - ✓ White Hat Hackers
  - ✓ Grey Hat Hackers

Hacking

- Devices Most Vulnerable To Hacking:
  - ✓ Smart Devices
  - ✓ Webcams
  - ✓ Routers
  - ✓ Email
  - ✓ Jailbroken Phones
- Prevention from Getting Hacked:
  - ✓ Software Update
  - ✓ Use Unique Passwords for Different Accounts
  - ✓ Avoid Clicking on Ads or Strange Links
  - ✓ Change the Default Username and Password on Your Router and Smart Devices

References

- https://www.mbaknol.com/information-systems-management/data-tampering-meaning-types-and-countermeasures/#:~:text=Tampering%20is%20one%20of%20the%20biggest%20security%20threats,a%20malicious%20or%20destructive%20program%20to%20delete%20data.
- https://robots.net/tech/wiretapping-everything-you-need-to-know/
- https://www.mimecast.com/content/impersonation-attack/#:~:text=An%20impersonation%20attack%20is%20a%20form%20of%20fraud,used%20to%20hack%20into%20a%20company%E2%80%99s%20computer%20network.
- https://www.fortinet.com/resources/cyberglossary/what-is-hacking

01074305 Computer Security

Chapter 3

Program Security

Charles P. Pfleeger & Shari Lawrence Pfleeger, Security in Computing, 4th Ed., Pearson Education, 2007

- **In this chapter**
  - Programming errors with security implications: buffer overflows, incomplete access control
  - Malicious code: viruses, worms, Trojan horses
  - Program development controls against malicious code and vulnerabilities: software engineering principles and practices
  - Controls to protect against program flaws in execution: operating system support and administrative controls

3.1. Secure Programs

- Security implies some degree of trust that the program enforces expected confidentiality, integrity, and availability
- An assessment of security can also be influenced by someone's general perspective on software quality
  - E.g., if your manager's idea of quality is conformance to specifications, then she might consider the code secure if it meets security requirements, whether or not the requirements are complete or correct.

- **IEEE Terminology for Quality**
  - A **bug** can be a mistake in interpreting a requirement, a syntax error in a piece of code, or the (as-yet-unknown) cause of a system crash.
  - When a human makes a mistake, called an **error**, in performing some software activity, the error may lead to a **fault**, or an incorrect step, command, process, or data definition in a computer program.
  - A **failure** is a departure from the system's required behavior.
  - a fault is an inside view of the system, as seen by the eyes of the developers, whereas a failure is an outside view: a problem that the user sees.

- **Fixing Faults**
  - A module in which 100 faults were discovered and fixed is better than another in which only 20 faults were discovered and fixed, suggesting that more rigorous analysis and testing had led to the finding of the larger number of faults (?)
  - Early work in computer security was based on the paradigm of "**penetrate and patch**," in which analysts searched for and repaired faults.

- **Fixing Faults (Cont'd)**
  - However, the patch efforts were largely useless, making the system less secure rather than more secure because they frequently introduced *new* faults.
    - The pressure to repair a specific problem encouraged a narrow focus on the fault itself and not on its context.
    - The fault often had nonobvious side effects in places other than the immediate area of the fault.
    - Fixing one problem often caused a failure somewhere else
    - The fault could not be fixed properly because system functionality or performance would suffer as a consequence

- **Unexpected Behavior**
  - To understand program security, we can examine programs to see whether they behave as their designers intended or users expected.
  - Such unexpected behavior a **program security flaw**; it is inappropriate program behavior caused by a program vulnerability.
  - Program security flaws can derive from any kind of software fault
  - Divide program flaws into two separate logical categories: **inadvertent human errors** versus **malicious, intentionally induced flaws**.

- Regrettably, we do not have techniques to eliminate or address all program security flaws.
- Security *is* fundamentally hard, security often conflicts with usefulness and performance, there is no ""silver bullet" to achieve security effortlessly, and false security solutions impede real progress toward more secure programming

- There are two reasons for this distressing situation.
  1. Program controls apply at the level of the individual program and programmer.
  2. Programming and software engineering techniques change and evolve far more rapidly than do computer security techniques.

- **Types of Flaws**
  - *validation* error (incomplete or inconsistent): permission checks
  - *domain* error: controlled access to data
  - *serialization* and *aliasing*: program flow order
  - inadequate *identification and authentication*: basis for authorization
  - *boundary condition* violation: failure on first or last case
  - other exploitable *logic errors*

3.2. Nonmalicious Program Errors

- **Buffer Overflows**
  - A buffer (or array or string) is a space in which data can be held.
  - A buffer's capacity is finite.

- Suppose a C language program contains the declaration:

```
char sample[10];
```

- Now we execute the statement:

```
sample[10] = 'B';
```

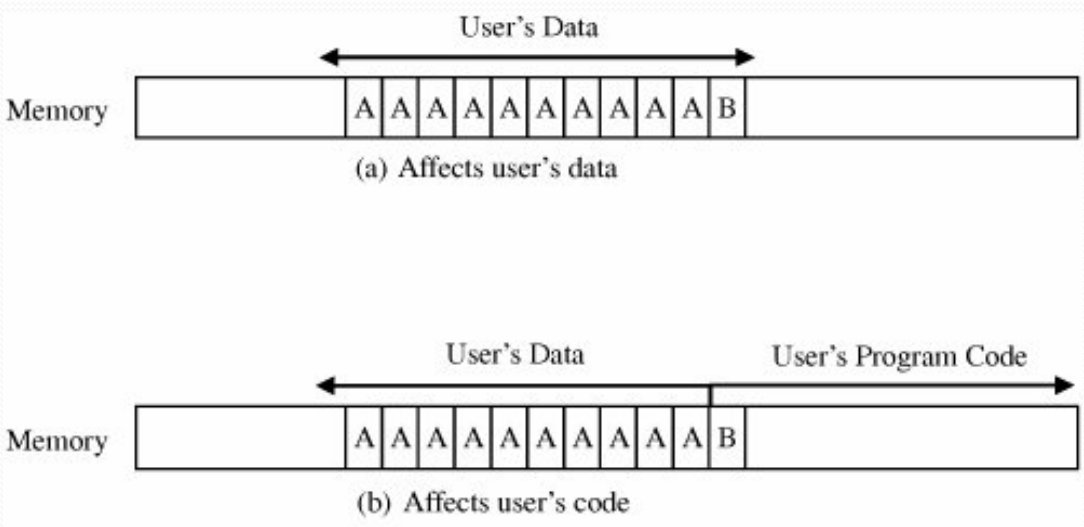
- However, if the statement were

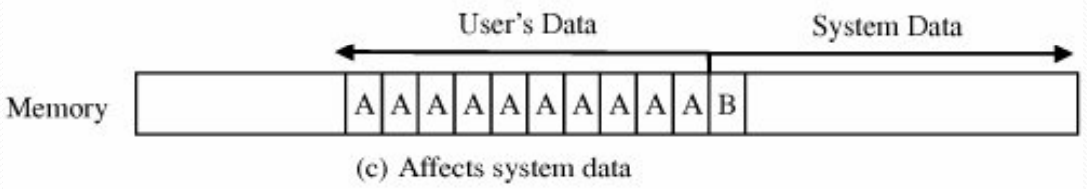
```
sample[i] = 'B';
```

we could not identify the problem until i was set during execution to a too-big subscript.

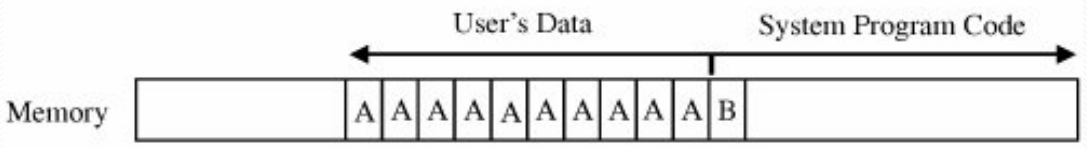
- Suppose each of the ten elements of the array sample is filled with the letter A and the erroneous reference uses the letter B, as follows:

```
for (i=0; i<=9; i++)
      sample[i] = 'A';
sample[10] = 'B‘;
```



(c) Affects system data



(d) Affects system code

- **Security Implication**
  - Two buffer overflow attacks that are used frequently
    1. The attacker may replace code in the system space. By replacing a few instructions right after returning from his or her own procedure, the attacker regains control from the operating system, possibly with raised privileges.
    2. On the other hand, the attacker may make use of the stack pointer or the return register. Subprocedure calls are handled with a stack, a data structure in which the most recent item inserted is the next one removed (last arrived, first served).

- An alternative style of buffer overflow occurs when parameter values are passed into a routine, especially when the parameters are passed to a web server on the Internet. Parameters are passed in the URL line, with a syntax similar to

```
http://www.somesite.com/subpage/userinput.asp?
parm1=(808)555-1212 &parm2=2009Jan17
```

The attacker might question what the server would do with a really long telephone number, say, one with 500 or 1000 digits.

- **Incomplete Mediation**
  - Consider the example

```
http://www.somesite.com/subpage/userinput.asp?
parm1=(808)555-1212 &parm2=2009Jan17
```

  - What would happen if parm2 were submitted as 1800Jan01? Or 1800Feb30? Or 2048Min32? Or 1Aardvark2Many?

- Security Implication
  - Consider this example

```
http://www.things.com/order.asp?custID=101&part=555A&q
y=20&price =10&ship=boat&shipcost=5&total=205
```

  - A malicious attacker may decide to exploit this peculiarity by supplying instead the following URL, where the price has been reduced from $205 to $25:

```
http://www.things.com/order.asp?custID=101&part=555A&q
y=20&price =1&ship=boat&shipcost=5&total=25
```


- **Time-of-Check to Time-of-Use Errors**
  - The **time-of-check to time-of-use** (TOCTTOU) flaw concerns mediation that is performed with a "bait and switch" in the middle. It is also known as a serialization or synchronization flaw.

- Time-of-Check to Time-of-Use Errors (Cont'd)
  - Suppose a request to access a file were presented as a data structure, with the name of the file and the mode of access presented in the structure.

my_file	change byte 4 to "A"

  - To carry out this authorization sequence, the access control mediator would have to look up the file name in tables. The mediator could compare the names in the table to the file name in the data structure to determine whether access is appropriate. More likely, **the mediator would copy the file name into its own local storage area and compare from there**

- Time-of-Check to Time-of-Use Errors (Cont'd)
  - While the mediator is checking access rights for the file my_file, the user could change the file name descriptor to your_file

your_file	delete file

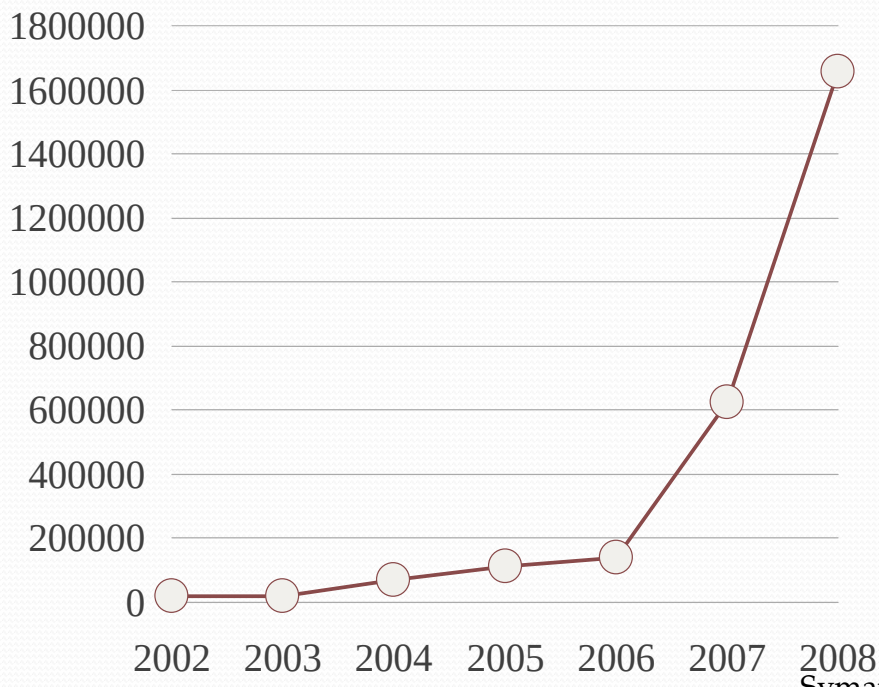
  - The problem is called a time-of-check to time-of-use flaw because **it exploits the delay between the two times**. That is, between the time the access was checked and the time the result of the check was used, a change occurred, invalidating the result of the check.

- Security Implication
  - Pretty clear
  - Checking one action and performing another is an example of ineffective access control
  - There are ways to prevent exploitation of the time lag.
    - One way is to ensure that critical parameters are not exposed during any loss of control.
    - Another way is to ensure serial integrity; that is, to allow no interruption (loss of control) during the validation.

3.3. Viruses and Other Malicious Code

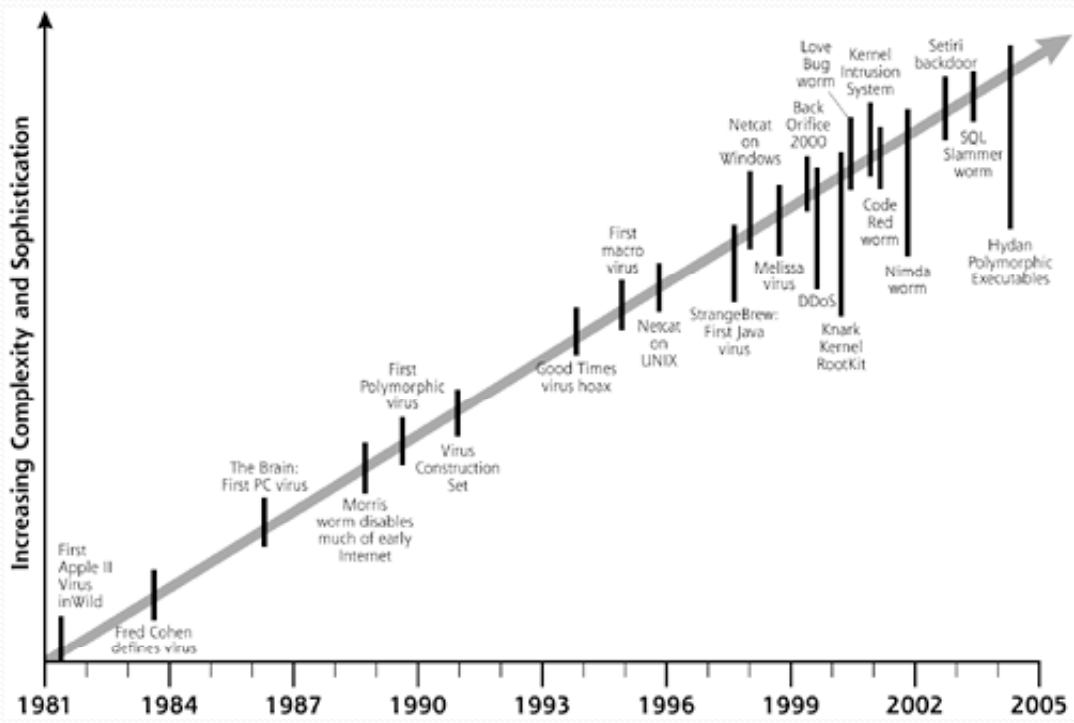
- **Malicious Code Can Do Much (Harm)**
  - Malicious code runs under the user's authority.
  - Thus, malicious code can touch everything the user can touch, and in the same ways.
  - Users typically have complete control over their own program code and data files; they can read, write, modify, append, and even delete them.
  - But malicious code can do the same, without the user's permission or even knowledge.

Number of malware signatures



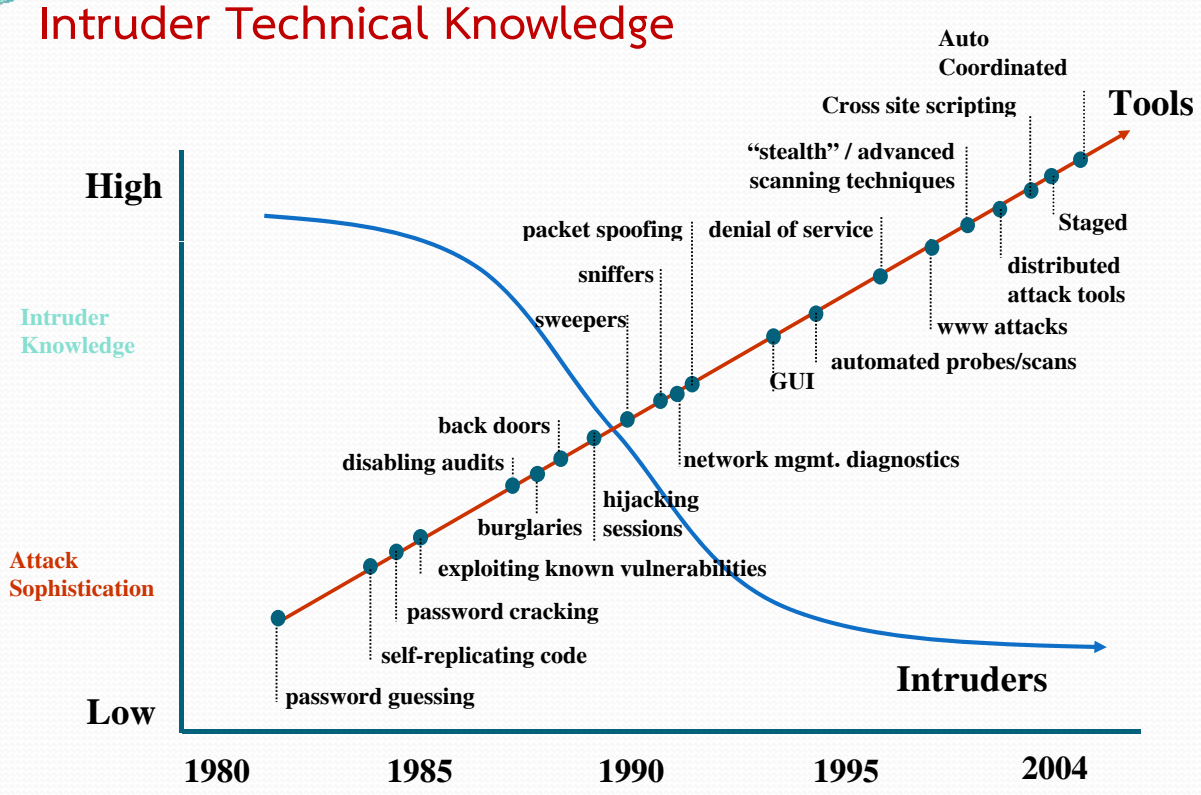
Symantec report 2009

Almost 30 years of Malware



From Malware fighting malicious code

Attack Sophistication vs. Intruder Technical Knowledge

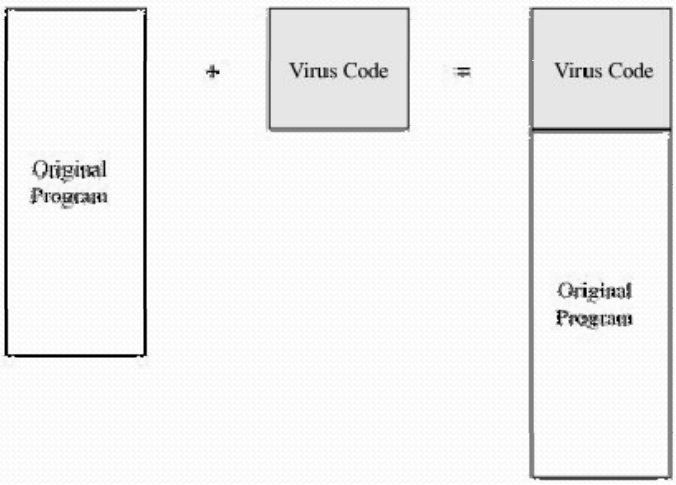


- Kinds of Malicious Code
  - **Malicious code** or **rogue program** is the general name for unanticipated or undesired effects in programs or program parts, caused by an agent intent on damage.
  - A **virus** is a program that can replicate itself and pass on malicious code to other nonmalicious programs by modifying them.
    - A **transient virus** has a life that depends on the life of its host
    - A **resident virus** locates itself in memory

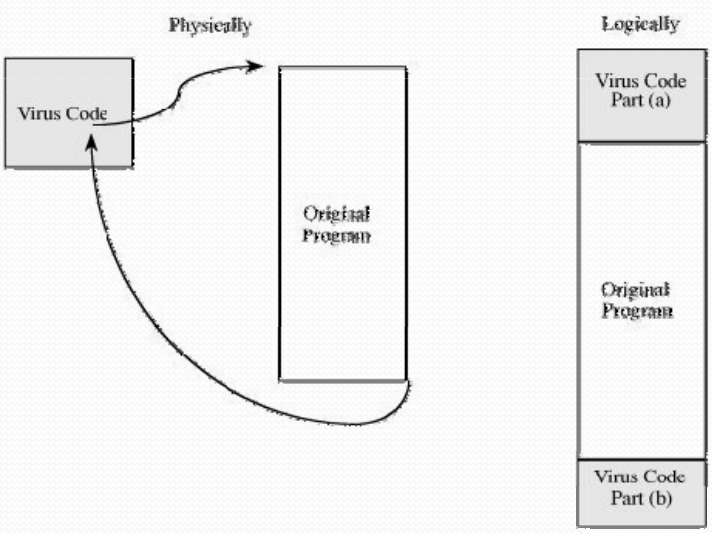
- Kinds of Malicious Code (Cont'd)
  - A **Trojan horse** is malicious code that, in addition to its primary effect, has a second, nonobvious malicious effect
  - A **logic bomb** is a class of malicious code that "detonates" or goes off when a specified condition occurs.
  - A **time bomb** is a logic bomb whose trigger is a time or date.
  - A **trapdoor** or **backdoor** is a feature in a program by which someone can access the program other than by the obvious, direct call

- Kinds of Malicious Code (Cont'd)
  - A **worm** is a program that spreads copies of itself through a network.
  - A **rabbit** is a virus or worm that self-replicates without bound, with the intention of exhausting some computing resource.

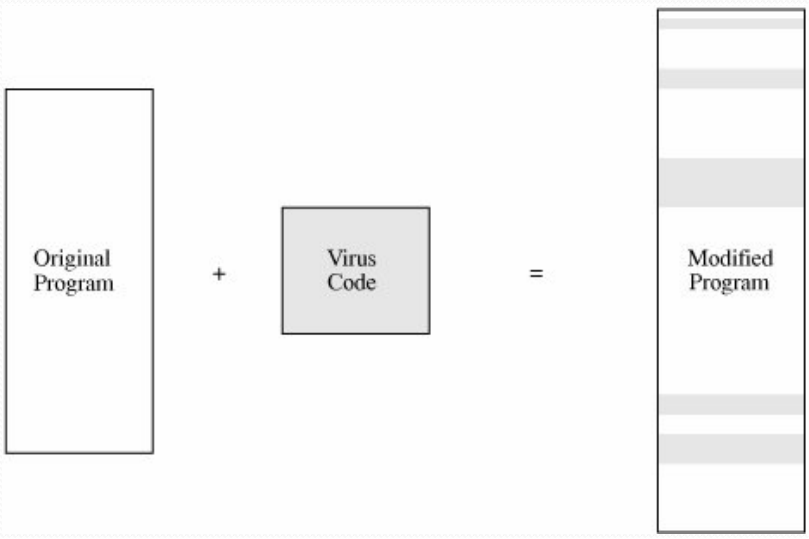
- How Viruses Attach
  - Appended Viruses



- How Viruses Attach (Cont'd)
  - Viruses That Surround a Program

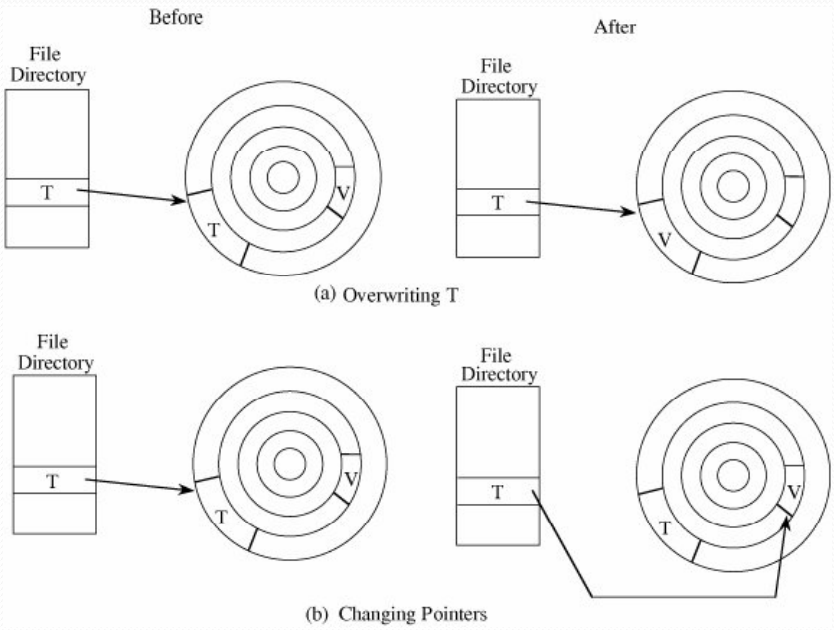


- How Viruses Attach (Cont'd)
  - Integrated Viruses and Replacements

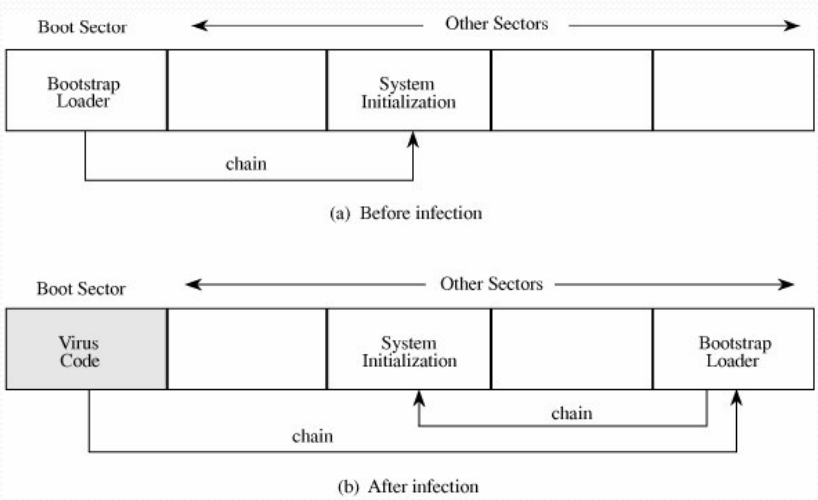


- How Viruses Attach (Cont'd)
  - Document Viruses
    - Implemented within a formatted document, such as a written document, a database, a slide presentation, a picture, or a spreadsheet.

- How Viruses Gain Control



- Homes for Viruses
  - **One-Time Execution** – the majority of viruses
  - **Boot Sector Viruses**

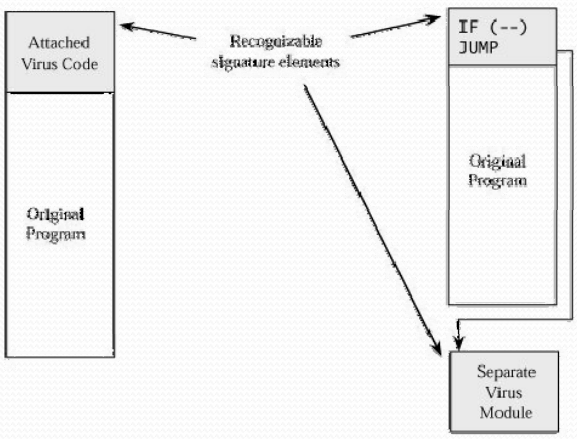


- Homes for Viruses (Cont'd)
  - Memory-Resident Viruses
  - Other Homes for Viruses
    - Application programs
    - Libraries
    - Data files – need a startup program

- Virus Signatures
  - A **signature** – a telltale pattern
    - E.g., signature for the Code Red

```
/default.ida?NNNN
```

- Virus Signatures
  - **Storage Patterns -** Most viruses attach to programs that are stored on media such as disks. The attached virus piece is invariant, so the start of the virus code becomes a detectable signature.



- Virus Signatures
  - **Execution Patterns** - A virus writer may want a virus to do several things at the same time, namely, spread infection, avoid detection, and cause harm.
  - **Transmission Patterns** - A virus is effective only if it has some means of transmission from one location to another.

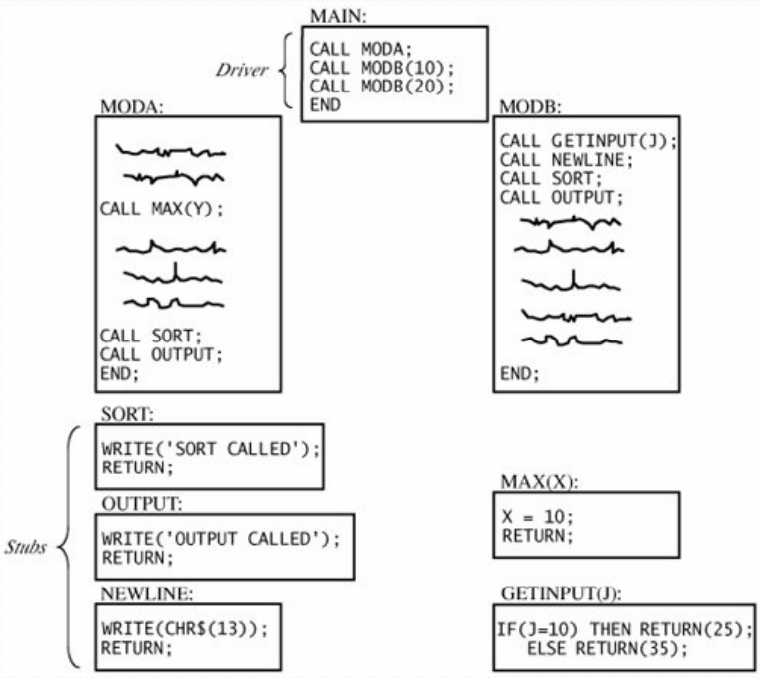
- **Polymorphic Viruses**
  - A virus that can change its appearance
  - **Encrypting viruses**
    - Uses encryption under various keys to make the stored form of the virus different.
    - Contain three distinct parts:
      - a decryption key,
      - the (encrypted) object code of the virus
      - the (unencrypted) object code of the decryption routine.

- **Prevention of Virus Infection**
  - The only way to prevent the infection of a virus is not to receive executable code from an infected source.

- **Prevention of Virus Infection (Cont'd)**
  - Several techniques for building a reasonably safe community for electronic contact, including the following:
    - *Use only commercial software acquired from reliable, well-established vendors.*
    - *Test all new software on an isolated computer.*
    - *Open attachments only when you know them to be safe*
    - *Make a recoverable system image and store it safely.*
    - *Make and retain backup copies of executable system files.*
    - *Use virus detectors (often called virus scanners) regularly and update them daily.*

3.4. Targeted Malicious Code

- **Trapdoors** - an undocumented entry point to a module
  - **Examples**
    - A system is composed of modules or components.
      - Programmers first test each small component of the system separate from the other components, in a step called **unit testing**, to ensure that the component works correctly by itself.
      - Then, developers test components together during **integration testing**, to see how they function as they send messages and data from one to the other.



- **Trapdoors** - an undocumented entry point to a module
  - **Examples**
    - Hardware processor design
      - The **undefined opcodes** sometimes implement peculiar instructions, either because of an intent to test the processor design or because of an oversight by the processor designer.
      - Undefined opcodes are the hardware counterpart of poor error checking for software.

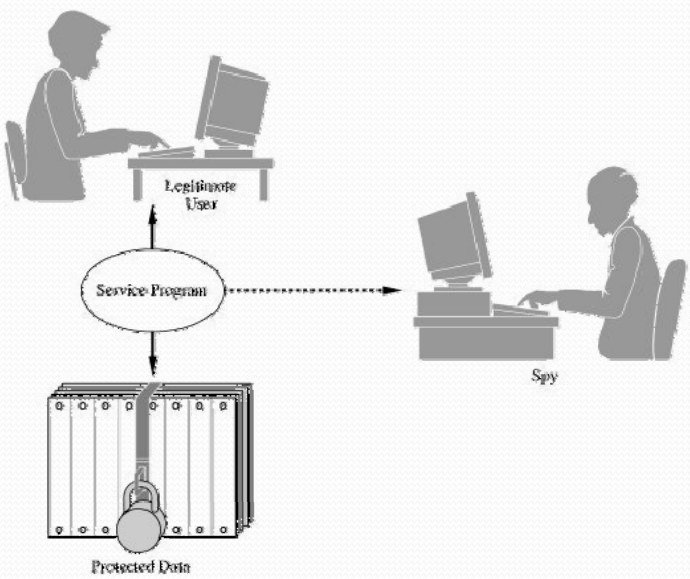
- Causes of Trapdoors
  - *forget* to remove them
  - intentionally leave them in the program for *testing*
  - intentionally leave them in the program for *maintenance* of the finished program, or
  - intentionally leave them in the program as a *covert means of access* to the component after it becomes an accepted part of a production system

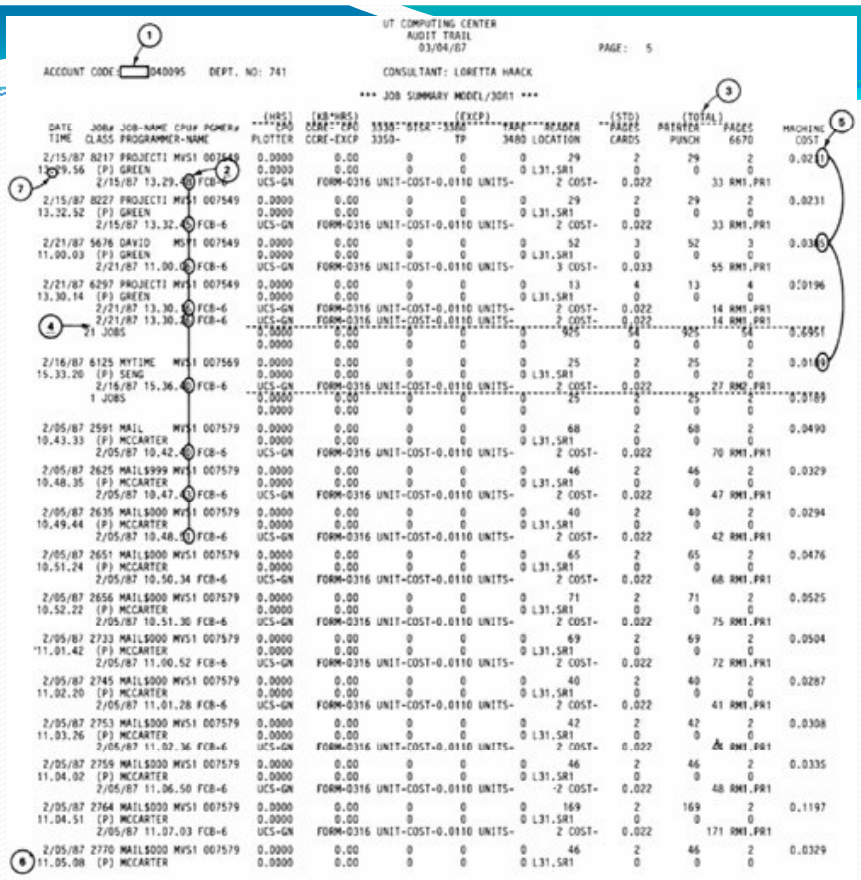
- Salami Attack
  - merges bits of seemingly inconsequential data to yield powerful results.

- Why Salami Attacks Persist
  - Computer computations are notoriously subject to small errors involving rounding and truncation, especially when large numbers are to be combined with small ones.

- **Privilege Escalation -** a means for malicious code to be launched by a user with lower privileges but run with higher privileges.
- **Interface Illusions -** a spoofing attack in which all or part of a web page is false.
- **Keystroke Logging -** retains a surreptitious copy of all keys pressed.
- **Man-in-the-Middle Attacks -** interjects itself between two other programs

- **Covert Channels: Programs That Leak Information**





UT COMPUTING CENTER
AUDIT TRAIL
03/04/87
PAGE: 5

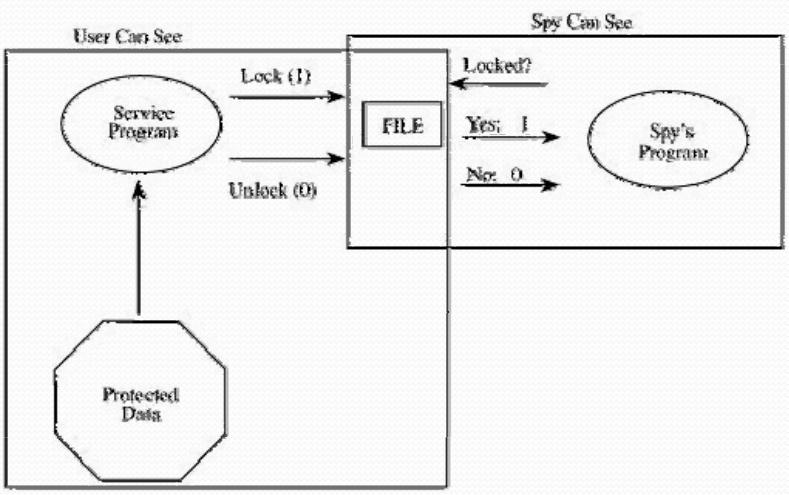
ACCOUNT CODE: 040095 DEPT. NO: 741 CONSULTANT: LORETTA HAACK

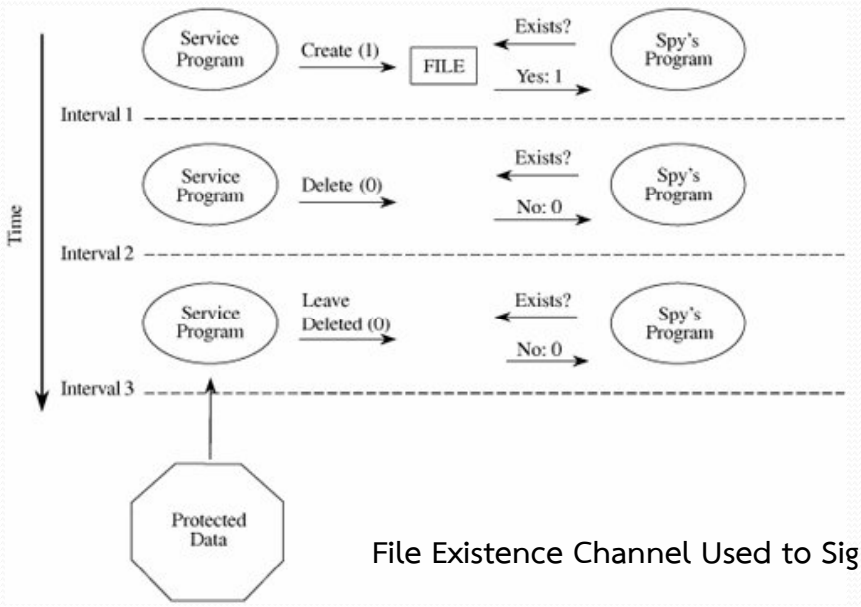
*** JOB SUMMARY MODEL/3081 ***

DATE TIME	JOB# CLASS	JOB-NAME PROGRAMMER-NAME	CPU# POWER#	(HRS) CPU PLOTTER	(KB*HRS) CORE-EXC	(EXCP) 3350-	TP	PAGE 3480	BEGPA LOCATION	(STD) PAGES CARDS	(TOTAL) PRINTER PUNCH	PAGES 6670	MACHINE COST
2/15/87 13:29.56	8217 (P)	PROJECT1 GREEN	MVS1 007549	0.0000	0.00	0	0	0	29	2	29	2	0.02
		2/15/87 13.29.56 FCB-6		0.0000	0.00	0	0	0	L31.SR1	0	0	0	
			UCS-GN		FORM-0316 UNIT-COST-0.0110 UNITS-				2 COST-	0.022		33 RM1.PR1	
2/15/87 13.32.52	8227 (P)	PROJECT1 GREEN	MVS1 007549	0.0000	0.00	0	0	0	29	2	29	2	0.0231
		2/15/87 13.32.52 FCB-6		0.0000	0.00	0	0	0	L31.SR1	0	0	0	
			UCS-GN		FORM-0316 UNIT-COST-0.0110 UNITS-				2 COST-	0.022		33 RM1.PR1	
2/21/87 11.00.03	5676 (P)	DAVID GREEN	MVS1 007549	0.0000	0.00	0	0	0	52	3	52	3	0.038
		2/21/87 11.00.03 FCB-6		0.0000	0.00	0	0	0	L31.SR1	0	0	0	
			UCS-GN		FORM-0316 UNIT-COST-0.0110 UNITS-				3 COST-	0.033		55 RM1.PR1	
2/21/87 13.30.14	6297 (P)	PROJECT1 GREEN	MVS1 007549	0.0000	0.00	0	0	0	13	4	13	4	0.0196
		2/21/87 13.30.14 FCB-6		0.0000	0.00	0	0	0	L31.SR1	0	0	0	
			UCS-GN		FORM-0316 UNIT-COST-0.0110 UNITS-				2 COST-	0.022		14 RM1.PR1	
		2/21/87 13.30.14 FCB-6	UCS-GN		FORM-0316 UNIT-COST-0.0110 UNITS-				2 COST-	0.022		14 RM1.PR1	
		21 JOBS		0.0000	0.00	0	0	0	925	64	925	64	0.6951
				0.0000	0.00	0	0	0		0	0	0	
2/16/87 15.33.20	6125 (P)	MYTIME SENG	MVS1 007569	0.0000	0.00	0	0	0	25	2	25	2	0.018
				0.0000	0.00	0	0	0	L31.SR1	0	0	0	
		2/16/87 15.36.20 FCB-6	UCS-GN		FORM-0316 UNIT-COST-0.0110 UNITS-				2 COST-	0.022		27 RM2.PR1	
		1 JOBS		0.0000	0.00	0	0	0	25	2	25	2	0.0189
				0.0000	0.00	0	0	0		0	0	0	
2/05/87 10.43.33	2591 (P)	MAIL MCCARTER	MVS1 007579	0.0000	0.00	0	0	0	68	2	68	2	0.0490
				0.0000	0.00	0	0	0	L31.SR1	0	0	0	
		2/05/87 10.42.33 FCB-6	UCS-GN		FORM-0316 UNIT-COST-0.0110 UNITS-				2 COST-	0.022		70 RM1.PR1	
2/05/87 10.48.35	2625 (P)	MAIL$999 MCCARTER	MVS1 007579	0.0000	0.00	0	0	0	46	2	46	2	0.0329
				0.0000	0.00	0	0	0	L31.SR1	0	0	0	
		2/05/87 10.47.35 FCB-6	UCS-GN		FORM-0316 UNIT-COST-0.0110 UNITS-				2 COST-	0.022		47 RM1.PR1	
2/05/87 10.49.44	2635 (P)	MAIL$000 MCCARTER	MVS1 007579	0.0000	0.00	0	0	0	40	2	40	2	0.0294
				0.0000	0.00	0	0	0	L31.SR1	0	0	0	
		2/05/87 10.48.44 FCB-6	UCS-GN		FORM-0316 UNIT-COST-0.0110 UNITS-				2 COST-	0.022		42 RM1.PR1	
2/05/87 10.51.24	2651 (P)	MAIL$000 MCCARTER	MVS1 007579	0.0000	0.00	0	0	0	65	2	65	2	0.0476
				0.0000	0.00	0	0	0	L31.SR1	0	0	0	
		2/05/87 10.50.34 FCB-6	UCS-GN		FORM-0316 UNIT-COST-0.0110 UNITS-				2 COST-	0.022		68 RM1.PR1	
2/05/87 10.52.22	2656 (P)	MAIL$000 MCCARTER	MVS1 007579	0.0000	0.00	0	0	0	71	2	71	2	0.0525
				0.0000	0.00	0	0	0	L31.SR1	0	0	0	
		2/05/87 10.51.30 FCB-6	UCS-GN		FORM-0316 UNIT-COST-0.0110 UNITS-				2 COST-	0.022		75 RM1.PR1	
2/05/87 11.01.42	2733 (P)	MAIL$000 MCCARTER	MVS1 007579	0.0000	0.00	0	0	0	69	2	69	2	0.0504
				0.0000	0.00	0	0	0	L31.SR1	0	0	0	
		2/05/87 11.00.52 FCB-6	UCS-GN		FORM-0316 UNIT-COST-0.0110 UNITS-				2 COST-	0.022		72 RM1.PR1	
2/05/87 11.02.20	2745 (P)	MAIL$000 MCCARTER	MVS1 007579	0.0000	0.00	0	0	0	40	2	40	2	0.0287
				0.0000	0.00	0	0	0	L31.SR1	0	0	0	
		2/05/87 11.01.28 FCB-6	UCS-GN		FORM-0316 UNIT-COST-0.0110 UNITS-				2 COST-	0.022		41 RM1.PR1	
2/05/87 11.03.26	2753 (P)	MAIL$000 MCCARTER	MVS1 007579	0.0000	0.00	0	0	0	42	2	42	2	0.0308
				0.0000	0.00	0	0	0	L31.SR1	0	0	0	
		2/05/87 11.02.36 FCB-6	UCS-GN		FORM-0316 UNIT-COST-0.0110 UNITS-				2 COST-	0.022		43 RM1.PR1	
2/05/87 11.04.02	2759 (P)	MAIL$000 MCCARTER	MVS1 007579	0.0000	0.00	0	0	0	46	2	46	2	0.0335
				0.0000	0.00	0	0	0	L31.SR1	0	0	0	
		2/05/87 11.06.50 FCB-6	UCS-GN		FORM-0316 UNIT-COST-0.0110 UNITS-				-2 COST-	0.022		48 RM1.PR1	
2/05/87 11.04.51	2764 (P)	MAIL$000 MCCARTER	MVS1 007579	0.0000	0.00	0	0	0	169	2	169	2	0.1197
				0.0000	0.00	0	0	0	L31.SR1	0	0	0	
		2/05/87 11.07.03 FCB-6	UCS-GN		FORM-0316 UNIT-COST-0.0110 UNITS-				2 COST-	0.022		171 RM1.PR1	
2/05/87 11.05.08	2770 (P)	MAIL$000 MCCARTER	MVS1 007579	0.0000	0.00	0	0	0	46	2	46	2	0.0329
				0.0000	0.00	0	0	0	L31.SR1	0	0	0	

① Number of spaces after :
② Last digit in field that would not be checked
③ Presence or absence of word (TOTAL) in header line
④ No space after last line of subtotal
⑤ Last digit in insignificant field
⑥ Number of lines per page
⑦ Use of . instead of :

- **Storage Channels** - pass information by using the presence or absence of objects in storage.





File Existence Channel Used to Signal 100

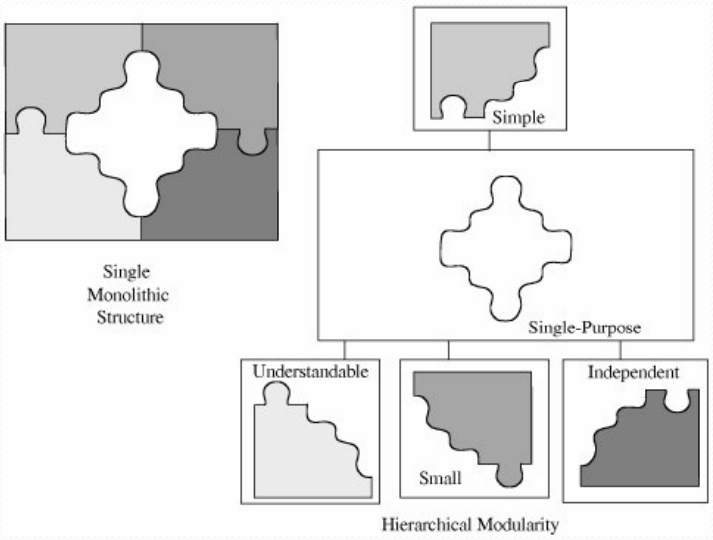
3.5. Controls Against Program Threats

- Three types of controls:
  - Developmental
  - Operating system
  - Administrative

- Developmental Controls
  - The Nature of Software Development
    - Collaborative effort, involving people with different skill sets who combine their expertise to produce a working product
    - Development requires people who can *specify*, *design*, *implement*, *test*, *review*, document, manage, *maintain* the system.

- Developmental Controls (Cont'd)
  - Modularity, Encapsulation, and Information Hiding
    - A key principle of software engineering is to create a design or code in small, self-contained units, called **components** or **modules**
    - If a component is isolated from the effects of other components, then it is easier to trace a problem to the fault that caused it and to limit the damage the fault causes. This isolation is called **encapsulation**.
    - **Information hiding** is another characteristic of modular software.

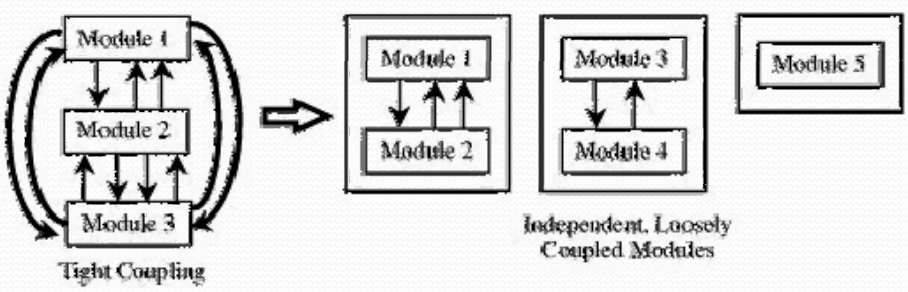
- Developmental Controls (Cont'd)
  - **Modularization** is the process of dividing a task into subtasks.



- Developmental Controls (Cont'd)
  - Modularization
    - The goal is to have each component meet four conditions:
      1. ***single-purpose***: performs one function
      2. ***small***: consists of an amount of information for which a human can readily grasp both structure and content
      3. ***simple***: is of a low degree of complexity so that a human can readily understand the purpose and structure of the module
      4. ***independent***: performs a task isolated from other modules

- Developmental Controls (Cont'd)
  - Modularization
    - Several advantages to having small, independent components.
      - *Maintenance*. If a component implements a single function, it can be replaced easily with a revised one if necessary.
      - *Understandability*
      - *Reuse*
      - *Correctness*
      - *Testing*.

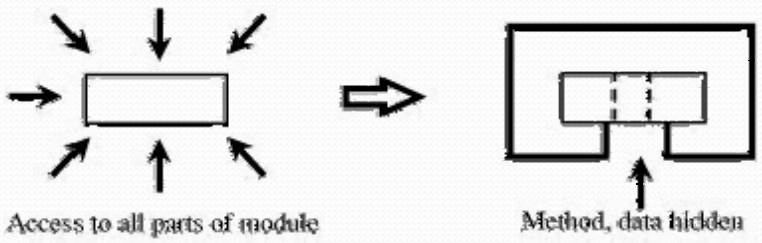
- Developmental Controls (Cont'd)
  - Modularization
    - A modular component usually has **high cohesion** and **low coupling**
    - Cohesion, we mean that all the elements of a component have a logical and functional reason for being there.
    - Coupling refers to the degree with which a component depends on other components in the system.



- Developmental Controls (Cont'd)
  - Encapsulation
    - Encapsulation hides a component's implementation details, but it does not necessarily mean complete isolation
    - Berard [BER00] notes that encapsulation is the "*technique for packaging the information [inside a component] in such a way as to hide what should be hidden and make visible what is intended to be visible.*"

- Developmental Controls (Cont'd)
  - Information Hiding
    - Think of a component as a kind of black box, with certain well-defined inputs and outputs and a well-defined function.
    - Other components' designers do not need to know *how* the module completes its function; it is enough to be assured that the component performs its task in some correct manner.
    - This concealment is the information hiding.
    - Information hiding is desirable because developers cannot easily and maliciously alter the components of others if they do not know how the components work.

- Developmental Controls (Cont'd)
  - Information Hiding (Cont'd)



- Developmental Controls (Cont'd)
  - Mutual Suspicion
    - Mutually suspicious programs operate as if other routines in the system were malicious or incorrect.
    - A calling program cannot trust its called subprocedures to be correct, and a called subprocedure cannot trust its calling program to be correct.
    - Each protects its interface data so that the other has only limited access.

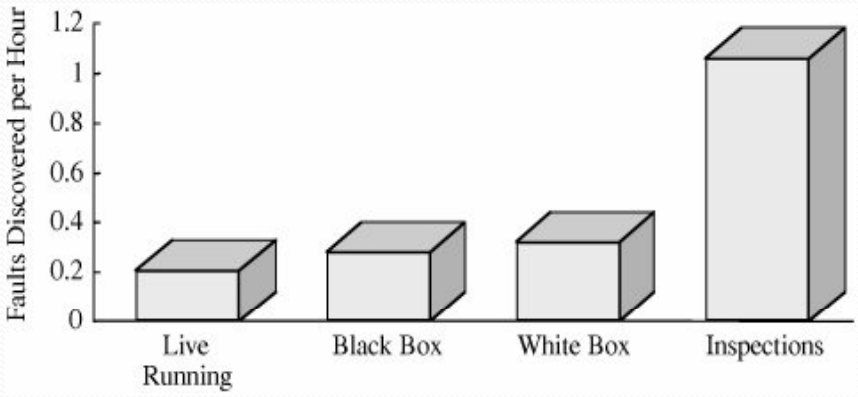
- Developmental Controls (Cont'd)
  - Confinement
    - A **confined** program is strictly limited in what system resources it can access. If a program is not trustworthy, the data it can access are strictly limited.
  - Genetic Diversity
    - Tight integration of products is a concern.
    - A vulnerability in one of these can also affect the others.
    - Fixing a vulnerability in one can have an impact on the others.

- Developmental Controls (Cont'd)
  - Pfleeger et al. [PFL01] recommend several key techniques for building what they call "solid software":
    - peer reviews
    - hazard analysis
    - testing
    - good design
    - prediction
    - static analysis
    - configuration management
    - analysis of mistakes

- Developmental Controls (Cont'd)
  - Peer review
    - *Review:* The artifact is presented informally to a team of reviewers; the goal is consensus and buy-in before development proceeds further.
    - *Walk-through:* The artifact is presented to the team by its creator, who leads and controls the discussion. Here, education is the goal, and the focus is on learning about a single document.
    - *Inspection*: The artifact is checked against a prepared list of concerns. The creator does not lead the discussion, and the fault identification and correction are often controlled by statistical measurements.

- Developmental Controls (Cont'd)
  - Peer review (Cont'd)
    - A wise engineer who finds a fault can deal with it in at least three ways:
      1. by learning how, when, and why errors occur
      2. by taking action to prevent mistakes
      3. by scrutinizing products to find the instances and effects of errors that were missed

- Developmental Controls (Cont'd)
  - Peer review (Cont'd)



Fault Discovery Rate Reported at Hewlett-Packard

- Developmental Controls (Cont'd)
  - Peer review (Cont'd)

Discovery Activity	Faults Found (Per Thousand Lines of Code)
Requirements review	2.5
Design review	5.0
Code inspection	10.0
Integration test	3.0
Acceptance test	2.0

- Developmental Controls (Cont'd)
  - Hazard analysis
    - A set of systematic techniques intended to expose potentially hazardous system states.
    - Usually involves developing hazard lists, as well as procedures for exploring "what if" scenarios to trigger consideration of nonobvious hazards.
    - A variety of techniques support the identification and management of potential hazards. Among the most effective are
      - Hazard and operability studies (HAZOP)
      - Failure modes and effects analysis (FMEA)
      - Fault tree analysis (FTA)

- Developmental Controls (Cont'd)
  - Hazard analysis (Cont'd)

	Known Cause	Unknown Cause
Known effect	Description of system behavior	Deductive analysis, including fault tree analysis
Unknown effect	Inductive analysis, including failure modes and effects analysis studies	Exploratory analysis, including hazard and operability

- Developmental Controls (Cont'd)
  - Testing
    - A process activity that homes in on product quality: making the product failure free or failure tolerant.

- Developmental Controls (Cont'd)
  - Testing (Cont'd)
    - Usually involves several stages.
      - **Unit testing** is done in a controlled environment whenever possible so that the test team can feed a predetermined set of data to the component being tested and observe what output actions and data are produced.
      - **Integration testing** is the process of verifying that the system components work together as described in the system and program design specifications.

- Developmental Controls (Cont'd)
  - Testing (Cont'd)
    - Usually involves several stages. (Cont'd)
      - A **function test** evaluates the system to determine whether the functions described by the requirements specification are actually performed by the integrated system.
      - A **performance test** compares the system with the remainder of these software and hardware requirements.
      - An **acceptance test**, in which the system is checked against the customer's requirements description.

- Developmental Controls (Cont'd)
  - Testing (Cont'd)
    - Usually involves several stages. (Cont'd)
      - A final **installation test** is run to make sure that the system still functions as it should.
      - After a change is made to enhance the system or fix a problem, **regression testing** ensures that all remaining functions are still working and that performance has not been degraded by the change.

- Developmental Controls (Cont'd)
  - Testing (Cont'd)
    - Each of the types of tests listed here can be performed from two perspectives
      - **Black-box testing** treats a system or its components as black boxes; testers cannot "see inside" the system
      - **Clear-box testing** (a.k.a. **white box**). - testers can examine the design and code directly, generating test cases based on the code's actual construction

- Developmental Controls (Cont'd)
  - Testing (Cont'd)
    - Olsen [OLS93] describes the development at Contel IPC of a system containing 184,000 lines of code and tracked faults discovered during various activities, and found differences:
      - 17.3 percent of the faults were found during inspections of the system design
      - 19.1 percent during component design inspection
      - 15.1 percent during code inspection
      - 29.4 percent during integration testing
      - 16.6 percent during system and regression testing
      - Only 0.1 percent of the faults were revealed after the system was placed in the field.

- Developmental Controls (Cont'd)
  - Testing (Cont'd)
    - From a security standpoint **independent testing** is desirable for the testing
    - **Penetration testing** is unique to computer security – the testers try to see if the software does what it is not supposed to do, which is to fail or fail to enforce security.

- Developmental Controls (Cont'd)
  - Good Design
    - Designers should try to anticipate faults and handle them in ways that minimize disruption and maximize safety and security.
      - **Passive fault detection** - construct the system so that it reacts in an acceptable way to a failure's occurrence.
      - **Active fault detection** - adopting a philosophy of mutual suspicion. Instead of assuming that data passed from other systems or components are correct, we always check that the data are within bounds and of the right type or format.
      - We can also use **redundancy**, comparing the results of two or more processes to see that they agree, before we use their result in a task.

- Developmental Controls (Cont'd)
  - Good Design
    - Fault tolerance: isolating the damage caused by the fault and minimizing disruption to users.
    - Typically, failures include
      - failing to provide a service
      - providing the wrong service or data
      - corrupting data

- Developmental Controls (Cont'd)
  - Good Design
    - We can build into the design a particular way of handling each problem
      1. *Retrying*: restoring the system to its previous state and performing the service again, using a different strategy
      2. *Correcting*: restoring the system to its previous state, correcting some system characteristic, and performing the service again, using the same strategy
      3. *Reporting*: restoring the system to its previous state, reporting the problem to an error-handling component, and not providing the service again

- Developmental Controls (Cont'd)
  - Static Analysis - examine its design and code to locate and repair security flaws before a system is up and running
  - several aspects of the design and code:
    - control flow structure - the sequence in which instructions are executed, including iterations and loops.
    - data flow structure - follows the trail of a data item as it is accessed and modified by the system
    - data structure - the way in which the data are organized, independent of the system itself.

- Developmental Controls (Cont'd)
  - **Configuration Management -** the process by which we control changes during development and maintenance
    - It is important to know who is making which changes to what and when:
      - *corrective changes*: maintaining control of the system's day-to-day functions
      - *adaptive changes*: maintaining control over system modifications
      - *perfective changes*: perfecting existing acceptable functions
      - *preventive changes*: preventing system performance from degrading to unacceptable levels

- Developmental Controls (Cont'd)
  - Configuration Management (Cont'd)
    - Four activities are involved in configuration management:
      - configuration identification
      - configuration control and change management
      - configuration auditing
      - status accounting

- Developmental Controls (Cont'd)
  - Configuration Management (Cont'd)
    - Configuration identification
      - Sets up baselines to which all other code will be compared after changes are made that is building and document an inventory of all components that comprise the system.
      - "Freeze" the baseline and carefully control what happens to it.
      - When a change is proposed and made, it is described in terms of how the baseline changes.

- Developmental Controls (Cont'd)
  - Configuration Management (Cont'd)
    - **Configuration control and configuration management** - ensure we can coordinate separate, related versions.
      - Three ways to control the changes
        - Separate files - have different files for each release or version.
        - Delta - designate a particular version as the main version of a system and then define other versions in terms of what is different.
        - Conditional compilation, whereby a single code component addresses all versions , relying on the compiler to determine which statements to apply to which versions.

- Developmental Controls (Cont'd)
  - Configuration Management (Cont'd)
    - A configuration audit confirms that the baseline is complete and accurate, that changes are recorded, that recorded changes are made, and that the actual software (that is, the software as used in the field) is reflected accurately in the documents
    - Finally, status accounting records information about the components: where they came from (for instance, purchased, reused, or written from scratch), the current version, the change history, and pending change requests.

- Developmental Controls (Cont'd)
  - Configuration Management (Cont'd)
    - All activities are performed by a configuration and change control board, or CCB.
    - The CCB contains representatives from all organizations with a vested interest in the system
    - The board reviews all proposed changes and approves changes based on need, design integrity, future plans for the software, cost, and more.

- Developmental Controls (Cont'd)
  - Proofs of Program Correctness
    - **Program verification** can demonstrate formally the "correctness" of certain specific programs.
      - Making initial assertions about the inputs and then checking to see if the desired output is generated.
      - Each program statement is translated into a logical description about its contribution to the logical flow of the program.
      - Finally, the terminal statement of the program is associated with the desired output.

- Developmental Controls (Cont'd)
  - Proofs of Program Correctness
    - Proving program correctness is hindered by several factors.
      - Correctness proofs depend on a programmer or logician to translate a program's statements into logical implications.
      - Deriving the correctness proof from the initial assertions and the implications of statements is difficult, and the logical engine to generate proofs runs slowly.
      - The current state of program verification is less well developed than code production.

Format String Vulnerability

```
printf ( user_input );
```

The above statement is quite common in C programs. In the lecture, we will find out what can go wrong if the program is running with privileges (e.g. `Set-UID` program).

1 Format String

- What is a format string?

```
printf ("The magic number is: %d\n", 1911);
```

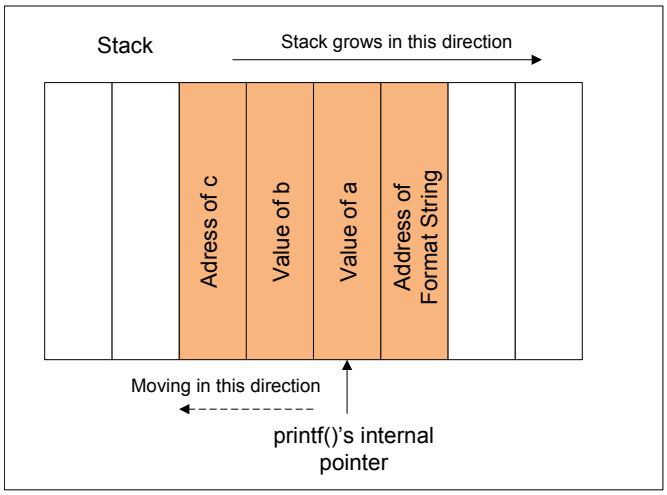
The text to be printed is "The magic number is:", followed by a format parameter '%d', which is replaced with the parameter (1911) in the output. Therefore the output looks like: The magic number is: 1911. In addition to %d, there are several other format parameters, each having different meaning. The following table summarizes these format parameters:

Parameter	Meaning	Passed as
%d	decimal (int)	value
%u	unsigned decimal (unsigned int)	value
%x	hexadecimal (unsigned int)	value
%s	string ((const) (unsigned) char *)	reference
%n	number of bytes written so far, (* int)	reference

- The stack and its role at format strings

The behavior of the format function is controlled by the format string. The function retrieves the parameters requested by the format string from the stack.

```
printf ("a has value %d, b has value %d, c is at address: %08x\n",
                            a, b, &c);
```



- What if there is a miss-match between the format string and the actual arguments?

```
printf ("a has value %d, b has value %d, c is at address: %08x\n",
                                a, b);
```

  - In the above example, the format string asks for 3 arguments, but the program actually provides only two (i.e. a and b).
  - Can this program pass the compiler?
    - The function `printf()` is defined as function with variable length of arguments. Therefore, by looking at the number of arguments, everything looks fine.
    - To find the miss-match, compilers needs to understand how `printf()` works and what the meaning of the format string is. However, compilers usually do not do this kind of analysis.
    - Sometimes, the format string is not a constant string, it is generated during the execution of the program. Therefore, there is no way for the compiler to find the miss-match in this case.
  - Can `printf()` detect the miss-match?
    - The function `printf()` fetches the arguments from the stack. If the format string needs 3 arguments, it will fetch 3 data items from the stack. Unless the stack is marked with a boundary, `printf()` does not know that it runs out of the arguments that are provided to it.
    - Since there is no such a marking. `printf()` will continue fetching data from the stack. In a miss-match case, it will fetch some data that do not belong to this function call.
  - What trouble can be caused by `printf()` when it starts to fetch data that is meant for it?

2 Attacks on Format String Vulnerability

- Crashing the program

```
printf ("%s%s%s%s%s%s%s%s%s%s%s%s");
```

  - For each `%s`, `printf()` will fetch a number from the stack, treat this number as an address, and print out the memory contents pointed by this address as a string, until a NULL character (i.e., number 0, not character 0) is encountered.
  - Since the number fetched by `printf()` might not be an address, the memory pointed by this number might not exist (i.e. no physical memory has been assigned to such an address), and the program will crash.
  - It is also possible that the number happens to be a good address, but the address space is protected (e.g. it is reserved for kernel memory). In this case, the program will also crash.

- Viewing the stack

```
printf ("%08x %08x %08x %08x %08x\n");
```

  - This instructs the printf-function to retrieve five parameters from the stack and display them as 8-digit padded hexadecimal numbers. So a possible output may look like:

```
40012980 080628c4 bffff7a4 00000005 08059c04
```

- Viewing memory at any location
  - We have to supply an address of the memory. However, we cannot change the code; we can only supply the format string.
  - If we use `printf(%s)` without specifying a memory address, the target address will be obtained from the stack anyway by the `printf()` function. The function maintains an initial stack pointer, so it knows the location of the parameters in the stack.
  - Observation: the format string is usually located on the stack. If we can encode the target address in the format string, the target address will be in the stack. In the following example, the format string is stored in a buffer, which is located on the stack.

```
int main(int argc, char *argv[])
{
  char user_input[100];
  ... ... /* other variable definitions and statements */

  scanf("%s", user_input); /* getting a string from user */
  printf(user_input); /* Vulnerable place */

  return 0;
}
```

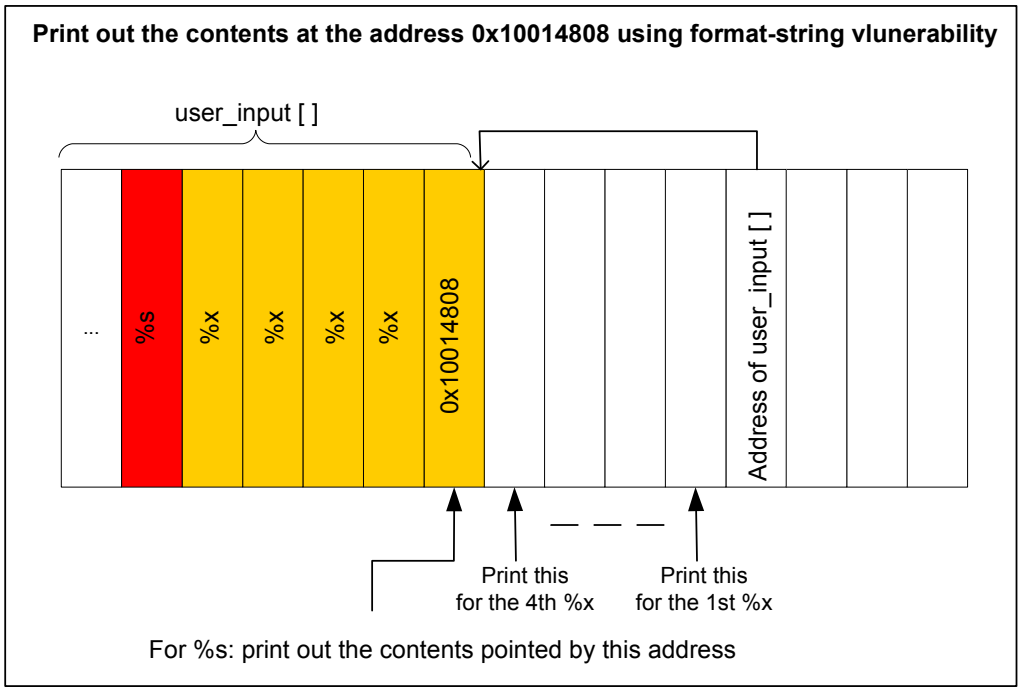
  - If we can force the printf to obtain the address from the format string (also on the stack), we can control the address.

```
printf ("\x10\x01\x48\x08  %x %x %x %x %s");
```

  - `\x10\x01\x48\x08` are the four bytes of the target address. In C language, `\x10` in a string tells the compiler to put a hexadecimal value `0x10` in the current position. The value will take up just one byte. Without using `\x`, if we directly put `"10"` in a string, the ASCII values of the characters `'1'` and `'0'` will be stored. Their ASCII values are 49 and 48, respectively.

- `%x` causes the stack pointer to move towards the format string.
- Here is how the attack works if `user_input[]` contains the following format string:

  `"\x10\x01\x48\x08  %x %x %x %x %s".`



- Basically, we use four `%x` to move the `printf()`'s pointer towards the address that we stored in the format string. Once we reach the destination, we will give `%s` to `print()`, causing it to print out the contents in the memory address `0x10014808`. The function `printf()` will treat the contents as a string, and print out the string until reaching the end of the string (i.e. 0).
- The stack space between `user_input[]` and the address passed to the `printf()` function is not for `printf()`. However, because of the format-string vulnerability in the program, `printf()` considers them as the arguments to match with the `%x` in the format string.
- The key challenge in this attack is to figure out the distance between the `user_input[]` and the address passed to the `printf()` function. This distance decides how many `%x` you need to insert into the format string, before giving `%s`.

- Writing an integer to nearly any location in the process memory
  - `%n`: The number of characters written so far is stored into the integer indicated by the corresponding argument.

```
int i;
printf ("12345%n", &i);
```

  - It causes `printf()` to write 5 into variable i.
  - Using the same approach as that for viewing memory at any location, we can cause `printf()` to write an integer into any location. Just replace the `%s` in the above example with `%n`, and the contents at the address `0x10014808` will be overwritten.

  - Using this attack, attackers can do the following:
    * Overwrite important program flags that control access privileges
    * Overwrite return addresses on the stack, function pointers, etc.
  - However, the value written is determined by the number of characters printed before the `%n` is reached. Is it really possible to write arbitrary integer values?
    * Use dummy output characters. To write a value of 1000, a simple padding of 1000 dummy characters would do.
    * To avoid long format strings, we can use a width specification of the format indicators.

- Countermeasures.
  - Address randomization: just like the countermeasures used to protect against buffer-overflow attacks, address randomization makes it difficult for the attackers to find out what address they want to read/write.

Attacks

Dola Das
Assistant Professor
Dept. of CSE, KUET

Salami Attack

- a technique by which Cyber-criminals steal money or resources a bit at a time
- no noticeable difference in overall size
- accumulates a considerable amount over a period of time
- the essence of this method is the failure to detect the misappropriation.

Salami Attack

- **Types of Salami attacks:**
  - **Salami Slicing:** when attacker/hacker deduct an awfully touch of cash from each account.
  - **Penny Shaving:** when the attackers/hacker steal money by using rounding to the closest within the transactions.
- **Prevention From Salami attack:**
  - by overseeing the weekly transactions and month-to-month bank statements
  - financial institutions, like banks, should also update their security so that the attacker doesn’t become conversant

Denial of Service (DoS) Attack

- A denial of service or DoS attack is used to tie up a website's resources so that users who need to access the site cannot do so.

- to force the shutdown of the targeted website, server or network.

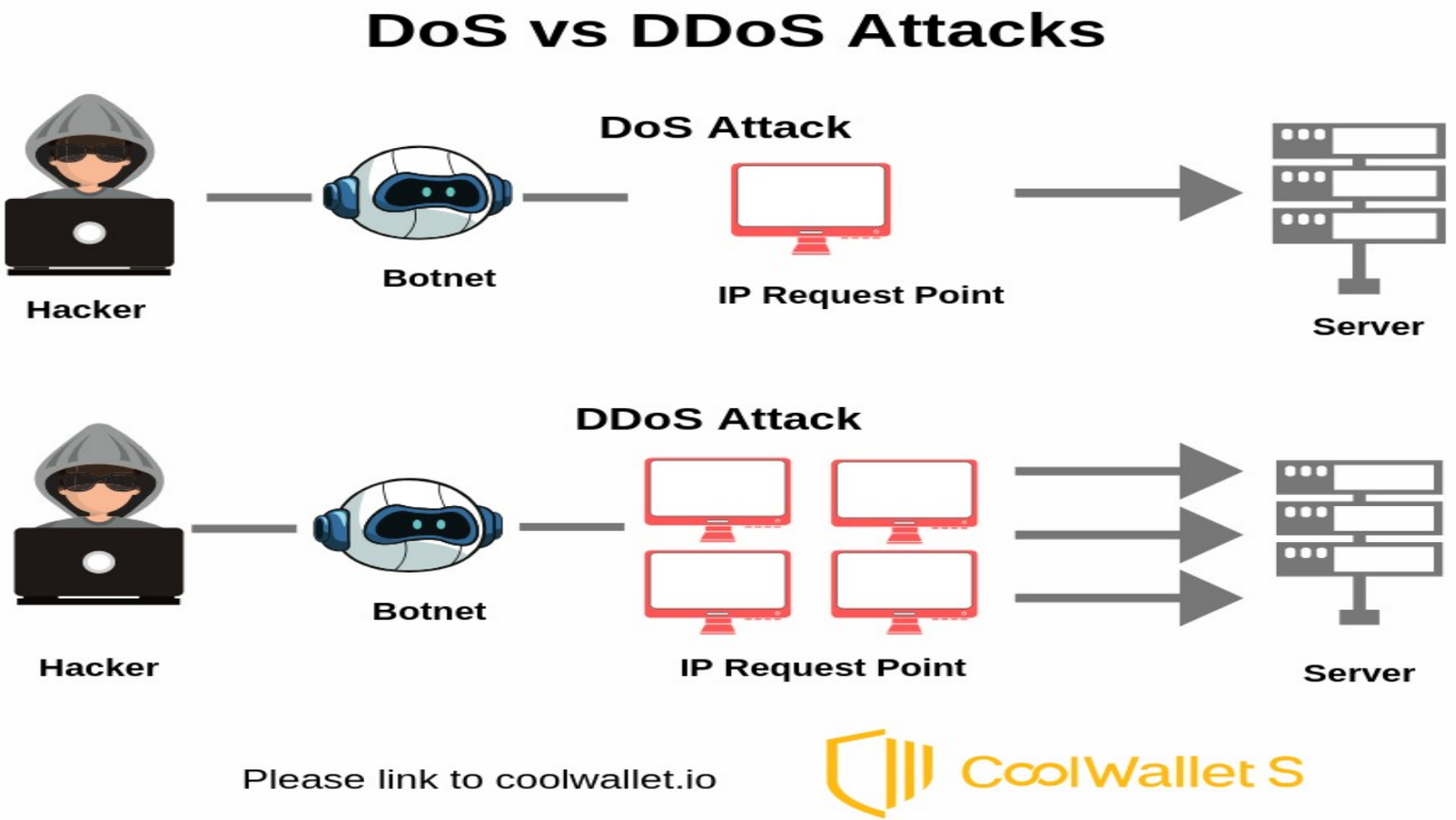
Denial of Service (DoS) Attack

- So, How do DoS attacks work?

Distributed Denial of Service (DDoS) Attack

- the next step in the evolution of DoS attacks
- use large numbers of compromised computers, as well as other electronic devices — such as webcams and smart televisions

DoS and DDoS


DoS vs DDoS Attacks
DoS Attack
Hacker
Botnet
IP Request Point
Server
DDoS Attack
Hacker
Botnet
IP Request Point
Server
Please link to coolwallet.io
CoolWallet S

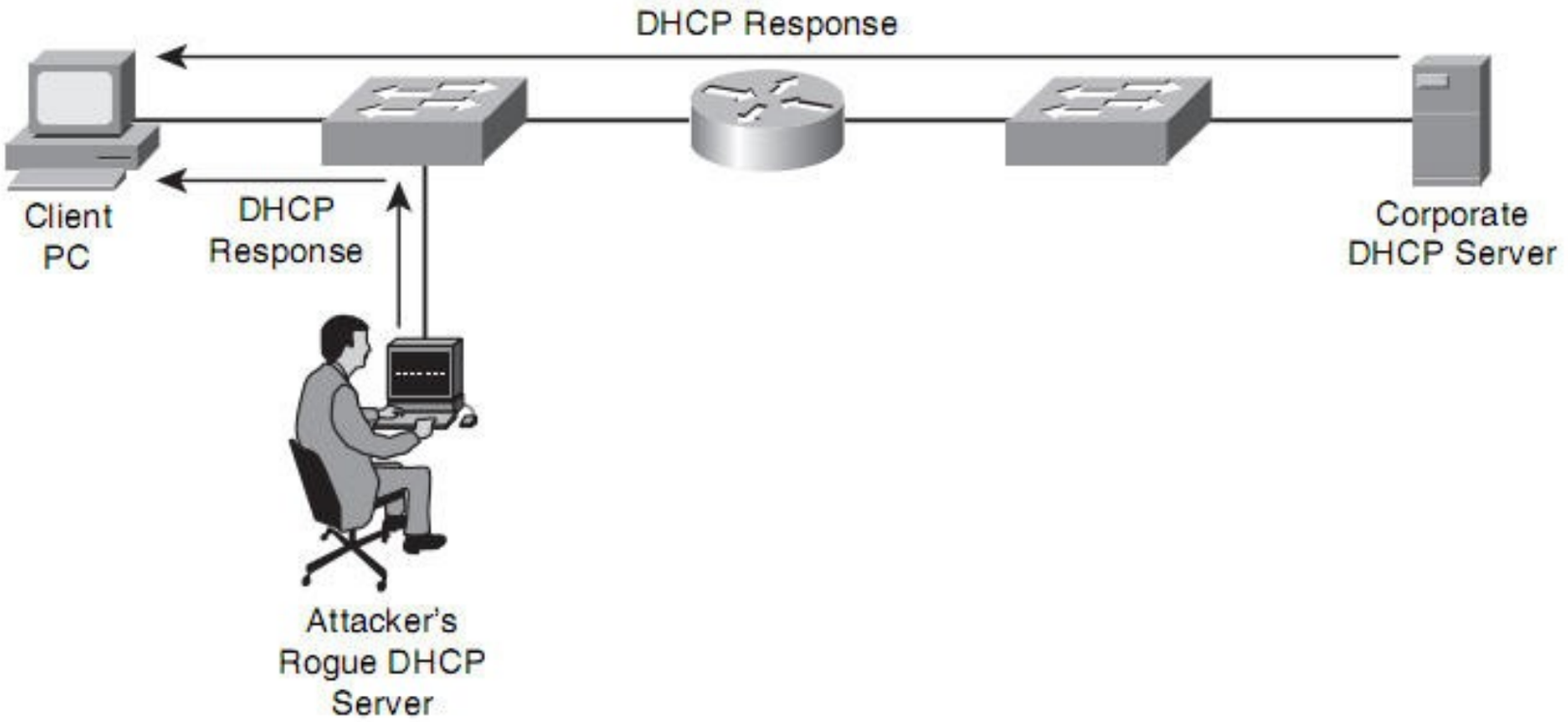
Preventions

- Get help recognizing attacks
- Contact your internet service provider
- Investigate black hole routing
- Configure firewalls and routers

DHCP Spoofing Attack

- DHCP spoofing is when a malicious actor sets up an alternate DHCP server on a network to provide false addressing and configuration information to clients.
- If the rogue server responds first and the offer is accepted, there are a few incorrect pieces of information that could have negative impacts on clients and the network as a whole.

DHCP Spoofing Attack



DHCP Spoofing Attack

- Impacts:
  - could create a DoS attack by giving clients the incorrect Default Gateway, IP Address, Subnet Mask, or DNS server
  - could create a Man-In-The-Middle attack by giving clients an alternate Default Gateway
  - if their communication isn’t encrypted, the alternate Default Gateway could intercept their messages and gain information from them
  - etc.

DHCP Spoofing Attack Prevention

- All access ports that connect to end-users should be marked as untrusted
- Once DHCP snooping is enabled, this is configured on all ports by default
  - DHCP Snooping is a layer 2 security technology incorporated into the operating system of a capable network switch that drops DHCP traffic determined to be unacceptable.
  - DHCP Snooping prevents unauthorized (rogue) DHCP servers offering IP addresses to DHCP clients.
- Any DHCP Offer messages will be blocked on untrusted ports, preventing them from acting as a DHCP server
- ports that connect to other network devices that are managed by the admin should be marked as trusted. These devices include routers, switches, and servers
- But if anyone can get physical access, can change configurations.

Dictionary Attack

- a systematic method of guessing a password by trying many common words

- Hackers use this attack to gain access to online accounts, but also for file decryption – and that’s an even bigger problem

How does a dictionary attack work?

- can be performed both online and offline
- Online
  - attacker repeatedly tries to log in or gain access
  - works better if the hacker has a list of likely passwords
  - If the attack takes too long, it might get noticed by a system administrator or the original user.
- Offline
  - no network limitations to how many times you can guess the password
  - To do it, hackers need to get their hands on the password storage file from the system they want to access, so it’s more complicated than an online attack
  - once they have the correct password, they will be able to log in without anyone noticing

Difference between a brute force attack and a dictionary attack

- Brute force
  - mostly rely on the computing power of the attacker's computer
  - a program also automatically enters combinations of letters, symbols, and numbers, but in this case, they are entirely random.
  - Brute force attacks can also be performed online and offline.
- Dictionary attack
  - 1,022,000 words in the English language and numbers 0-9, 218,340,105,584,896 eight-character passwords.
  - In this case, a dictionary attack is much more likely to succeed, given that the password will be a simple English word.
  - A basic brute force attack would take much more time and is less likely to be successful.
- Dictionary attacks are brute force attacks in nature. The only difference is that dictionary attacks are more efficient – they usually don't need to try as many combinations to succeed. However, if the password is a truly unique one, a dictionary attack won't work. In that case, using brute force is the only option.

How to avoid a password dictionary attack?

- use captchas,
- implement mandatory two-factor authentication,
- limit how many times one user can attempt to log in before their account is locked

Rainbow Table Attack

- a password cracking method that uses a special table (a “rainbow table”) to crack the password hashes in a database
- The rainbow table itself refers to a precomputed table that contains the password hash value for each plain text character used during the authentication process.
- If hackers gain access to the list of password hashes, they can crack all passwords very quickly with a rainbow table.

Rainbow Table Attack

- The prevalence of rainbow table attacks has dramatically decreased due to a technique known as “salting.”
- Salting is a modern technique used to thwart rainbow table attacks.
- It involves adding an extra random value to every hashed password to create a different hash value.

How a Rainbow Table Attack Works

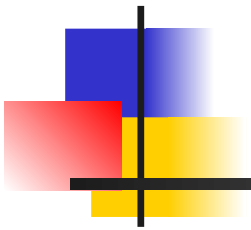
- Hackers must first gain access to leaked hashes in order to carry out rainbow table attacks.
- The password database itself might be poorly secured, or they may have gained access to the Active Directory.
- Others gain access through phishing techniques of those that might have access to the password database.
- On top of all these techniques, there are already millions and millions of leaked password hashes on the dark web that are available to hackers.

How to Protect Against a Rainbow Table Attack

- Eliminate passwords
- Use salting
- Use biometrics
- Monitor your servers.
- Don’t use outdated hashing algorithms.

Thank you!!

CS 5950/6030 – Computer Security and Information Assurance



Section 4: Protection in General-Purpose Operating Systems

Dr. Leszek Lilien
Department of Computer Science
Western Michigan University

Slides based on *Security in Computing. Third Edition* by Pfleeger and Pfleeger.
Using some slides courtesy of:
Prof. Aaron Striegel — course taught at U. of Notre Dame
Prof. Barbara Endicott-Popovsky and Prof. Deborah Frincke (U. Idaho) — taught at U. Washington
Prof. Jussipekka Leiwo — taught at Vrije Universiteit (Free U.), Amsterdam, The Netherlands

Slides not created by the above authors are © 2006 by Leszek T. Lilien
Requests to use original slides for non-profit purposes will be gladly granted upon a written request.

Protection in General-Purpose OSs – Outline (1)

4.1. Protected Objects, Methods, and Levels of Protection

a. History of protection in OSs
b. Protected objects in OSs
c. Security methods in OSs
d. Levels of protection in OSs
e. Three dimensions of protection in OSs
f. Granularity of *data* protection

4.2. Memory and Address Protection

a. Fence
b. Relocation
c. Base/Bounds Registers
d. Tagged Architecture
e. Segmentation
f. Paging
g. Combined Paging with Segmentation

© 2006 by Leszek T. Lilien

Protection in General-Purpose OSs – Outline (2)

4.3. Control of Access to General Objects

a. Introduction to access control for general objects
b. Directory-like mechanism for access control
c. Acces control lists
d. Access control matrices
e. Capabilities for access control
f. Procedure-oriented access control
g. Conclusions

4.4. File Protection Mechanisms

a. Basic forms of protection
b. Single file permissions
c. Per-object and per-user protection

© 2006 by Leszek T. Lilien

Protection in General-Purpose OSs – Outline (3)

4.5. User Authentication

a. Introduction
b. Use of passwords
c. Attacks on passwords
   i. Try *all* possible pwds (exhaustive, brute force attack)
   ii. Try many *probable* pwds
   iii. Try *likely* pwds
   iv. Search system list of pwds
   v. Exploiting indiscreet users (social engg)
d. Password selection criteria
e. One-time passwords (challenge-response systems)
f. The authentication process
g. Authentication other than passwords
h. Conclusions

4.6. Summary

© 2006 by Leszek T. Lilien

4. Protection in General-Purpose OSs

- This section:

  User's side of protection in general-purpose OS:

  - Functions that directly address security
  - Functions that have security as a byproduct

  [cf. B. Endicott-Popovsky and D. Frincke]

- Section 5 in the textbook (we will skip it):
  How OS design is affected by protection requirements

- Outline:

  4.1. Protected Objects, Methods, and Levels of Protection
  4.2. Memory and Address Protection
  4.3. Control of Access to General Objects
  4.4. File Protection Mechanisms
  4.5. User Authentication
  4.6. Summary

© 2006 by Leszek T. Lilien

4.1. Protected Objects, Methods, and Levels of Protection

- Outline
  a. History of protection in OSs
  b. Protected objects in OSs
  c. Security methods in OSs
  d. Levels of protection in OSs
  e. Three dimensions of protection in OSs
  f. Granularity of *data* protection

© 2006 by Leszek T. Lilien

a. History of protection in OSs (1)

- Predecessors of OSs:

1) No system s/w
   - User entered pgms in binary
     - Via switches or via keyboard
   - Single user had full control of computer
     - Scheduled time for exclusive computer use
     - Prepare before use
       - Load assembler, compiler, shared subroutines, etc.
     - Clean up after use

2) Executive
   - Assist *single* user with preparation and cleanup
   - Entirely passive:
     - Waited for user's request
     - Provided service on demand

© 2006 by Leszek T. Lilien

History of protection in OSs (2)

3) Monitor
- Assisted *multiple* users in *multiprogramming* systems
- Actively controled system resources
  - Provided service if consistent with system policies, denying otherwise
- Protect one user from interference (malicious or acceidental or malicious) by another

- Impact of multiprogramming on security:
  - Before multiprogramming - *no* need to protect one user from another
  - With multiprogramming - need to

© 2006 by Leszek T. Lilien

b. Protected objects in OSs

- Multiprogramming — sharing computer by multiple users
- Multiprogramming necessitates protecting OS objects:
  - Memory
  - I/O Devices
    - Sharable I/O devices (e.g., disks)
    - Serially reusable I/O devices (e.g., printers)
  - Sharable programs and subroutines
  - Networks
  - Sharable data
- Since OS controls system resources, OS must provide such protection
  - Many protection mechanism supported by hardware

© 2006 by Leszek T. Lilien

c. Security methods in OSs (1)

- Basis of security in OS: separation
  = keeping one user's objects secure from interference by other users
- Kinds of separation:
  1) Physical separation
     - Different processes use different physical objects
       - E.g., different printers for different 'confidentiality levels' of output
  2) Temporal separation
     - Processes having different security req's executed at different times
  3) Logical separation
     - Illusion that OS executes processes only for single user
  4) Cryptographic separation
     - Processes conceal their data and computations from other processes
  5) Combinations of the above

© 2006 by Leszek T. Lilien

Security methods in OSs (2)

- Strength of security via separation (least to most secure):
  - Logical separation
  - Temporal separation
  - Physical separation

- Complexity of implementation of separation (least to most complex):
  - Physical separation
  - Temporal separation
  - Logical separation
  - Cryptographic separation

- Resource utilization in different kinds of separation:
  - Poor: physical separation / temporal separation
  - Good: logical separation / cryptographic separation

© 2006 by Leszek T. Lilien

d. Levels of protection in OSs (1)

- Absolute separation reduces efficiency
  – need to share some resources for efficiency
- Full sharing-separation spectrum = levels of protection by OS:
  1) No protection
     - *Caveat emptor* („Let the buyer beware" in Latin)
     - User can still protect self by, e.g, temporal separation
  2) Isolation
     - Concurrently running processes hidden from each other => unaware of each other
       - Own address space, files, other objects for each process
  3) Full sharing or no sharing
     - Object/resource owner declares it as:
       - public (can be shared by all)

     or

       - private (not shared)

...cont...

© 2006 by Leszek T. Lilien

Levels of protection in OSs (2)

...cont...

4) Sharing via access limitation

- Access to each object by each user determined by access rights

5) Sharing by *capabilities*

- Extension to „ Sharing via access limitation”

  — dynamic access rights

- Can be changed by owner, subject, computation context, object itself

6) Limited object use

- Limits not only object *access* — limit object *use*
  - E.g., can view a doc *but* can’t copy it
  - E.g., can view statistical summary of data *but* can’t view individual data records (e.g., can see average salary but not John Smith’s salary)

© 2006 by Leszek T. Lilien

Levels of protection in OSs (3)

- OS can provide different levels of protection for different objects/resources

- Complexity of implementation and *fineness* of protection:
  1) No protection
  2) Isolation
  3) Full sharing or no sharing
  4) Sharing via access limitation
  5) Sharing by capabilities
  6) Limited object use

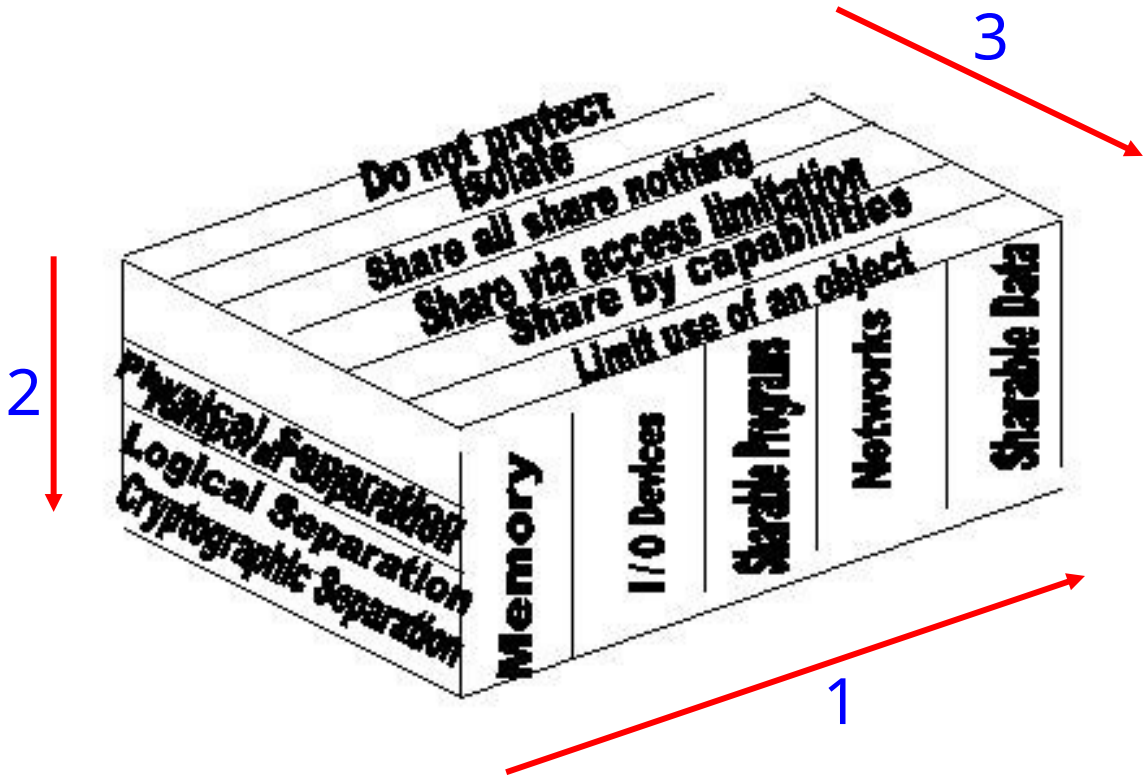


Complexity of implementation and Fineness of protection

© 2006 by Leszek T. Lilien

e. Three dimensions of protection in OSs

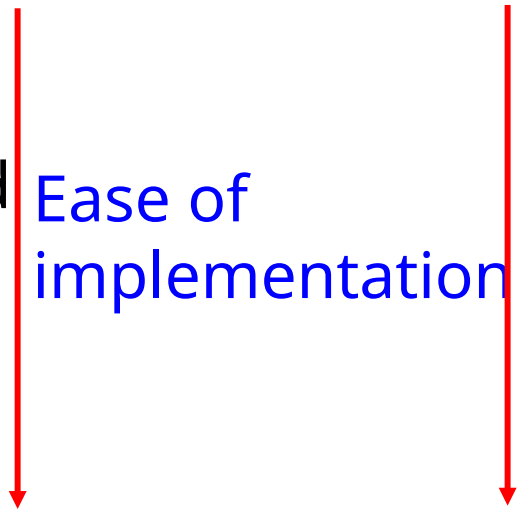
Dimensions:
1—protected objects
2—security methods
3—protection levels



[cf. B. Endicott-Popovsky and D. Frincke]

f. Granularity of *data* protection

- Granularity of data protection
  - Aplicable only to data
  - Protect by:
    - Bit
    - Byte
    - Element/word
    - Field
    - Record
    - File
    - Volume



(*) If no control at proper granularity level, OS must grant access to more data than necessary

E.g., if no field-level data control, user can be given whole record

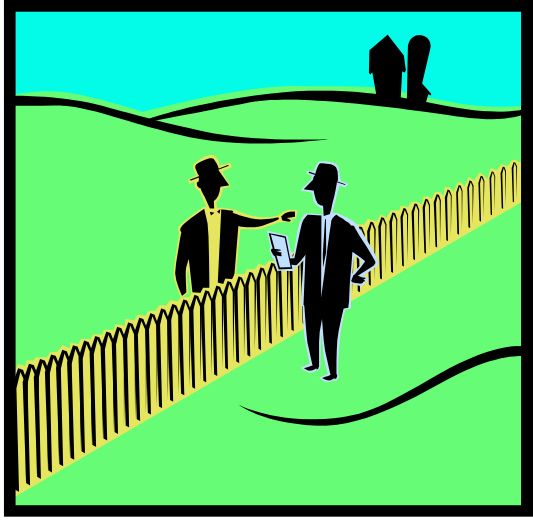
4.2. Memory and Address Protection (1)

- Most obvious protection:
  Protect pgm *memory* from being affected by other pgms

- Outline
  a. Fence
  b. Relocation
  c. Base/Bounds Registers
  d. Tagged Architecture
  e. Segmentation
  f. Paging
  g. Combined Paging with Segmentation

© 2006 by Leszek T. Lilien

Memory and Address Protection (2)



a. Fence

- Confining users to one side of a boundary
- E.g., predefined memory address n between OS and user

  User pgm instruction at address ≤ n (OS's side of the fence) not allowed to execute
- *Fixed* fence (cf. Fig. 4-1, p. 184)

  (wastes space if unusued by OS or blocks IOS from growing)

or

Variable fence (cf. Fig. 4-2, p. 185)

- Using *fence register* — h/w register

[cf. B. Endicott-Popovsky and D. Frincke]

b. Relocation

- Pgms written as if starting at location 0 in memory
- Actually, starting at location n — determined by OS
- Before user instruction executed, each address *relocated* by adding *relocation factor* n to it
  - Relocation factor = starting address of pgm in memory
- Fence register (h/w register) plays role of relocation register as well
  - Bec. adding n to pgm addresses prevents it from accessing addresses below n

© 2006 by Leszek T. Lilien

c. Base/Bounds Registers (cf. Fig. 4-3, p. 186)

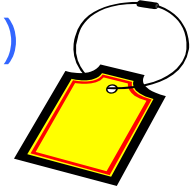
- Base register = variable fence register
  - Determines starting address, i.e. *lower limit*, for user pgm addresses
- Bounds register
  - Determines *upper limit* for user pgm addresses
- Each pgm address *forced* to be above base address
  - Bec. base reg contents added to it

  & each pgm address *checked* to be below bounds address

- To protect user's instructions from *user's own* data address errors – use *two pairs* of registers: (cf. Fig. 4-4, p. 187)

  1) Register pair for data
  2) Register pair for for instructions

© 2006 by Leszek T. Lilien



d. Tagged Architecture

- Problem with base/bounds registers:
  high granularity of access rights (ARs)
  - Can allow another module to access *all* or *none of* its data
    - „All or none" data within limits of data base-bounds registers

- Solution: tagged architecture (gives *low* granularity of access rights)
  - Every word of *ma* :1 *tag bits* defining access rights to th on!)
    Access bits set by OS
  - Tested every time instruction accesses its location

(# of bits ~ # of different ARs)

Tag	Word
R	0001
RW	0137
R	4091
R	0002
X	

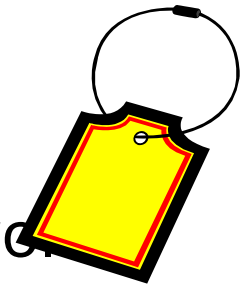
R = Read only

RW = Read/Write

X = Execute only

[cf. B. Endicott-Popovsky and D. Frincke]

Memory and Address Protection (6)



- Benefit of tagged architecture:
  Low (good!) granularity of memory access control
  – at memory word level

- Problems with tagged architecture:
  - Requires special h/w
    - Incompatible with code of most OSs
      - OS compatible with it must:
        - Accommodate tags in each memory word
        - Test each memory word accessed
      - Rewriting OS would be costly
    - Higher memory costs (extra bits per word)
  - More modern solutions available (below)

© 2006 by Leszek T. Lilien

e. Segmentation

- Benefits addressing + enhances memory protection for free
- Effect of an unbounded number of base/bounds registers
- Pgm segmentation:
  - Program divided into logical pieces (called segments)
    - E.g. Pieces are: code for single procedure / data of an array / collection of local data values
  - Consecutive pgm segments can be easily stored in nonconsecutive memory locations (cf. Fig. 4-6 p.190)

© 2006 by Leszek T. Lilien

Memory and Address Protection (8)

- Addressing w/ segmentation
  - Data item D addressed as:

    (segment_name_of_D, offset_of_D_within_segment)

    Instructions addressed analogously
  - For each process, OS keeps a separate

    Segment Translation Table (STT)

    Rows in STT: (segment_name, segment_offset)

    segment_name – name of segment containg data item

    segment_offset – starting location for named segment
  - OS translates each data or instruction address using STT
    - Cf. Fig. 4-7 p. 191
- Two processes can share a segment S by having the same segment_name and segment_offset value in their STTs

© 2006 by Leszek T. Lilien

Memory and Address Protection (9)

- Security-related benefits of segmentation
  - Strong segment protection
    - Bec.: STT under exclusive OS control
      - each address requires STT access to get segment_offset for segment S
      - OS checks that address translates into S's memory space (not beyond its end)
  - Different protection levels for different segments (approximates tagging at higher granularity)
    - E.g. segments with: R-only data / X-only code / W data
  - Different protection levels for different processes accessing the *same* segment

© 2006 by Leszek T. Lilien

Memory and Address Protection (10)

- Problems w/ segmentation
  - Programmer must be aware of segmentation
  - Efficiency
    - OS lookup of STT is slow
    - Symbolic segment names difficult to encode in pgm instructions
  - Fragmentation of main memory (by variable-sized holes left after „old” segments)

© 2006 by Leszek T. Lilien

f. Paging

- Principles:
  - Programs divided into equal-sized *pages*
    Memory divided into same-sized *page frames*
    - Size is usually 2^n, from 512 B to 4096 B
  - Address format for item (data or instruction) I:
    (page_nr_of_I, offset_of_I_within_page)
  - OS maintains *Page Translation Table* (PTT)
    — maps pages into page frames
- Address translation similar as for segmentation (cf. Fig. 4-8)
  - But *guaranteed* that offset falls within page limit
    - E.g., for page size of 1024 = 2^{10},
      10 bits are allocated for page_offset

© 2006 by Leszek T. Lilien

Memory and Address Protection (12)

- Benefits of paging
  - Programmer can be oblivious to page boundaries (automatic)
    - Paging completely hidden from programmer
  - No fragmentation of main memory
- Problem w/ paging
  - Can't associate access rights with pages
    - Pages are *random collections of items* that require different protection level in general
    - Pages are *not 'access rights' units* (*logical units*) to be protected at the same level

© 2006 by Leszek T. Lilien

g. Combined paging with segmentation

- Principle:
  - Paging offers efficiency
    - Hiding from programmer
    - No fragmentation
  - Segmentation offers 'logical protection'
    - Grouping items w/ similar protection needs within the same segment
- Paged segmentation: (cf. Fig. 4-9, p. 195)
    - Programmer defines segments
    - Segments broken into pages automatically
- Benefits of paging and segmentation

  but extra layer of address translation
  - Additional h/w deals with this overhead

© 2006 by Leszek T. Lilien

4.3. Control of Access to General Objects

- Outline
  a. Introduction to access control for general objects
  b. Directory-like mechanism for access control
  c. Access control lists
  d. Access control matrices
  e. Capabilities for access control
  f. Procedure-oriented access control
  g. Conclusions

© 2006 by Leszek T. Lilien

a. Introduction to access control for general objects (1)

- *Objects* and *subjects* accessing them
  - *General objects* in OS that need protection (examples)
    - Memory / File or data set on auxiliary storage device
    - Pgm executing in memory / Directory of files / Hardware device
    - Data structure / OS tables / Instructions, esp. privileged instructions
    - Passwords and authentication mechanism / Protection mechanism
  - *Subjects*
    - User / Administrator / Programmer / Pgm
    - Another object / Anything that seeks to use object

© 2006 by Leszek T. Lilien

Introduction to access control for general objects (2)

- Complementary *goals* in access control:
  1) Check *every* access
     - Access is not granted forever—can be suspended or revoked
  2) Enforce least privilege
     - Give subject access to the smallest number of objects necessary to perform subject's task
  3) Verify acceptable use
     - E.g., verify if requested *kind* of access is acceptable
       - E.g., R is OK, W/X is not

© 2006 by Leszek T. Lilien

Introduction to access control for general objects (3)

- *Complexity of access control* depends on:

  1) Object homogeneity
     - Homogeneous memory objects vs. heterogeneous h/w device

  2) Number of points of access
     - Access aways via memory manager vs. access via different device drivers

  3) Existence of central access authority
     - Central memory manager vs. different device drivers

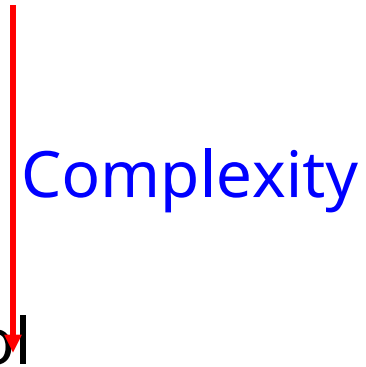
  4) Kind of access
     - R/W/X vs. big set of possible kinds of access

- In general:
  Acces control for more uniform objects with fewer kinds of access is simpler (e.g., simpler for memory than h/w devices)

© 2006 by Leszek T. Lilien

Introduction to access control for general objects (4)

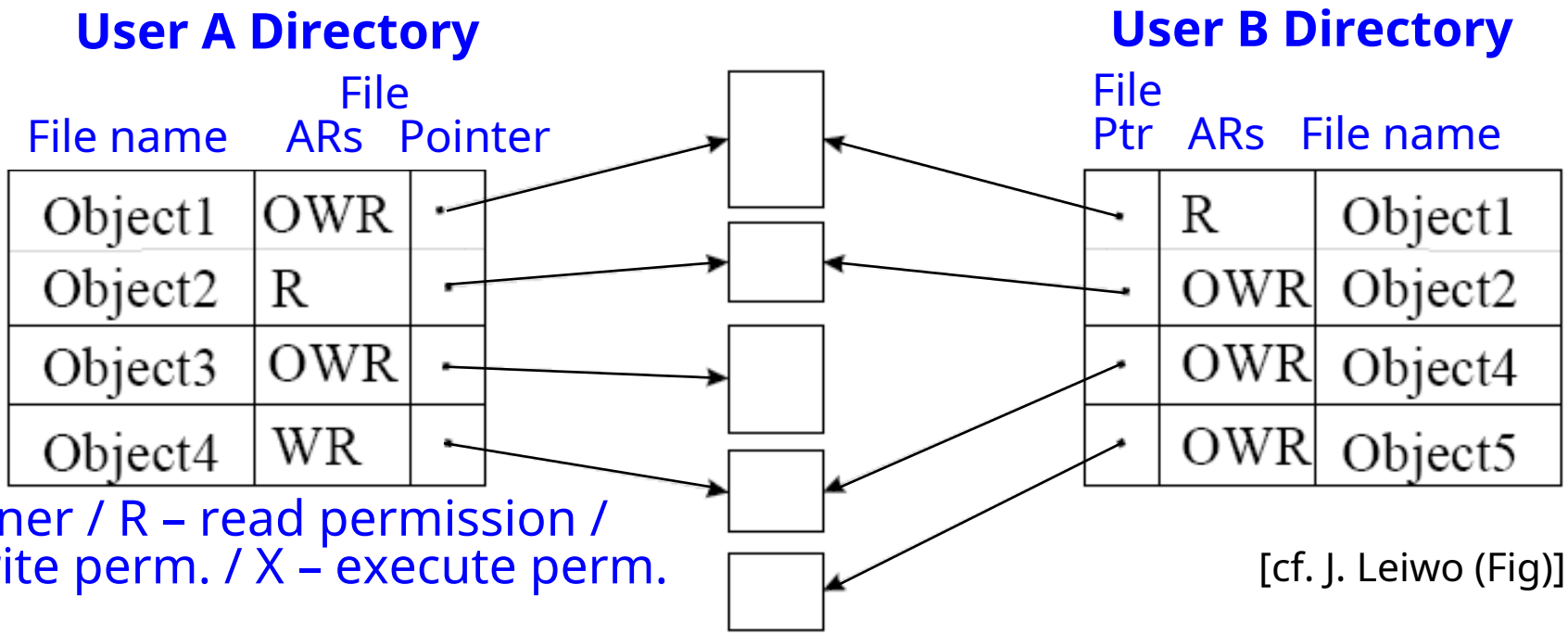
- Growing complexity of access control *mechanisms*
  - Directory
  - Access Control List
  - Access Control Matrix
  - Capability
  - Procedure-Oriented Access Control



[cf. B. Endicott-Popovsky and D. Frincke]

b. Directory-like mechanism for AC (1)

- File directory mechanism to control file access
  - Unique object *owner*
    - Owner controls *access rights*: *assigns*/*revokes* them
      - Access rights (ARs): Read, write, execute (possible others)
  - Each user has access rights directory
- Example: (User A owns O1 and O3. User B owns O2, O4, O5)



O - owner / R – read permission /
W – write perm. / X – execute perm.

[cf. J. Leiwo (Fig)]

© 2006 by Leszek T. Lilien

Directory-like mechanism for access control (2)

- Directory-like mechanism to control access to general objects
  - Analogous to file directory mechanism
- Advantage: Easy to implement
  - Just one list (directory) per user
- Difficulties
  - All user directories get too big for large # of *shared* objects — bec. each shared object in dir. of each user sharing it
  - Maintenance difficulties:
    - Deletion of shared objects
      - Requires deleting entry from each directory referencing it
    - Revocation of access
      - If owner A revokes access rights for X from every subject, OS must search dir's of all subjects to remove entries for X
    - Pseudonyms
      - An example in textbook (p. 197, Fi

[cf. B. Endicott-Popovsky and D. Frincke]

c. Access control lists (1)

- Access control list
  - A list attached to an *object*
  - Specifying ARs for each *subject* (who accesses this object)
    - For some subjects specified individually, for others — via being member of a group

Note: This „reverses" directory approach where:
- lists are attached to a *subject*
- specifying ARs for each *object* (accessed by this subject)

- Example 1 [cf. J. Leiwo]
  - Subjects: A, B, C, D, E
  - Use of wild card (*) for 'any' (any subject other than B can R/W Object 4)

```
Object1: {{A: OWR}, {B: R}, {C: R}, {D: R}}
Object2: {{A: R}, {B: OWR}, {C: R}, {E: R}}
Object3: {{A: OWR}}
Object4: {{B: OWR}, {*: WR}}
Object5: {{B: OWR}, {E: R}}
```


© 2006 by Leszek T. Lilien

Access control lists (2)

- Significant advantages over directory approach
  - Can have *default ARs* for subjects w/o specific ARs
- Example 2: Unix approach
  - File ARs for: user (owner) / (owner's) group / others (default)
    - E.g.: drwxr-xr-x 34 jones faculty 1476 Oct 17 08:26 secClass
- Example 3: Multics OS approach (textbook – p. 199)
  - user / group / compartment
    - user – ARs for individual subject
    - group – ARs for a group of subjects (e.g., for all project members)
    - compartment – confines untrusted objects or collects related objects (see text)
  - Use of wild cards: any user / any group / any comp't
    - Object1: { {Sanjay—Web_Proj—Midwest: X} }
      Object2: { {Sanjay—*—*: RW}, {*—*—*: R} }
    - Meaning: Only Sanjay can execute O1 within the 'Midwest' compartment when working on the 'Web' project.
      Only Sanjay can write O2, but everybody can read it.

© 2006 by Leszek T. Lilien

d. Access control matrices

- Previous access control mechanism used *lists*
  - Directory – subject's *list* of ARs for objects acessible by the subject
  - Access list – object's *list* of ARs for subjects that can access the object
- Access Control Matrix
  - A sparse matrix (a table)
    - Rows — subjects / columns — objects
    - Cell (i, j) — subjects i's ARs for access to object j

	Object 1	Object 2	Object 3	Object 4	Object 5
Subject A	OWR	R	OWR	WR	-
Subject B	R	OWR	-	OWR	OWR
Subject C	R	R	-	WR	-
Subject D	R	-	-	WR	-
Subject E	-	R	-	WR	R

[Fig. - cf. J. Leiwo]

© 2006 by Leszek T. Lilien

e. Capabilities for access control (1)

- Capability mechanism
  - Subjects access objects only via *capabilities*
  - *Capability* — a kind of token/ticket/pass giving to subject certain ARs for an object
    - To see (kind of access) a movie (object), a moviegoers (subject) must have a ticket (*capability*)
  - Capability to transfer ARs — allows subject to pass copies of its capabilities to other subjects
    - S1 can copy its capability to access O1 and transfer it to S2
    - If S1 omits 'transfer' rights for O1 in capability passed to S2, S2 can't transfer these rights to any other subject
  - Capability is limited by its domain (= local name space)
    - Not all cap's passed from caller domain to subroutine domain
    - Subr. can have cap's that its calling pgm doesn't

© 2006 by Leszek T. Lilien

Capabilities for access control (2)

- Capabilities help OS keep track of ARs during execution
  - Backed up by more detailed table (e.g. acc. ctrl matrix)
  - Capabilities for objects accessed by current process are kept readily available (for speed)

- Protecting capabilities
  - Capabilities in memory are accessible to OS only
    - E.g., stored in protected memory
  - Capability are unforgeable - two basic ways:

    1) Only OS holds and writes capabilities
       OS issues to subjects only pointers to capabilities

    2) Capability is encrypted
       Key known only to OS's access control mechanism

  - Problem: *Capability revokation* can be complicated
    - When capability revoked by its issuing subject, OS must find & stop corresponding accesses

© 2006 by Leszek T. Lilien

f. Procedure-oriented access control

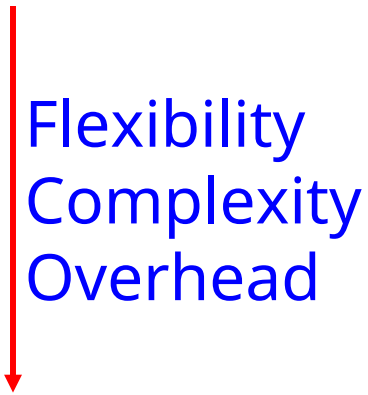
- Need to control actions that subject can do on object
  - More actions than just R or W or X

  => procedure-oriented acces control
- Procedure-oriented access control mechanism:
  - Procedure *encapsulates* object
    - Controls accesses to object
    - Provides trusted interface to object
    - Implements information hiding
- Example: P-OAC to perform additional user authentication
- Use of P-OAC results in efficiency penalty

© 2006 by Leszek T. Lilien

g. Conclusions

- Growing flexibility — but also complexity and overhead
  - Directory-like mechanism
  - Access control lists
  - Access control matrices
  - Capabilities for access control
  - Procedure-oriented access control



© 2006 by Leszek T. Lilien

4.4. File Protection Mechanisms

- Previous section: general object protection
  Now: file protection *examples* (more file protections exist)
  — as examples of object-specific protection

- Outline
  a. Basic forms of protection
  b. Single file permissions
  c. Per-object and per-user protection

© 2006 by Leszek T. Lilien

a. Basic forms of protection (1)

- Basic forms of protection
  1) All-none protection
  2) Group protection

1) All-none protection (in early IBM OS)

- Public files (all) or files protec'd by passwords (none)
  - Access to public files required knowing their names
  - Ignorance (not knowing file name) was an extra barrier
- Problems w/ this approach
  - Lack of trust for public files in large systems
    - Difficult to limit access to trusted users only
  - Complexity – for password-protected files, human response (password) required for each file access
  - File names easy to find
    - File listings eliminate ignorance barrier

© 2006 by Leszek T. Lilien

2) Group protection

- Groups w/ common relationship:
  - I.e., group – if has need to share sth
  - User belongs to one group
    - Otherwise can leak info objects groups
- Example — In Unix: user, (trusted) group, others
  - E.g., u+r+w+x,g+r+w-x,o+r-w-x
- Advantage: Ease of implementation
  - OS recognizes user by user ID and group ID (upon login)
  - File directory stores for each file:

    File owner's *user ID* and file owner's *group ID*

© 2006 by Leszek T. Lilien

Basic forms of protection (3)

- Problems w/ group protection

  a) User can't belong to > 1 group

  Solution: Single user gets multiple accounts
  - E.g., Tom gets accounts Tom1 and Tom2
  - Tom1 in Group1, Tom2 in Group2
  - Problem: Files owned by Tom1 can't be accessed by Tom2 (unless they are public – available to 'others')

  Problems: account proliferation, inconvenience, redundancy (e.g., if admin copies Tom1 files to Tom2 acct)

  b) User might become responsible for file sharing

  E.g., admin makes files from all groups visible to a user (e.g., by copying them into one of user's accts and making them private user's files)

  => User becomes responsible for 'manually' preventing unauthorized sharing of his files between his different 'groups'

  c) Limited file sharing choices

  Only 3 choices for any file: private, group, public

© 2006 by Leszek T. Lilien

b. Single file permissions (1)

- Single permissions – associating permission with single file
- Types of single file permissions:
  1) Password or other token
  2) Temporary acquired permission

1) Password or other token
   - Provide a password for each file
     - File pwd for W only
     - File pwd for any access
   - Finer degree of protection
     - Like having a different group for each file
       - file X group = all those who know file X pwd

© 2006 by Leszek T. Lilien

Single permissions (2)

- Problems with file pwds
  - Loss of pwd
    - Requires admin unprotecting file, then assigning new
    - Requires notifying all legitimate users
  - Using them inconvenient, takes time
  - Pwd disclosure allows unauthorized file accesses
    - Change of pwd requires notifying all legitimate users
  - Revocation of (just) a single user requires pwd change
    - Then, must notify all legitimate users

© 2006 by Leszek T. Lilien

Single permissions (3)

2) Temporary acquired permission

- Used in UNIX – the approach:
  - Based on user-group-others access hierarchy
  - Permission called *set userid* (suid)
    - If „user" (owner) of executable file X sets *suid* for X for his group, any group member executing X has „user" access rights (ARs) for X
      - Rather than having just „regular" group ARs for X
- Allows users to share data files
  - Access only via procedures that access them
    - Procedures encapsulate files
  - E.g., convenient for OS pwd file
    - Pwd change pgm with suid - any user can access own pwd record
    - OS owns this pgm (only OS, as „user" can access whole pwd file)

© 2006 by Leszek T. Lilien

c. Per-object and per-user protection

- Per-object and per-user protection
  - Approach:
    - File owner specifies access rights (ARs) for *each file* he owns for *each user*
    - Can implement with ACL (access control list) or ACM (access ctrl matrix)
  - Advantages:
    - Fine granularity of file access
    - Allows to create groups of users with similar ARs
  - Problem: Complex to create and maintain groups
    - File owner's overhead to specify ARs for *each file* for *each user* he owns

© 2006 by Leszek T. Lilien

4.5. User Authentication

- Outline
  a. Introduction
  b. Use of passwords
  c. Attacks on passwords
  d. Password selection criteria
  e. One-time passwords (challenge-response systems)
  f. The authentication process
  g. Authentication other than passwords
  h. Conclusions

© 2006 by Leszek T. Lilien

a. Introduction (1)

- Identification and Authentication (I&A) in Daily Life
  - Using library services
    - Librarian asks for student's name – *identification*
      - To learn who you are
    - Librarian asks for a proof of identity – *authentication*
      - To prove that you are who you say you are
        - E.g., show a picture ID
    - Once you are identified and authenticated, you can use library services (borrow books, use computers, etc.)

© 2006 by Leszek T. Lilien

Introduction (2)

- I&A in Cyberspace
  - Using computer services
    - Dialog box asks for student's username (login name) – *identification*
      - To learn who you are
    - Dialog box asks for a password – *authentication*
      - To prove that you are who you say you are
    - Once you are identified and authenticated, you can use computer services (access files, dial up, surf the 'net, etc.)

© 2006 by Leszek T. Lilien

Introduction (3)

- Basic Definitions
  - *Principal:* a unique entity (a person named Robert Kowalski)
  - *Identity:* specifies a principal ("Robert Kowalski")
  - *Identification:* obtaining identity from the principal (getting username "rkowals3" – 8 characters)
  - *Authentication:* ensuring that principal matches the purported identity (a person named Robert Kowalski matches the "Robert Kowalski" identity)
  - Note:

    The same principal may have many different identities.

    E.g., a working student might have 2 identities for 2 roles:
    - Computer consultant
    - Student

    Still, each of these identities specifies the same principal.

© 2006 by Leszek T. Lilien

- Identification Problems
  - In using library services
    - Librarian asks for student's name
    - What if there are two students named Joan Smith?
  - Librarian must find a unique identification
    - Can ask for a home phone number, address, etc.
  - Computer resolves "shared" names as follows:
    - In a *closed* system (e.g. campus system):

      each user has a unique *pre-registered* username
    - In an *open* system (e.g. a Web service with user registration):

      each user tries to create a unique username

      many attempts allowed until unique username found

© 2006 by Leszek T. Lilien

- Authentication Problems
  - In using library services
    - Librarian asks for a proof of identity
      - Student ID card proves identity
    - What if the ID expired?
  - Librarian must authenticate the student
    - Can ask for a driver's license and a Registrar's receipt
  - Computer must authenticate principal
    - Correct and current password
    - If invalid after *n* attempts, computer denies access to its resources
    - If expired, computer tells principal to get a new pwd

© 2006 by Leszek T. Lilien

Introduction (6)

- I&A is very important — basis for system to define user's access rights
- I&A can be based on:
  1. What entity *knows* – passwords
     - E.g., simple password, challenge-response authentication
  2. What entity *is* – biometrics
     - E.g., fingerprints, retinal characteristics
  3. What entity *has* - access tokens
     - E.g., badges, smart cards
  4. *Where* entity is – location
     - E.g., in the accounting department
  5. Any combinations of the above - hybrid approaches

© 2006 by Leszek T. Lilien

- Types of Passwords
  1) Sequence of characters
     - Examples:
       - 10 digits, a string of characters, *etc*.
     - Generated:
       - Randomly – often the very first password supplied by sysadmin
       - By user – most popular
       - By computer with user input
  2) Sequence of words
     - Examples: pass-phrases (complex sentences)
  3) Challenge-response authentication
     - Examples: one-time passwords (discussed below), pass algorithms

© 2006 by Leszek T. Lilien

b. Use of passwords (1)

- Password – most common *authentication* mechanism
  - Relatively secure
  - Endangered by human negligence
    - Too short pwd, not changed for a long time, etc.
  - Selected by system or user

- Loose-lipped I&A
  - Disclose more info than necessary before successful logging
    - Example – textbook p.211
  - Good I&A – user given no info until logging successul
    - Example – textbook p.212

© 2006 by Leszek T. Lilien

Use of passwords (2)

- Additional authentication information
  - E.g., principal can access only:
    - From specific location
    - At specific times
    - From specific location at specific times

© 2006 by Leszek T. Lilien

c. Attacks on passwords

- Kinds of password attacks
  i. Try *all* possible pwds (exhaustive, brute force attack)
  ii. Try many *probable* pwds
  iii. Try *likely* passwords pwds
  iv. Search system list of pwds
  v. Find pwds by exploiting indiscreet users (social engg)

© 2006 by Leszek T. Lilien

i. Try *all* possible pwds (1)

- Try all possible = exhaustive attack / brute force attack
- Approach: Try all possible character combinations
- Example
  - Suppose: - only 26 chars (a-z) allowed in pwd
    - pwd length: 8 chars
  - nr_of_pwds $= \Sigma_{i=1}^{8}$ nr_of_*i*-char_pwd

    $= \Sigma_{i=1}^{8} 26^i = 26^9 - 1 \approx 5 * 10^{12}$
  - If attacker's computer checks 1 pwd/μs => $5* 10^{12}$ μs = 5 mln s ≈ 2 months to check all possible char combinations for a given pwd (max. exhaustive attack time)
    - With uniform distribution (neither good nor bad luck), expected successful attack time is = ½ of max. exh. attack time (1 month)
  - Is the attack target worth such attacker's investment?

    Might be – e.g., a bank acct, credit card nr

© 2006 by Leszek T. Lilien

Try *all* possible pwds (2)

- Countering brute force pwd attacks - finding *minimum required pwd length* to limit probability of attack success
  - Assumptions
    - Passwords drawn from a 96-char alphabet
    - Attacker can test $G = 10^4$ guesses per second
  - Goal
    - Find the required minimum password length s of passwords so that probability P of a successful attack is 0.5 over a 365-day guessing attack period

© 2006 by Leszek T. Lilien

Try *all* possible pwds (3)

- Solution
  - We know that: $P \geq TG / N$

    P - probability of a successful attack

    T - number of time units [sec] during which guessing occurs

    G - number of guesses per time unit [sec]

    N - number of possible passwords

    $$P \geq TG / N \Rightarrow N \geq TG / P$$
  - Calculations:

    $N \geq TG / P$ =

    $= (365 \text{ days} \times 24\text{hrs} \times 60\text{min} \times 60\text{s}) \times 10^4 / 0.5 = 6.31 \times 10^{11}$

    Choose password length s such that at least N passwords are possible, i.e.

    $$\Sigma^{s}_{j=1} 96^j \geq N = 6.31 \times 10^{11}$$

    (96 1-char "words" + 96^2 2-char "words" + ...96^s s-char "words")

    => $s \geq 6$

    i.e., passwords must be at least 6 chars long

© 2006 by Leszek T. Lilien

ii. Try many *probable pwds* (1)

- Can reduce expected successful attack time by checking *most probable char combinations* for a pwd first:
  - Check short pwds first
  - Check common words, etc. first

- Example – check short pwds first
  - People prefer short pwds => check pwds of length ≤ k
  - Assume 1 pwd checked per µs (per ms in text – p.213)
  - k=3: $26^1 + 26^2 + 26^3 = 18{,}278$ possible pwds
    => 18,278 µs ≈ 18.3 ms to check all combinations
  - k=4: … ≈ 475 ms ≈ 0.5 s
  - k=5: … ≈ 12,356 ms ≈ 12.4 s

© 2006 by Leszek T. Lilien

Try many *probable pwds* (2)

- Expected time can be further reduced

  bec. people use common words rather than random char combinations

  - E.g., prefer ‘jenny’ or ‘beer’ to ‘vprw’ or ‘qipd’

  => attacker can use spell checker dictionaries

  => dictionary attack (more later)

- Limiting succes of attacks on short passwords:
  - ATM swallows the cash card after *k* bad attempts of entering the PIN code (extremely short 4-digit code! Only 10,000 combinations)
  - Computer locks up after *n* tries (e.g. freezes the attacked account)

[cf. B. Endicott-Popovsky and D. Frincke]

iii. Try *likely* pwds (1)

- People are predictable in pwd selection
  - Attacker can restrict attack dictionary first to names of: family, pets, celebrities, sports stars, streets, projects,...
- Example: 1979 study of pwds [Morris and Thompson]
  - Table 4-2 – p.214 (see):
    - Even single char pwds!
    - 86% of pwds extremely simplistic!
      - All could be discovered in a week even at 1 msec/pwd checking rate
  - Study repeated in 1990 [Klein] and 1992 [Spafford] with similarly dismal results!
    - Klein: 21% guessed in a week
    - Spafford: ~29% od pwds consisted of lowercase a-z only!

© 2006 by Leszek T. Lilien

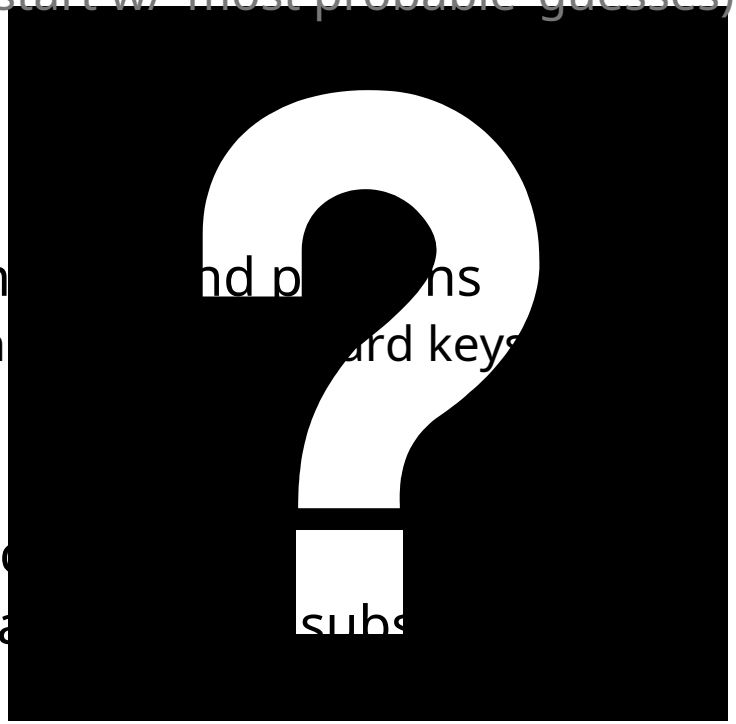
Try *likely* pwds (2)

- Utilites helping admins to identify bad pwds
  - COPS
  - Crack
  - SATAN

  Can be used by attackers, too

[cf. B. Endicott-Popovsky and D. Frincke]

- 12 steps an attacker might try (start w/ 'most probable' guesses)
  1) No password
  2) Same as user ID
  3) User's name or derived from it
  4) Common word list plus common n nd p ns
     - Ex. common patterns: 'asdfg' – con rd key
  5) Short college dictionary
  6) Complete English word list
  7) Common non-English language di
  8) Short college dictionary with capita subs
     - E.g. PaSsWoRd, pa$$w0rd
     - Substitutions include: a -> @, e -> 3, i/l -> 1, o -> 0, s -> $, …
  9) Complete English with capitalization and substitutions
  10) Common non-English dictionaries with capitalization and substitutions
  11) Brute force, lowercase alphabetic characters
  12) Brute force, full character set



© 2006 by Leszek T. Lilien

iv. Search system list of pwds

- System must keep list of passwords to authenticate logging users
- Attacker may try to capture pwd list
- Pwd lists:
  1) Plaintext system pwd file
  2) Encrypted pwd file
     a. Conventional encryption
     b. One-way encryption

© 2006 by Leszek T. Lilien

Search system list of pwds (2)

1) Plaintext system pwd file
   - Protected w/ strong access controls
     - Only OS can access it
     - Better: only *some* OS modules that really need access to pwd list can access it
       - Otherwise any OS penetration is pwd file penetration
   - Attacker's ways od getting plaintext pwd files:
     - Memory dump and searching for pwd table
     - Get pwd table from system backups
       - Backups often include no file protection – security of backups relies on physical security an access controls
     - Get pwd file by attacking disk

© 2006 by Leszek T. Lilien

Search system list of pwds (3)

2) Encrypted pwd file

- Two approaches:
  a. Conventional encryption / b. One-way encryption

a. Conventional encryption
   - Encrypts entire pwd table
     OR
     encrypts pwd column of pwd table
- Pwd comparison procedure:
  - When logging principal provides (cleartext) pwd, OS *decrypts* pwd from pwd table
  - OS compares principal's (clrtxt) pwd w/ *decrypted* pwd
- Exposure 1: when decrypted pwd is for an instant in memory
  - Attacker who penetrates memory can get it
- Exposure 2: attacker finding encryption key

© 2006 by Leszek T. Lilien

Search system list of pwds (4)

b. One-way encryption (hashing)

- Better solution - no pwd exposure in memory
- Pwd encrypted w/ one-way hash function and store

- Pwd comparison procedure:
  - When logging principal provides (cleartext) pwd, OS *hashes* principal's pwd (w/ one-way encryption)
  - *Hash* of principal's pwd is compared with pwd hash from pwd table

- Advantages of one-way encryption:
  - Pwd file can be stored in plain view
  - Backup files not a problem any more

© 2006 by Leszek T. Lilien

Search system list of pwds (5)

- Problem: If Alice and Bill selected the same pwd (e.g., Kalamazoo) and Bill reads pwd file (stored in plain view), Bill learns Alice's pwd

- Solution: *salt* value is used to perturb hash fcn
  - Hashed value and salt stored in pwd table:
    - [Alice, $salt_{Alice}$, $E(pwd_{Alice}+salt_{Alice})$] stored for Alice
    - [Bill, $salt_{Bill}$, $E(pwd_{Bill}+salt_{Bill})$] stored for Bill

  => hashed Alice's pwd ≠ hashed Bill's pwd (even if $pwd_{Alice} = pwd_{Bill}$)

  - When Principal X logs in, system gets $salt_X$ and calculates $E(pwd_X+salt_X)$

    If result is the same as hash stored for X, X is authenticated

© 2006 by Leszek T. Lilien

OPTIONAL -- Search system list of pwds (6)

- Example: Vanilla UNIX method (see next slide)
  - When password set, the *salt* is chosen randomly as an integer from [0, 4095]
  - One-way function changed by the salt value
- In a sense, salt value selects one of *n* hash functions
  - E.g., salt viewed as a parameter that selects one of 4,096 hash functions
- Example of UNIX pwd file record [cf. A. Striegel]

Up to 8 chars of principal's pwd used (above 8 – ignored), 12-bit salt added, hashed into 11+2 chars

Pwd file record:

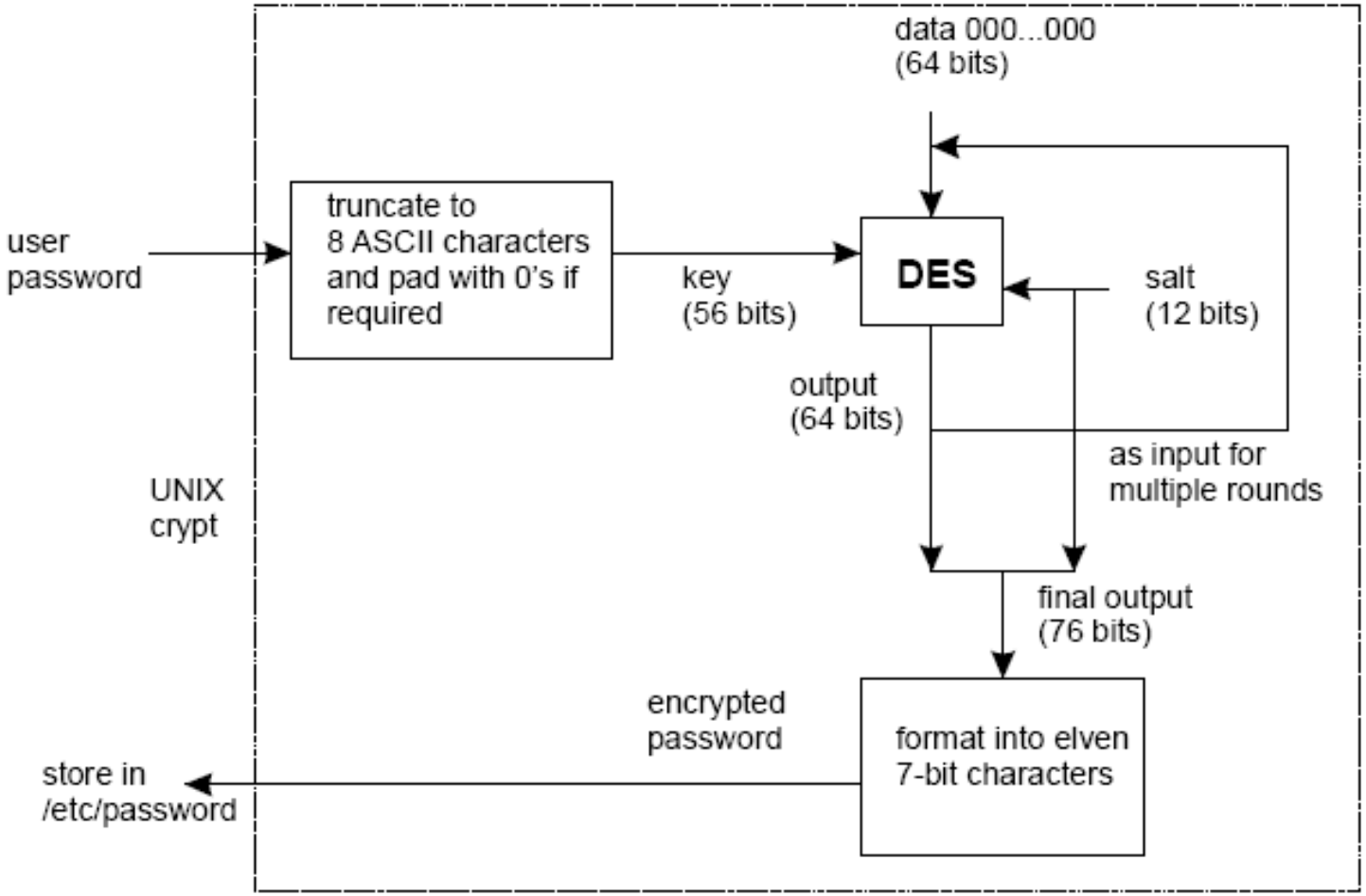
djones:EhYpHWagUoVhM:0:1:BERT:/:/bin/false

where: djones– username, EhYpHWagUoVhM - hashed password+salt (11+2 letters), 0 - userID, 1 - group nr, BERT- home dir, bin/false – shell

© 2006 by Leszek T. Lilien

OPTIONAL -- Search system list of pwds (7)

- One-way encryption of passwords in UNIX with salt



Search system list of pwds (8)

- Example: Dictionary attack on a single pwd in a one-way encrypted file
  - Dictionary attack phases:
    - Try in turn all words from an „attack dictionary” (from the most probable to the least probable)
    - If unsuccessful, try reversed words (e.g., “password” -> “drowssap”)
    - If unsuccessful, try all possible character combinations:

      lower case letters / some letters in upper case /

      characters such as !@#$ / etc.

© 2006 by Leszek T. Lilien

Search system list of pwds (9)

- Dictionary attack procedure:
  - Create an “attack dictionary” of words
    - Words: 1,000,000 most common passwords

    OR:

    - Words: All possible character combinations starting w/ most probable (names, words, reversed words, include upper case, include special chars, etc.)
  - For each “attack dictionary” word, calculate its hash, and store it in “hashed attack dictionary” (HAD)
    - For 1,000,000 words, HAD needs 8MB only (if 8 bit hash result)

  Note: If, e.g., 12-bit salt used, for each dictionary word must create 2^{12} = 4,096 hash values!

  => salt makes attacker’s job 4,096 times longer!
  - Steal an encrypted password file (EPF)
  - If a word from HAD matches any EPF entry, a password is found

© 2006 by Leszek T. Lilien

v. Exploiting indiscreet users

- A case of social engg
  - Can be much simpler than guessing pwds or breaking pwd file encryption
- Indiscreet principals
  - Pwd taped to PC or monitor
  - Principals sharing work tempted to share acct pwds
    - Rather than satisfy Alice's requests for data from file X, Bill might give Alice his account pwd and have her get the file herself

© 2006 by Leszek T. Lilien

d. Password selection criteria (1)

- Password selection criteria
  - Use characters other than just A – Z
  - Choose long passwords
  - Avoid actual names or words
  - Choose an unlikely password
  - Change password regularly
  - Don’t write it down
  - Don’t tell anyone else

© 2006 by Leszek T. Lilien

Password selection criteria (2)

- Good Password Examples
  - "LjMa*2^As"(^A = CTRL-a)(Lea, Jay, Mary, Albert – Akhil, Shail)
    - Names of members of 2 families, alternating case, nonprintable and uncommon characters
  - "OoHeO/FSK"
    - Second letter of each word of length 4 or more in third line of third verse of Star-Spangled Banner ("A home and a country should leave us no more") alternating case, followed by "/", followed by author's initials (by Francis Scott Key)
  - What's good here may be bad there
    - "DMC/MHmh" bad at Dartmouth ("Dartmouth Medical Center/Mary Hitchcock memorial hospital"), ok here

© 2006 by Leszek T. Lilien

OPTIONAL -- Password selection criteria (3)

- Proactive Password Checker
  - S/w that analyzes proposed password for “goodness”
  - Requirements
    - Always invoked
    - Can detect and reject bad passwords for an appropriate definition of “bad”
    - Discriminates on per-user, per-site basis
      - E.g., per user: “^AHeidiu” is bad for a parent of Heidi
    - Pattern matching on words that are bad passwords
      - E.g., “aaaa” and “tt” matched by the pattern: “^\(.\)\1*$”
    - Needs to execute subprograms and use results
      - Spell checker, for example, to detect word inflections (conjugations and declensions)
    - Easy to set up and integrate into password selection system

© 2006 by Leszek T. Lilien

OPTIONAL -- Password selection criteria (4)

- Application Example 1: Proactive Password Checker OPUS
  - Checks pwds against large dictionaries quickly
    - OPUS dictionary represented as OPUS bit vector (OBV) of length n
    - Each password from dictionary run through *k* different hash functions, producing integer values h_1, ..., h_k, all less than *n*
    - Before putting a password into dictionary, set bits h_1, ..., h_k in OBV
    - To check a new password, get its bit vector h_1', ..., h_k'
      - If *any* of the bits h_1', ..., h_k' are *not* set in OBV, the candidate password is *definitely* not in OPUS dictionary (good password choice)
      - If *all* bits h_1', ..., h_k' *are* set in OBV, the candidate password is probably in OPUS dictionary (so, it is a poor password choice)

© 2006 by Leszek T. Lilien

OPTIONAL -- Password selection criteria (5)

- Application Example 2: Proactive Pwd Check with *passwd+*
  - Little language to describe proactive checking
    - test length("$p") < 6
      - If password under 6 characters, reject it
    - test infile("/usr/dict/words", "$p")
      - If password in file /usr/dict/words, reject it
    - test !inprog("spell", "$p", "$p")
      - If password not in the output from program spell, given the password as input, reject it (because it's a properly spelled word—poor password choice)

© 2006 by Leszek T. Lilien

- Password Aging
  - Force users to change passwords after some time has expired
    - How do you force principals not to re-use passwords?
      - Record *n* previous passwords
        - Problem:

          User changing passwords *n* times in a very short time to get back to his favorite one (entered as n+1st)
        - Solution:

          Block password changes for a period of time
    - Give users time to think of good passwords
      - Don't force them to change before they can log in
      - Warn them of expiration days in advance

© 2006 by Leszek T. Lilien

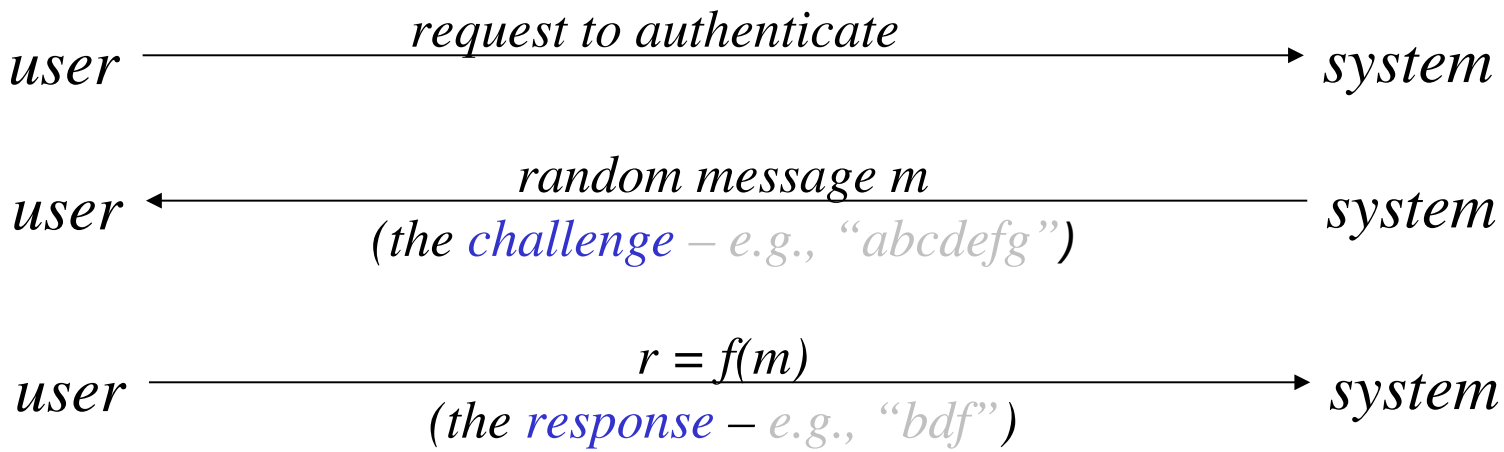
e. One-time passwords (1)

- One-time passwords = challenge-response systems
  - Pwd changes every time it is used => can be used exactly *once*
    - Immediately invalidated after its use
      - An ultimate form of *password aging*
  - Not a static word/phrase but a math function
  - Also for host-host authentication (not only user-host)
- Scenario (see next slide):
  - System provides challenge (argument)
    User returns response (computed fcn value)
    - *E.g., :*
      - *Challenge:* the number of authentication (NOA)
      - *Response:* the one-time password for that *NOA*
  - System evaluates response
    If response is valid, user is authenticated

© 2006 by Leszek T. Lilien

One-time passwords (2)

- Challenge-Response Authentication
  - Principal & system share a *secret* function *f* (*f* can be a known function with an unknown parameter, such as a cryptographic key)



- Example:
  - *Identification—friend or foe (IFF)* is a challenge-response technique used to identify friendly and enemy aircraft

© 2006 by Leszek T. Lilien

One-time passwords (3)

- Examples of challenge fcns:
  - Simple function
    - f(x) = x+1 / f(x) = 3x**2 – 9x +2
    - f(x) = „x-th prime number”
    - f(x) = (day of the month) * (hour of current time)
  - Pseudo-random number generator
    - f(x) = r(x) - random nr for seed x
      - Requires availability of the same pseudo-random generator to host and user
  - Character string fcns
    - f(<character_string>) = (transformed character string)
      - E.g. f(a1a2a3a4a5a6) = a3a1a1a4 [e.g., f(signon) = gssn]
  - Cryptographic fcns
    - f(E(x)) = E(D(E(x)) + 1) (decrypt, add 1, encrypt)

© 2006 by Leszek T. Lilien

One-time passwords (4)

- Advantage: Intercepted pwd is useless for attacker
- Problems
  - Synchronization of principals with system
    - System tells user which password it expects (e.g., pwd #73)
  - Reliable and secret distribution of pwds for response
  - Generation of good random pwds
  - Fcns for user authentication too complex
    Solution: equip users with proper h/w support (below)
- Hardware support for challenge-response authentication

1) Token-based devices

- Utilized by principal to compute response to challenge
  - May require PIN/password from the user
    - May be combined with the challenge to produce response
  - Can encipher or hash response

© 2006 by Leszek T. Lilien

One-time passwords (5)

2) Temporally-based devices

- Every minute device shows a different nr (range: 0 to $10^n - 1$)
  - Computer knows what nr to expect from user's device (synchronized!)
- Principal enters login name
- System requests password
- Principal provides nr shown on device followed by her fixed (reusable) pwd
- System validates if the number and password are as expected
- Example: RSA SecureID [cf. A. Striegel]
  - Number [0, $10^N - 1$], changes every minute
  - Small, server synchronized – knows next nr
  - User sends password + nr

© 2006 by Leszek T. Lilien

One-time passwords (6)

- Pass Algorithms

  - category of challenge-response where the fcn *f* is *secret*

  - Example:
    - Challenge: random string of characters
      - E.g., "abcdefg", "ageksido"
    - Response: some function of that string
      - E.g., select chars in *even* positions: "bdf", "gkio," respectively
  - Can alter algorithm based on context information
    - E.g., *network* connection — as above,

      dial-up might require "aceg", "aesd" (odd positions)
  - Usually used in conjunction with a fixed, reusable password

© 2006 by Leszek T. Lilien

OPTIONAL -- One-time passwords (7)

- Preventing Dictionary Attacks in Challenge-Response Authentication — Encrypted Key Exchange (EKE) Protocol
  - Defeats off-line dictionary attacks
  - Idea:

    random challenges enciphered =>

    attacker cannot verify correct decipherment of challenge

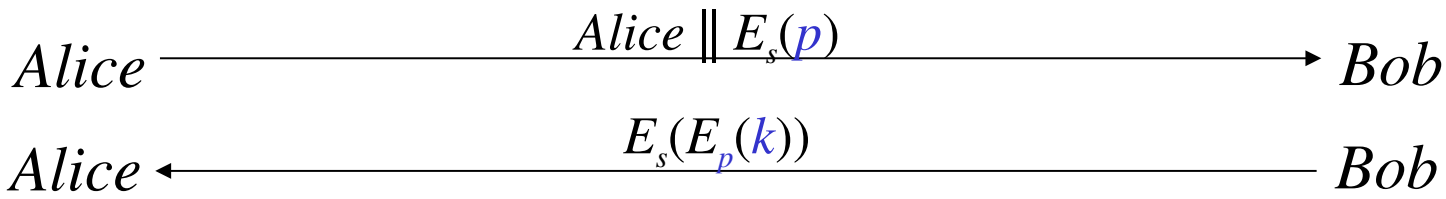
  - Assume:
    - Alice and Bob share secret password *s*
    - Alice generates a random *public* key *p* and *private* key *q*

© 2006 by Leszek T. Lilien

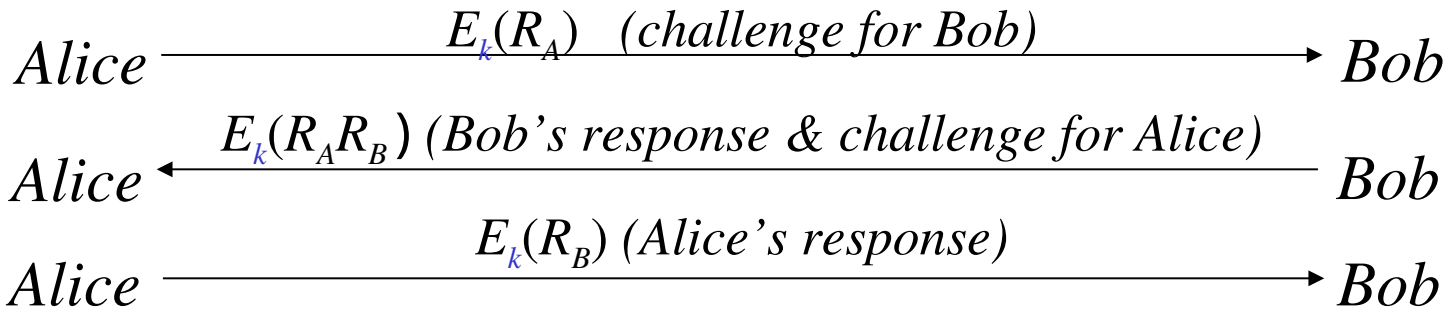
OPTIONAL -- One-time passwords (8)

secret password s / public key p / private key q /

randomly generated secret *session key* k / challenges R_A & R_B



Now Alice and Bob share a randomly generated secret session key *k*. The challenge-response phase of the protocol begins



© 2006 by Leszek T. Lilien

OPTIONAL -- One-time passwords (9)

- Immunity of EKE Protocol against Dictionary Attacks
  - EKE ensures that random challenges are always encrypted
  - Attacker cannot verify that her challenge deciphering is correct since:
    - Challenges are random
    - Challenges are unknown to attacker (never in plaintext)

© 2006 by Leszek T. Lilien

OPTIONAL -- One-time passwords (10)

- Example of one-time pwd system: S/Key Protocol
  - One-way hash fcn *h* (e.g., MD5 or SHA-1)
  - User chooses initial seed *k*
  - System calculates (example for n = 100):

$$h(k) = k_1,\ h(k_1) = k_2,\ \ldots,\ h(k_{99}) = k_{100}$$

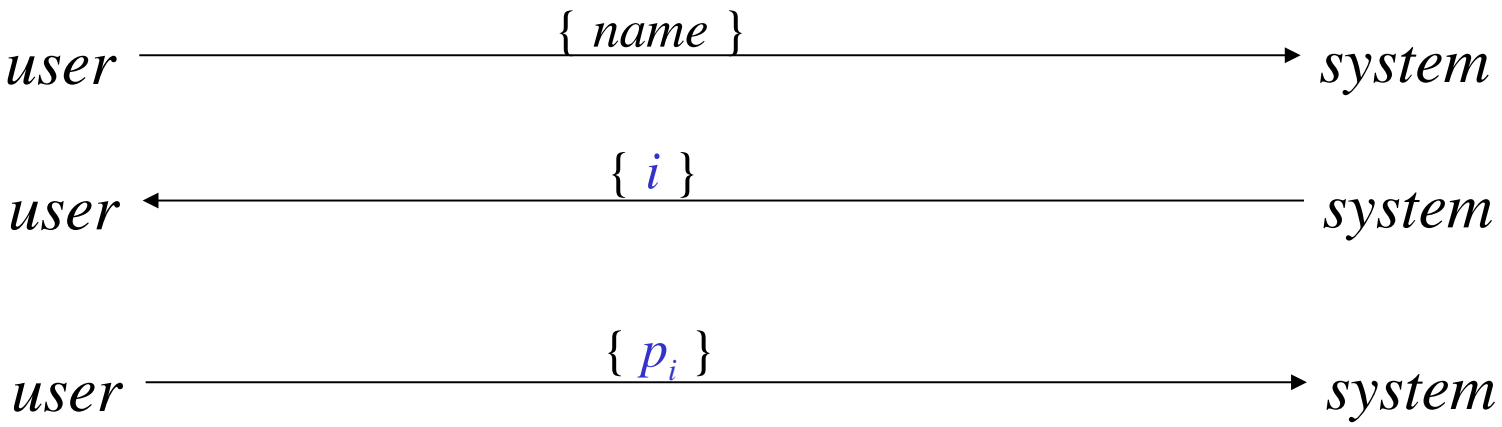
  - Passwords are reverse order:

$$p_1 = k_{100},\ p_2 = k_{99},\ \ldots,\ p_{99} = k_2,\ p_{100} = k_1$$

© 2006 by Leszek T. Lilien

OPTIONAL -- One-time passwords (11)

- System stores maximum number of authentications n *(e.g. 100)*, number of next authentication i, last correctly supplied password p_{i-1}.



System computes $h(p_i) = h(k_{101-i}) = k_{102-i} = p_{i-1}$.

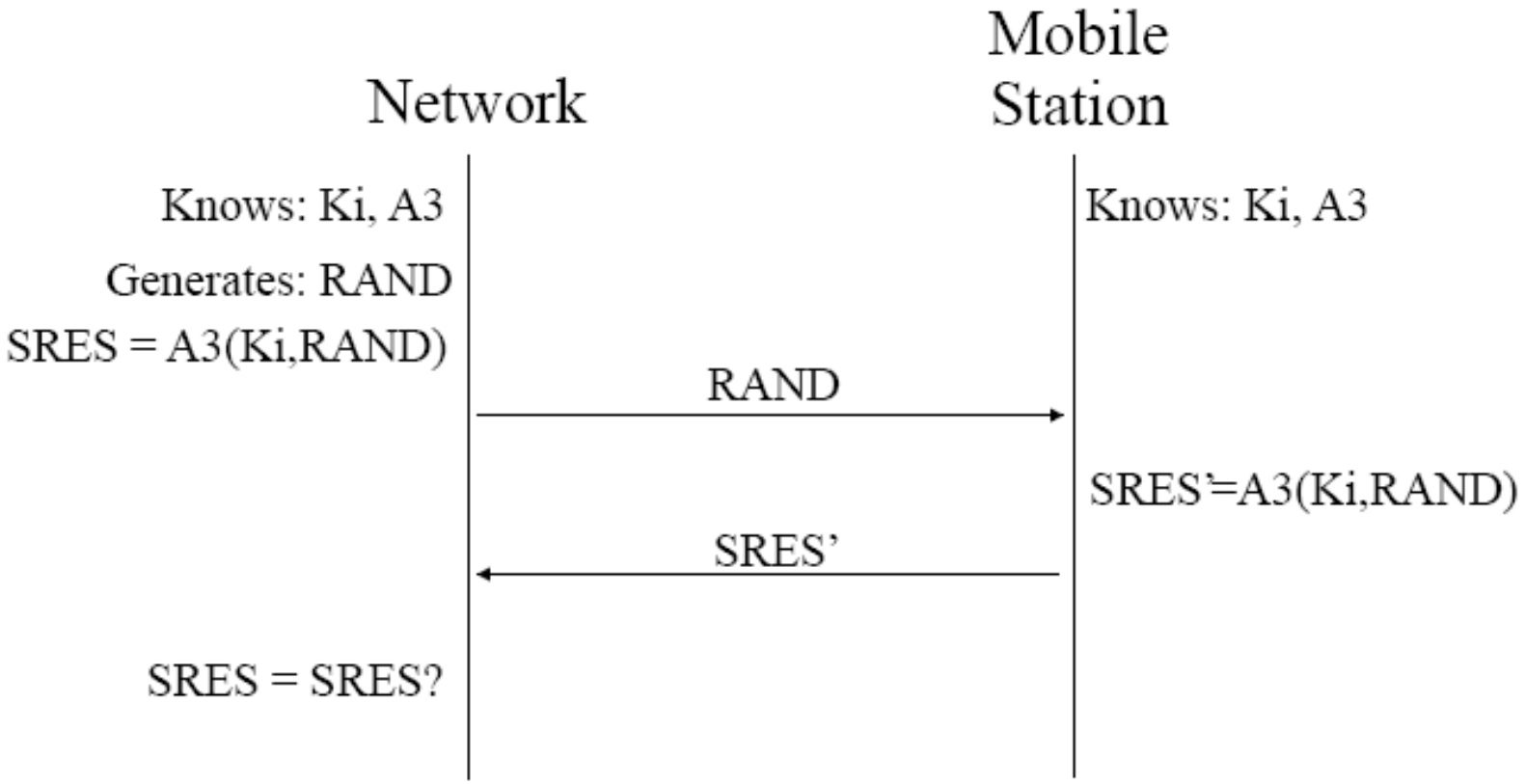
If match with p_{i-1}, system replaces p_{i-1} with p_i and increments i.

E.g. if i = 5: system computes $h(p_5) = h(k_{96}) = k_{97} = p_4$

Result matches p_4, so system replaces p_4 with *p5* and increments *i to 6*.

OPTIONAL -- One-time passwords (12)

- Challenge-Response Authentication á la GSM
  - Uses random numbers (RAND)



[cf. J. Leiwo]

f. The authentication process (1)

- Blocking attackers
  1) By deliberately slow authentication
     - Could take 5-10 s per login attempt
     - No problem for legitimate principals
       - barrier to brute-force attacks

       attacker can't check a pwd per µs or millisec any more
  2) By limiting nr of login attempts
     - Disconnects or delays user after *n* failed attempts
       - Or, disables account after *n* attempts - user must reset pwd
     - Legitimate principal will login in at most 2-3 attempts
     - Attacker would try thousands times

© 2006 by Leszek T. Lilien

The authentication process (2)

- n-factor authentication (nFA)
  - Makes authentication more trustworthy
  - Usually, two-factor authentication (2FA) and three-factor authentication (3FA)
  - nFA uses *n* means of authentication
    - E.g., for 2FA: password + challenge-response
- Fixing flaws in authentication process
  - By using nFA ($n \geq 2$)
  - By using challenge-response as one of factors
    - Variable response protects against intercepted pwds
  - By authentication of system to user
    - Otherwise, attacker impersonating system can harm user
      - E.g., phishing
      - E.g., „false login” Trojan setup on public-access computer

© 2006 by Leszek T. Lilien

The authentication process (3)

- Authenticating system to user

  to prevent impersonator pretending to be user's system

  - Reinitialize communication with system
    - E.g., turn terminal off and on
    - E.g, press BREAK key
    - E.g., CTRL-ALT-DEL on MS Windows machines
  - Computer displaying *plaintext* information that impersonator (probably) wouldn't know
    - E.g., „Your last login was on 15 october 2005 at 07:45"
  - Computer displaying *encrypted* information that impersonator wouldn't know
    - E.g., timestamp encrypted with user's key (if decrypted time is current – all's OK)

© 2006 by Leszek T. Lilien

g. Authentication other than passwords

- Authentication other than passwords
  - Using special biometric devices (h/w devices)
    - Fingerprint detectors / handprint detectors
    - Voice recognizers / retina pattern scanners
  - Using extra info for authentication
    - User location / User work hours
    - User access patterns / User work habits
      - An attacker who pretends to be a legitimate user „Jones” must act as Jones, or will be detected

© 2006 by Leszek T. Lilien

h. Conclusions

- Authentication is not cryptography
  - You have to consider system components
- Passwords are here to stay
  - They provide a basis for most forms of authentication
- Protocols are important
  - They can make masquerading harder
- Authentication methods can be combined
  - Examples: 2FA, 3FA

© 2006 by Leszek T. Lilien

End of Section 4: Protection in General-Purpose OSs



4.2 MEMORY AND ADDRESS PROTECTION

The most obvious problem of multiprogramming is preventing one program from affecting the memory of other programs. Fortunately, protection can be built into the hardware mechanisms that control efficient use of memory, so that solid protection can be provided at essentially no additional cost.

Fence

The simplest form of memory protection was introduced in single-user operating systems, to prevent a faulty user program from destroying part of the resident portion of the operating system. As its name implies, a **fence** is a method to confine users to one side of a boundary.

In one implementation, the fence was a predefined memory address, enabling the operating system to reside on one side and the user to stay on the other. An example of this situation is shown in Figure 4-1. Unfortunately, this kind of implementation was

Scanned by CamScanner

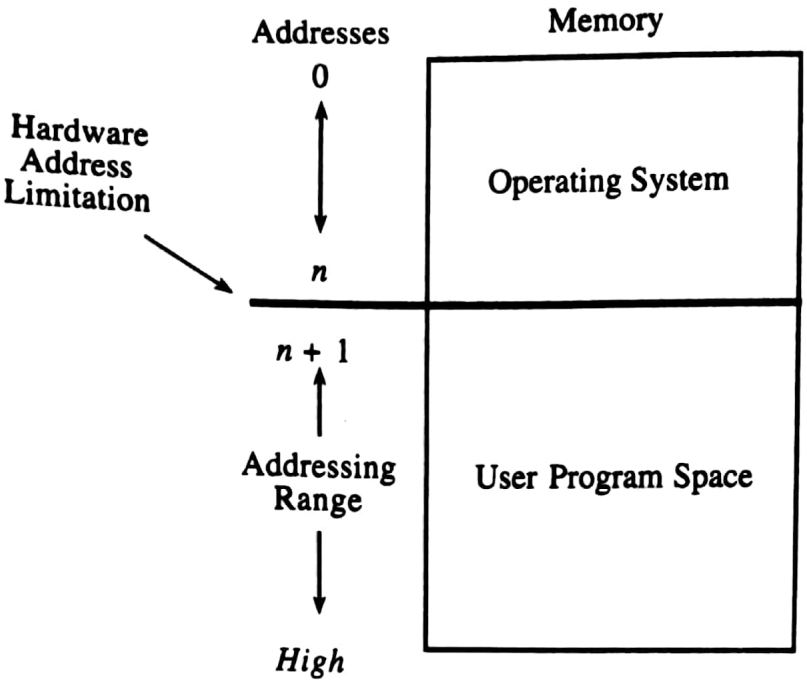


FIGURE 4-1 Fixed Fence.

very restrictive because a predefined amount of space was always reserved for the operating system, whether it was needed or not. If less than the predefined space was required, the excess space was wasted. Conversely, if the operating system needed more space, it could not grow beyond the fence boundary.

Another implementation used a hardware register, often called a **fence register**, containing the address of the end of the operating system. In contrast to a fixed fence, in this scheme the location of the fence could be changed. Each time a user program generated an address for data modification, the address was automatically compared with the fence address. If the address was greater than the fence address (that is, in the user area), the instruction was executed; if it was less than the fence address (that is, in the operating system area), an error condition was raised. The use of fence registers is shown in Figure 4-2.

A fence register protects only in one direction. In other words, an operating system can be protected from a single user, but the fence cannot protect one user from another user. Similarly, a user cannot identify certain areas of the program as inviolable (such as the code of the program itself or a read-only data area).

Relocation

If the operating system can be assumed to be of a fixed size, programmers can write their code assuming that the program begins at a constant address. This feature of the operating system makes it easy to determine the address of any object in the program. However, it also makes it essentially impossible to change the starting address if, for example, a new version of the operating system is larger or smaller than the old. If the size of the operating system is allowed to change, then programs must be written in a way that does not depend on placement at a specific location in memory.

Relocation is the process of taking a program written as if it began at address 0 and changing all addresses to reflect the actual address at which the program is located in memory. In many instances, this effort merely entails adding a constant **relocation**

Scanned by CamScanner

Security · Dola Mam

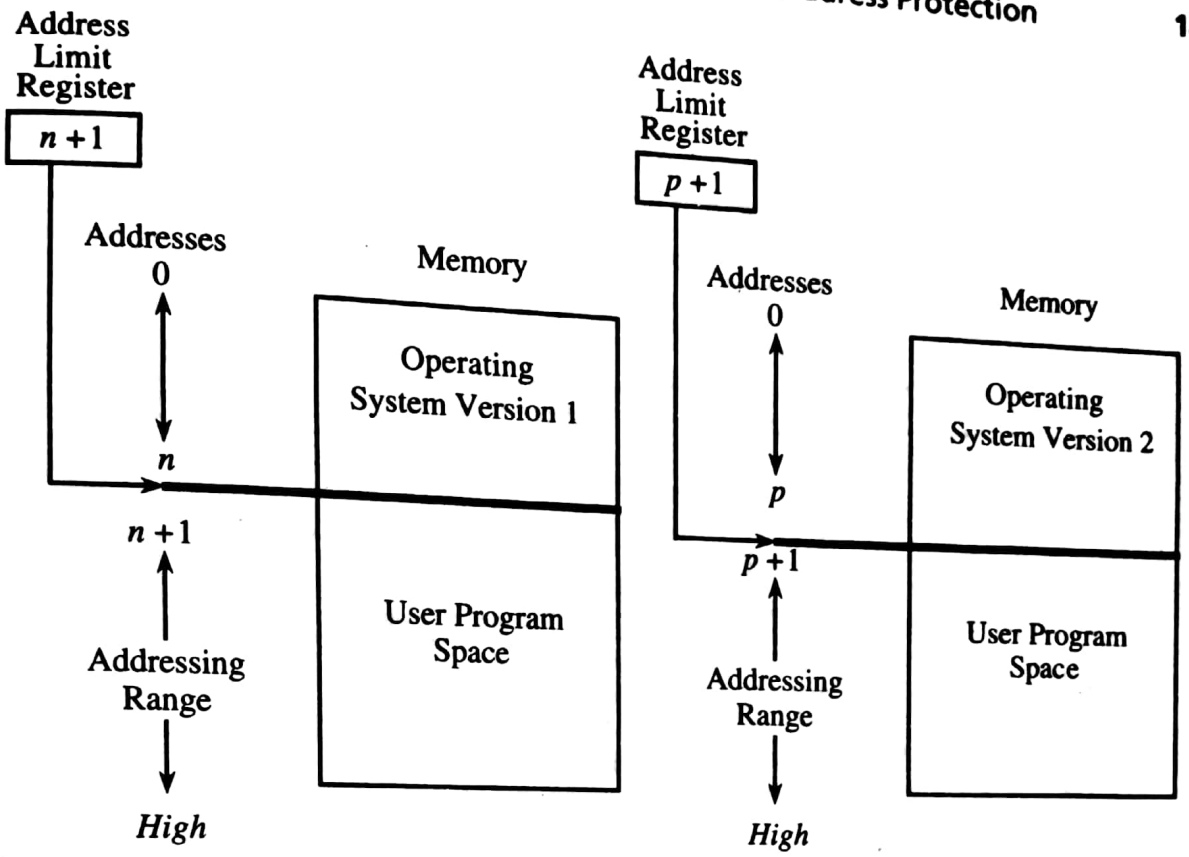


FIGURE 4-2 Variable Fence Register.

factor to each address of the program. That is, the relocation factor is the starting address of the memory assigned for the program.

Conveniently, the fence register can be used in this situation to provide an important extra benefit: The fence register can be a hardware relocation device. The contents of the fence register are added to each program address. This action both relocates the address and guarantees that no one can access a location lower than the fence address. (Addresses are treated as unsigned integers, so adding the value in the fence register to any number is guaranteed to produce a result at or above the fence address.) Special instructions can be added for the few times when a program legitimately intends to access a location of the operating system.

Base/Bounds Registers

A major advantage of an operating system with fence registers is the ability to relocate; this characteristic is especially important in a multiuser environment. With two or more users, none can know in advance where a program will be loaded for execution. The relocation register solves the problem by providing a base or starting address. All addresses inside a program are offsets from that base address. A variable fence register is generally known as a **base register**.

Fence registers provide a lower bound (a starting address) but not an upper one. An upper bound can be useful in knowing how much space is allotted and in checking for overflows into "forbidden" areas. To overcome this difficulty, a second register is often added, as shown in Figure 4-3. The second register, called a **bounds register**, is an upper address limit, in the same way that a base or fence register is a lower address

Scanned by CamScanner

Security · Dola Mam

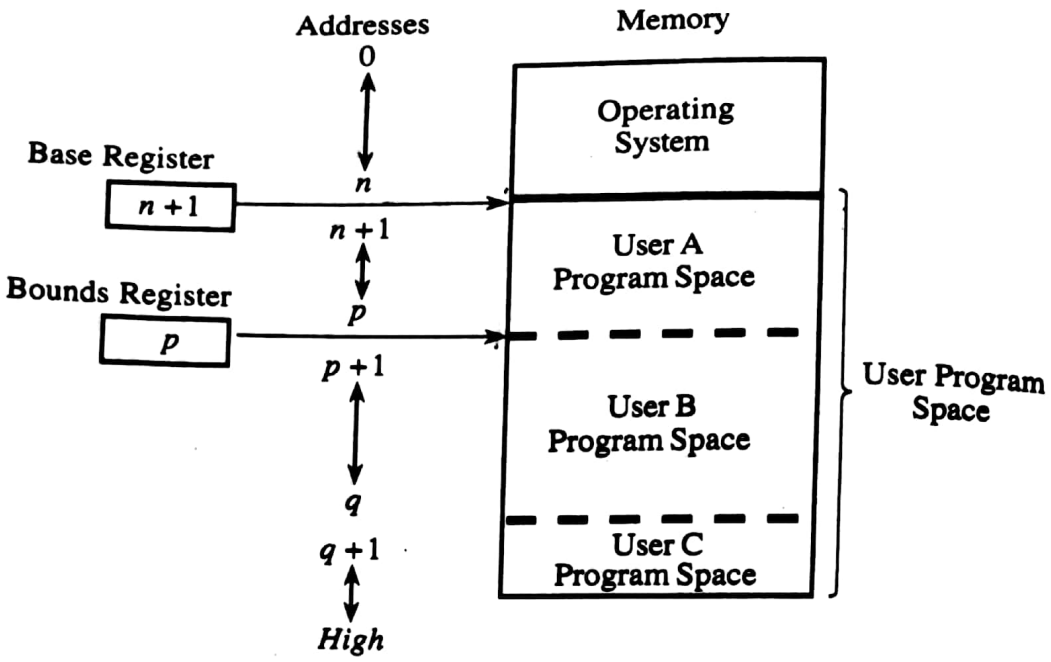


FIGURE 4-3 Pair of Base/Bounds Registers.

limit. Each program address is forced to be above the base address because the contents of the base register are added to the address; each address is also checked to ensure that it is below the bounds address. In this way, a program's addresses are neatly confined to the space between the base and the bounds registers.

This technique protects a program's addresses from modification by another user. When execution changes from one user's program to another's, the operating system must change the contents of the base and bounds registers to reflect the true address space for that user. This change is part of the general preparation, called a **context switch**, that the operating system must perform when transferring control from one user to another.

With a pair of base/bounds registers, a user is perfectly protected from outside users, or, more correctly, outside users are protected from errors in any other user's program. Erroneous addresses *inside* a user's address space can still affect that program because the base/bounds checking guarantees only that each address is inside the user's address space. For example, a user error might occur when a subscript is out of range or an undefined variable generates an address reference within the user's space but, unfortunately, inside the executable instructions of the user's program. In this manner, a user can accidentally store data on top of instructions. Such an error can let a user inadvertently destroy a program, but (fortunately) only the user's own program.

We can solve this overwriting problem by using another pair of base/bounds registers, one for the instructions (code) of the program and a second for the data space. Then, only instruction fetches (instructions to be executed) are relocated and checked with the first register pair, and only data accesses (operands of instructions) are relocated and checked with the second register pair. The use of two pairs of base/bounds registers is shown in Figure 4-4. Although two pairs of registers do not prevent all program errors, they limit the effect of data-manipulating instructions to the data space. The pairs of registers offer another more important advantage: the ability to split a program into two pieces that can be relocated separately.

Scanned by CamScanner

Security · Dola Mam

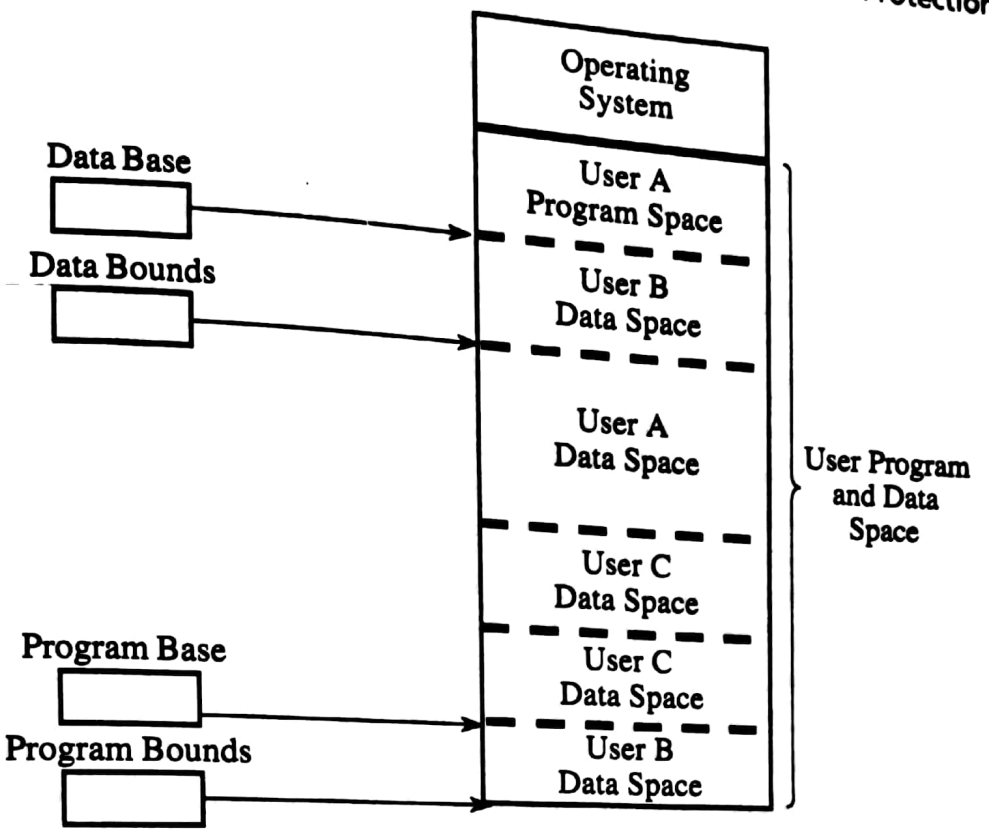


FIGURE 4-4 Two Pairs of Base/Bounds Registers.

These two features seem to call for the use of three or more pairs of registers: one for code, one for read-only data, and one for modifiable data values. Although in theory this concept can be extended, two pairs of registers are the limit for practical computer design. For each additional pair of registers (beyond two), something in the machine code of each instruction must indicate which relocation pair is to be used to address the instruction's operands. That is, with more than two pairs, each instruction specifies one of two or more data spaces. But with only two pairs, the decision can be automatic: instructions with one pair, data with the other.

Tagged Architecture

Another problem with using base/bounds registers for protection or relocation is their contiguous nature. Each pair of registers confines accesses to a consecutive range of addresses. A compiler or loader can easily rearrange a program so that all code sections are adjacent and all data sections are adjacent.

However, in some cases you may want to protect *some* data values but not *all*. For example, a personnel record may require protecting the field for salary but not office location and phone number. Moreover, a programmer may want to ensure the integrity of certain data values by allowing them to be written when the program is initialized but prohibiting the program from modifying them later. This scheme protects against errors in the programmer's own code. A programmer may also want to invoke a shared subprogram from a common library. We can address some of these issues by using good design, both in the operating system and in the other programs being run. Recall that in Chapter 3 we studied good design characteristics such as information hiding and

Scanned by CamScanner

Security · Dola Mam

modularity in program design. These characteristics dictate that one program module must share with another module only the *minimum* amount of data necessary for both of them to do their work.

Additional, operating-system-specific design features can help, too. Base/bounds registers create an all-or-nothing situation for sharing: Either a program makes all its data available to be accessed and modified or it prohibits access to all. Even if there were a third set of registers for shared data, all data would need to be located together. A procedure could not effectively share data items *A*, *B*, and *C* with one module, *A*, *C*, and *D* with a second, and *A*, *B*, and *D* with a third. The only way to accomplish the kind of sharing we want would be to move each appropriate set of data values to some contiguous space. However, this solution would not be acceptable if the data items were large records, arrays, or structures.

An alternative is **tagged architecture**, in which every word of machine memory has one or more extra bits to identify the access rights to that word. These access bits can be set only by privileged (operating system) instructions. The bits are tested every time an instruction accesses that location.

For example, as shown in Figure 4-5, one memory location may be protected as execute-only (e.g., the object code of instructions), whereas another is protected for fetch-only (e.g., read) data access, and another accessible for modification (e.g., write). In this way, two adjacent locations can have different access rights. Furthermore, with a few extra tag bits, different classes of data (numeric, character, address or pointer, and undefined) can be separated, and data fields can be protected for privileged (operating system) access only.

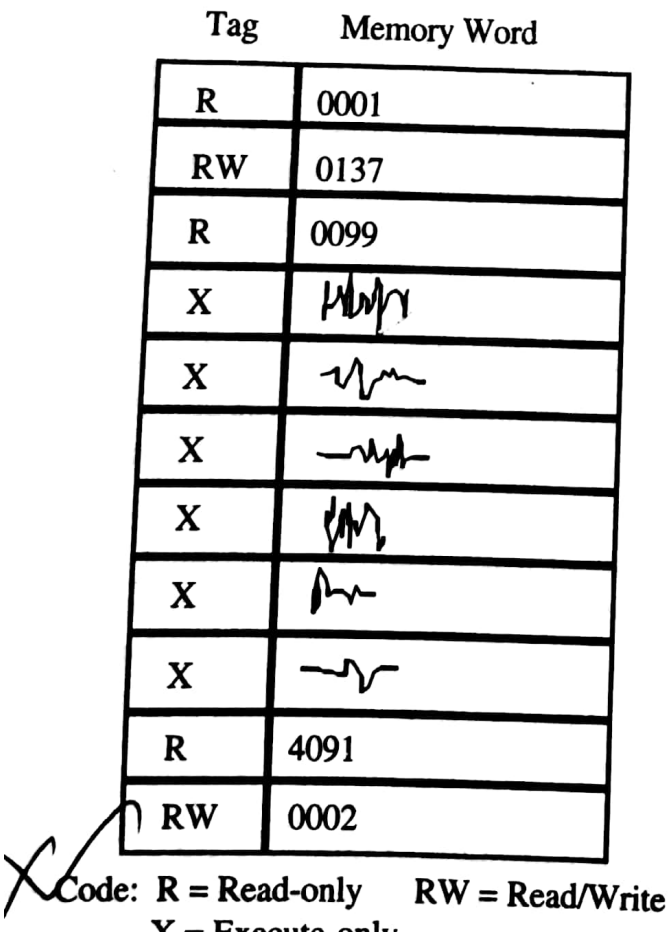


FIGURE 4-5 Example of Tagged Architecture.

Scanned by CamScanner

This protection technique has been used on a few systems, although the number of tag bits has been rather small. The Burroughs B6500-7500 system used three tag bits to separate data words (three types), descriptors (pointers), and control words (stack pointers and addressing control words). The IBM System/38 used a tag to control both integrity and access.

A variation used one tag that applied to a group of consecutive locations, such as 128 or 256 bytes. With one tag for a block of addresses, the added cost for implementing tags was not as high as with one tag per location. The Intel I960 extended architecture processor used a tagged architecture with a bit on each memory word that marked the word as a "capability," not as an ordinary location for data or instructions. A capability controlled access to a variable-sized memory block or segment. This large number of possible tag values supported memory segments that ranged in size from 64 to 4 billion bytes, with a potential 2^{256} different protection domains.

Compatibility of code presented a problem with the acceptance of a tagged architecture. A tagged architecture may not be as useful as more modern approaches, as we will see shortly. Some of the major computer vendors are still working with operating systems that were designed and implemented many years ago for architectures of that era. Indeed, most manufacturers are locked into a more conventional memory architecture because of the wide availability of components and a desire to maintain compatibility among operating systems and machine families. A tagged architecture would require fundamental changes to substantially all of the operating system code, a requirement that can be prohibitively expensive. But as the price of memory continues to fall, the implementation of a tagged architecture becomes more feasible.

Segmentation

We present two more approaches to protection, each of which can be implemented on top of a conventional machine structure, suggesting a better chance of acceptance. Although these approaches are ancient by computing's standards—they were designed between 1965 and 1975—they have been implemented on many machines since then. Furthermore, they offer important advantages in addressing, with memory protection being a delightful bonus.

The first of these two approaches, **segmentation**, involves the simple notion of dividing a program into separate pieces. Each piece has a logical unity, exhibiting a relationship among all of its code or data values. For example, a segment may be the code of a single procedure, the data of an array, or the collection of all local data values used by a particular module. Segmentation was developed as a feasible means to produce the effect of the equivalent of an unbounded number of base/bounds registers. In other words, segmentation allows a program to be divided into many pieces having different access rights.

Each segment has a unique name. A code or data item within a segment is addressed as the pair ⟨*name*, *offset*⟩, where *name* is the name of the segment containing the data item and *offset* is its location within the segment (that is, its distance from the start of the segment).

Logically, the programmer pictures a program as a long collection of segments. Segments can be separately relocated, allowing any segment to be placed in any

Scanned by CamScanner

Security · Dola Mam

available memory locations. The relationship between a logical segment and its true memory position is shown in Figure 4-6.

The operating system must maintain a table of segment names and their true addresses in memory. When a program generates an address of the form ⟨*name*, *offset*⟩, the operating system looks up *name* in the segment directory and determines its real beginning memory address. To that address the operating system adds *offset*, giving the true memory address of the code or data item. This translation is shown in Figure 4-7. For efficiency there is usually one operating system segment address table for each process in execution. Two processes that want to share access to a single segment would have the same segment name and address in their segment tables.

Thus, a user's program does not know what true memory addresses it uses. It has no way—and no need—to determine the actual address associated with a particular ⟨*name*, *offset*⟩. The ⟨*name*, *offset*⟩ pair is adequate to access any data or instruction to which a program should have access.

This hiding of addresses has three advantages for the operating system.

- The operating system can place any segment at any location or move any segment to any location, even after the program begins to execute. Because the operating system translates all address references by a segment address table, the operating system needs only to update the address in that one table when a segment is moved.

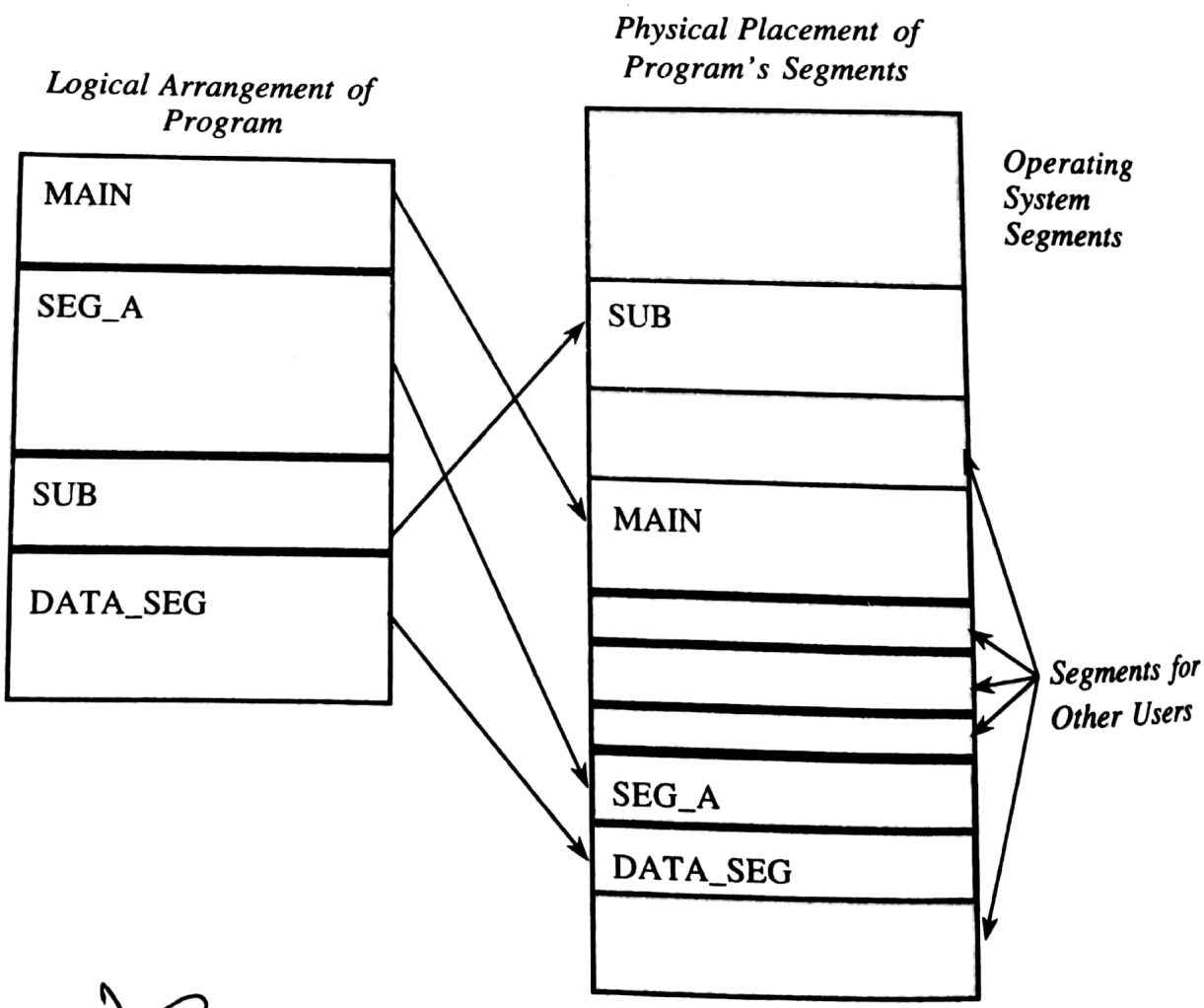


FIGURE 4-6 Logical and Physical Representation of Segments.

Scanned by CamScanner

Security · Dola Mam

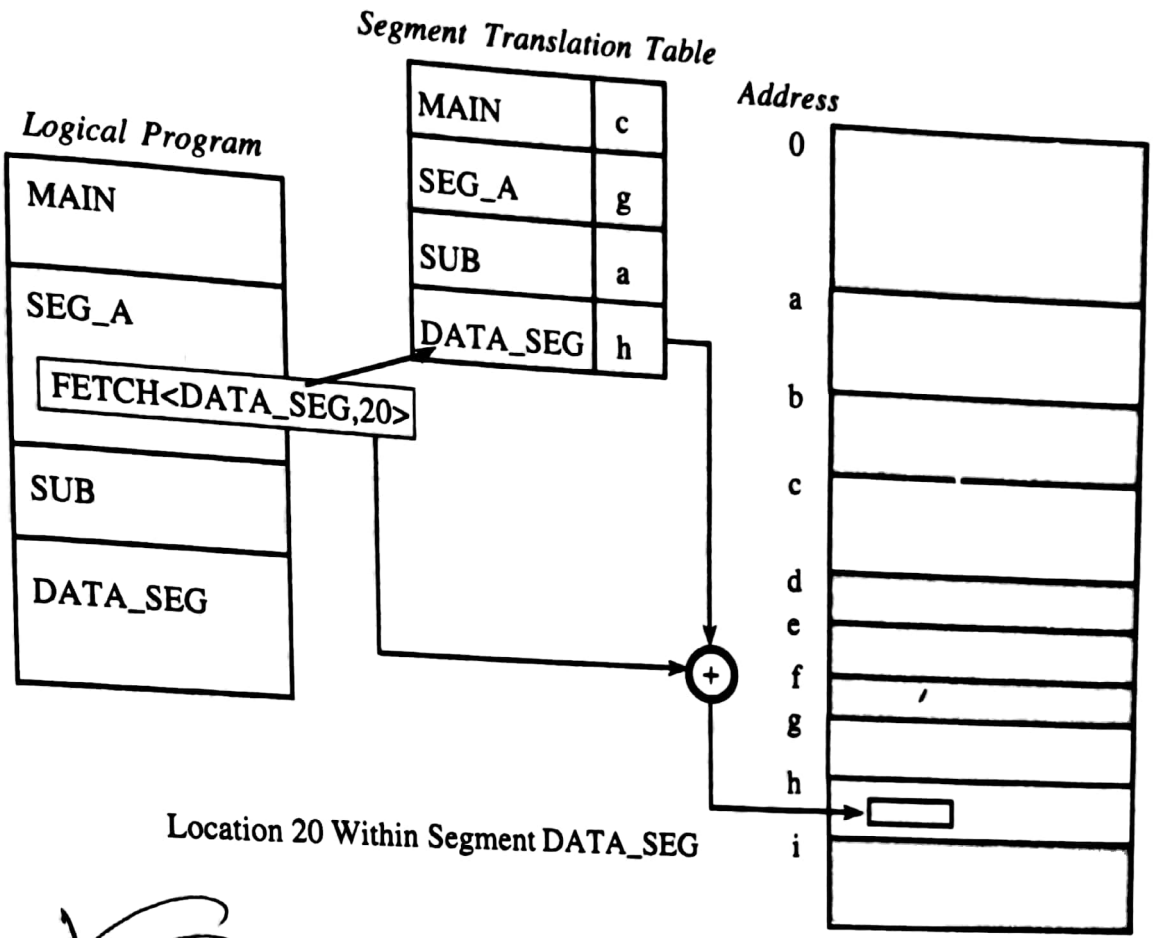


FIGURE 4-7 Translation of Segment Address.

- A segment can be removed from main memory (and stored on an auxiliary device) if it is not being used currently.
- Every address reference passes through the operating system, so there is an opportunity to check each one for protection.

Because of this last characteristic, a process can access a segment only if that segment appears in that process's segment-translation table. The operating system controls which programs have entries for a particular segment in their segment address tables. This control provides strong protection of segments from access by unpermitted processes. For example, program *A* might have access to segments *BLUE* and *GREEN* of user *X* but not to other segments of that user or of any other user.

We often want a user to have different protection classes for different segments of a program. To do this, the segmentation process uses both hardware and software. The overall system can associate certain levels of protection with certain segments, and it uses both the operating system and hardware to check that protection on each access to the segment. For example, one segment might be read-only data, a second might be execute-only code, and a third might be writeable data. In a situation like this one, segmentation can approximate the goal of separate protection of different pieces of a program, as outlined in the previous section on tagged architecture.

Segmentation offers these protective benefits.

- Each address reference is checked for protection.
- Many different classes of data items can be assigned different levels of protection.

Scanned by CamScanner

Security · Dola Mam

- Two or more users can share access to a segment, with potentially different access rights.
- A user cannot generate an address or access to an unpermitted segment.

One protection difficulty inherent in segmentation concerns segment size. Each segment has a particular size. However, a program can generate a reference to a valid segment *name*, but with an *offset* beyond the end of the segment. For example, reference ⟨*A*,9999⟩ looks perfectly valid, but in reality segment *A* may be only 200 bytes long. If left unplugged, this security hole could allow a program to access any memory address beyond the end of a segment just by using large values of *offset* in an address.

This problem cannot be stopped during compilation, or even when a program is loaded, because effective use of segments requires that they be allowed to grow in size during execution. For example, a segment might contain a dynamic data structure such as a stack. Therefore, secure implementation of segmentation requires checking a generated address to verify that it is not beyond the current end of the segment referenced. Although this checking results in extra expense (in terms of time and resources), segmentation systems must perform this check; the segmentation process must maintain the current segment length in the translation table and compare every address generated.

Thus, we need to balance protection with efficiency, finding ways to keep segmentation as efficient as possible. However, efficient implementation of segmentation presents two problems: Segment names are inconvenient to encode in instructions, and the operating system's lookup of the name in a table can be slow. To overcome these difficulties, segment names are often converted to numbers by the compiler when a program is translated; the compiler also appends a linkage table matching numbers to true segment names. Unfortunately, this scheme presents an implementation difficulty when two procedures need to share the same segment, because the assigned segment numbers of data accessed by that segment must be the same.

Segmentation can also present a third problem: Segments cause fragmentation of main memory because they are of varying sizes. After time, unused fragments of space can lead to poor memory utilization. A solution to fragmentation is the periodic compacting of memory; however, compacting and updating appropriate tables takes time.

Paging

One alternative to segmentation is **paging**. The program is divided into equal-sized pieces called **pages**, and memory is divided into equal-sized units, called **page frames**. (For implementation reasons, the page size is usually chosen to be a power of two between 512 and 4096 bytes.) As with segmentation, each address in a paging scheme is a two-part object, this time consisting of ⟨*page*, *offset*⟩.

Each address is again translated by a process similar to that of segmentation: The operating system maintains a table of user page numbers and their true addresses in memory. The *page* portion of every ⟨*page*, *offset*⟩ reference is converted to a page frame address by a table lookup; the *offset* portion is added to the page frame address to produce the real memory address of the object referred to as ⟨*page*, *offset*⟩. This process is illustrated in Figure 4-8.

Scanned by CamScanner

Security · Dola Mam

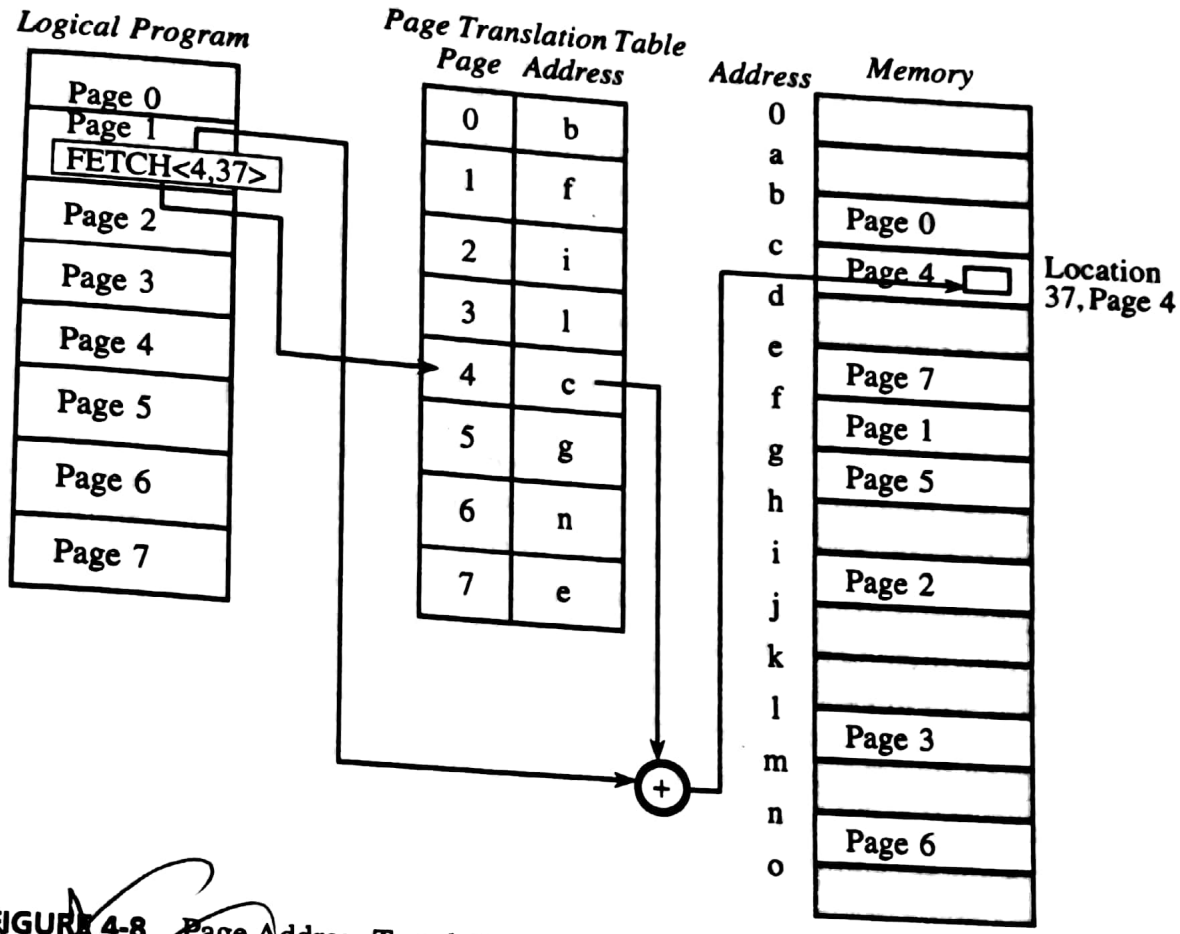


FIGURE 4-8 Page Address Translation.

Unlike segmentation, all pages in the paging approach are of the same fixed size, so fragmentation is not a problem; each page can fit in any available page in memory. Thus, there is no problem of addressing beyond the end of a page. The binary form of a ⟨*page*, *offset*⟩ address is designed so that the *offset* values fill a range of bits in the address. Therefore, an *offset* beyond the end of a particular page results in a carry into the *page* portion of the address, which changes the address.

To see how this idea works, consider a page size of 1024 bytes ($1024 = 2^{10}$), where 10 bits are allocated for the *offset* portion of each address. A program cannot generate an *offset* value larger than 1023 in 10 bits. Moving to the next location after ⟨*x*,1023⟩ causes a carry into the *page* portion, thereby moving translation to the next page. During the translation, the paging process checks to verify that a ⟨*page*, *offset*⟩ reference does not exceed the maximum number of pages the process has defined.

With a segmentation approach, a programmer must be conscious of segments. However, a programmer is oblivious to page boundaries when using a paging-based operating system. Moreover, with paging there is no logical unity to a page; a page is simply the next 2^n bytes of the program. Thus, a change to a program, such as the addition of one instruction, will push all subsequent instructions to lower addresses and move a few bytes from the end of each page to the start of the next. This shift is not something about which the programmer need be concerned, because the entire mechanism of paging and address translation is hidden from the programmer.

Scanned by CamScanner

Security · Dola Mam

However, when we consider protection, this shift is a serious problem. Because segments are logical units, we can associate different segments with individual protection rights, such as read-only or execute-only. The shifting can be handled efficiently during address translation. But with paging there is no necessary unity to the items on a page, so there is no way to establish that all values on a page should be protected at the same level, such as read-only or execute-only.

Combined Paging with Segmentation

We have seen how paging offers implementation efficiency, while segmentation offers logical protection characteristics. Since each approach has drawbacks as well as desirable features, the two approaches have been combined.

The IBM 390 family of mainframe systems used a form of paged segmentation. Similarly, the Multics operating system (implemented on a GE-645 machine) applied paging on top of segmentation. In both cases, the programmer could divide a program into logical segments. Each segment was then broken into fixed-size pages. In Multics, the segment *name* portion of an address was an 18-bit number with a 16-bit *offset*. The addresses were then broken into 1024-byte pages. The translation process is shown in Figure 4-9. This approach retained the logical unity of a segment and permitted differentiated protection for the segments, but it added an additional layer of translation for each address. Additional hardware improved the efficiency of the implementation.

4.3 CONTROL OF ACCESS TO GENERAL OBJECTS

Protecting memory is a specific case of the more general problem of protecting *objects*. As multiprogramming has developed, the numbers and kinds of objects shared have also increased. Here are some examples of the kinds of objects for which protection is desirable:

- memory
- a file or data set on an auxiliary storage device
- an executing program in memory
- a directory of files
- a hardware device
- a data structure, such as a stack
- a table of the operating system
- instructions, especially privileged instructions
- passwords and the user authentication mechanism
- the protection mechanism itself

The memory protection mechanism can be fairly simple because every memory access is guaranteed to go through certain points in the hardware. With more general objects, the number of points of access may be larger, a central authority through which all accesses pass may be lacking, and the kind of access may not simply be limited to read, write, or execute.

Scanned by CamScanner

Security · Dola Mam

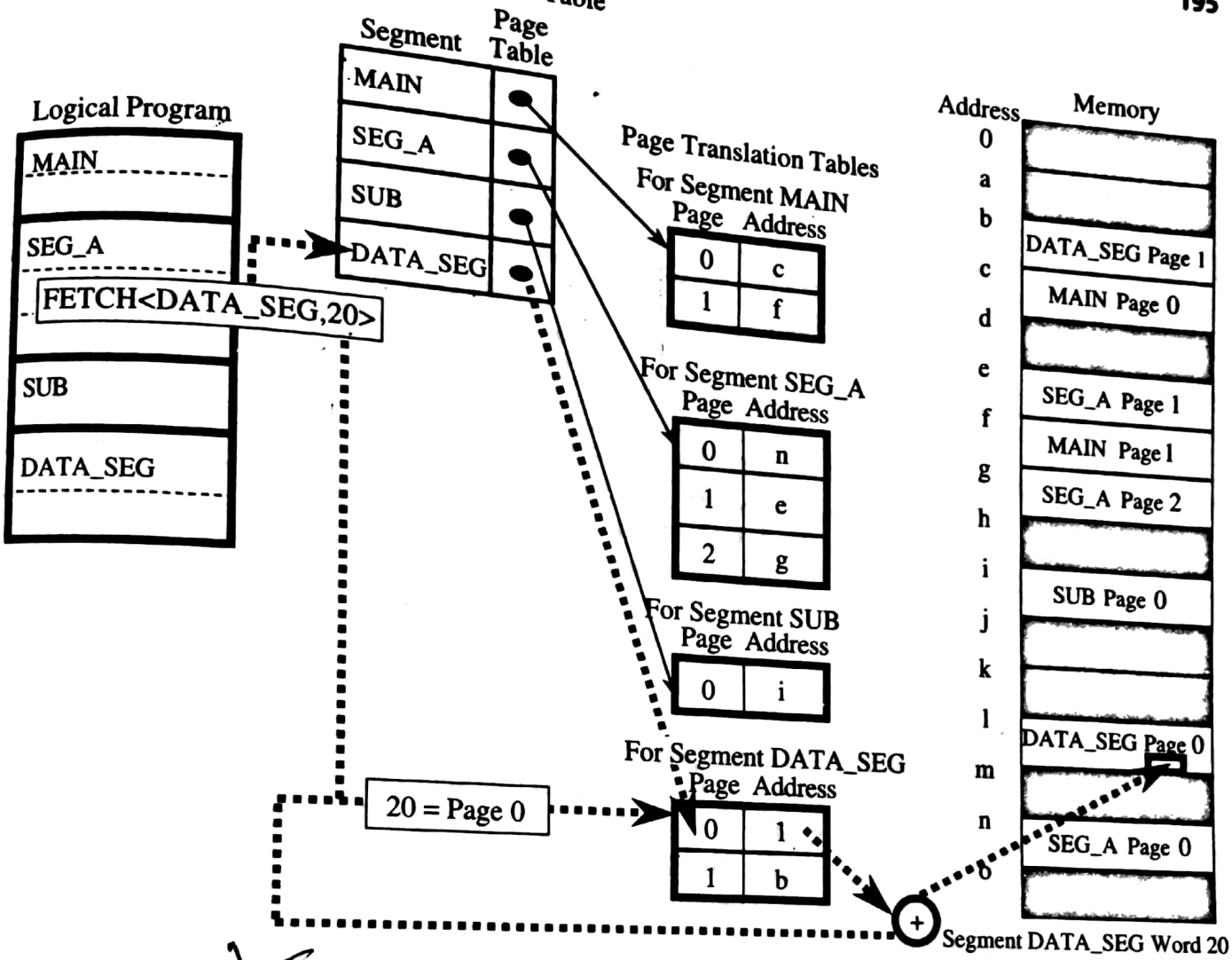


FIGURE 4-9 Paged Segmentation.

Furthermore, all accesses to memory occur through a program, so we can refer to the program or the programmer as the accessing agent. In this book, we use terms like *the user* or *the subject* in describing an access to a general object. This user or subject could be a person who uses a computing system, a programmer, a program, another object, or something else that seeks to use an object.

There are several complementary goals in protecting objects.

- *Check every access.* We may want to revoke a user's privilege to access an object. If we have previously authorized the user to access the object, we do not necessarily intend that the user should retain indefinite access to the object. In fact, in some situations, we may want to prevent further access immediately after we revoke authorization. For this reason, every access by a user to an object should be checked.
- *Enforce least privilege.* The principle of least privilege states that a subject should have access to the smallest number of objects necessary to perform some task. Even if extra information would be useless or harmless if the subject were to have access, the subject should not have that additional access. For

Scanned by CamScanner

Security · Dola Mam

Firewalls

Dola Das
Assistant Professor, Dept. of CSE,
KUET

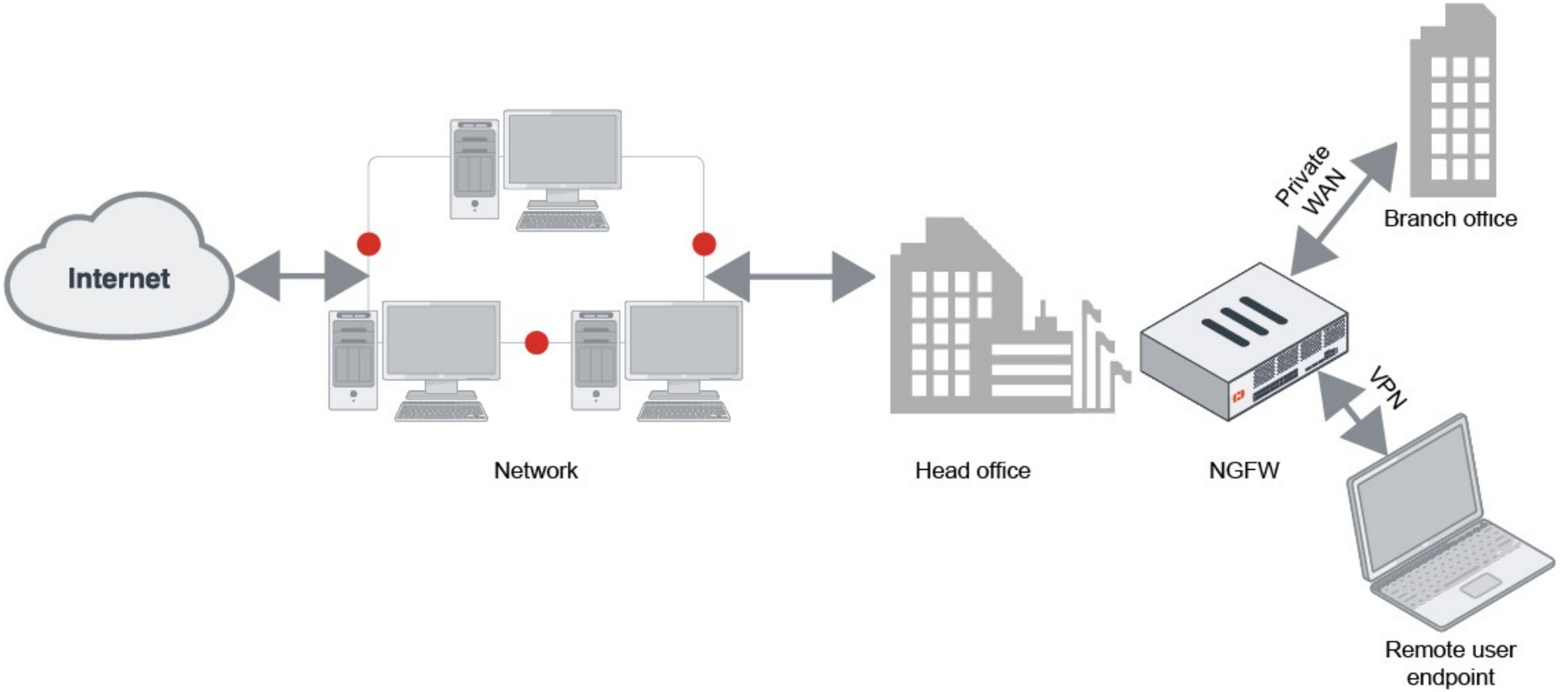
Firewall

- A firewall is a network security device placed at the perimeter of the corporate network
- all the packets entering and leaving the network go through the firewall first
- appropriate actions are taken based on the network rules configured by the organization

Firewall

- Major factors that come into consideration for architecting a firewall:
  - Organization‘s ability to implement and develop the architecture
  - The budget allotted by the organization
  - Objectives of the network

Firewall architecture



Firewall architecture

- There are four common architectural implementations of firewalls widely in use:
  - packet filtering routers,
  - screened host firewalls,
  - dual-homed firewalls and
  - screened subnet firewalls

Firewall architecture

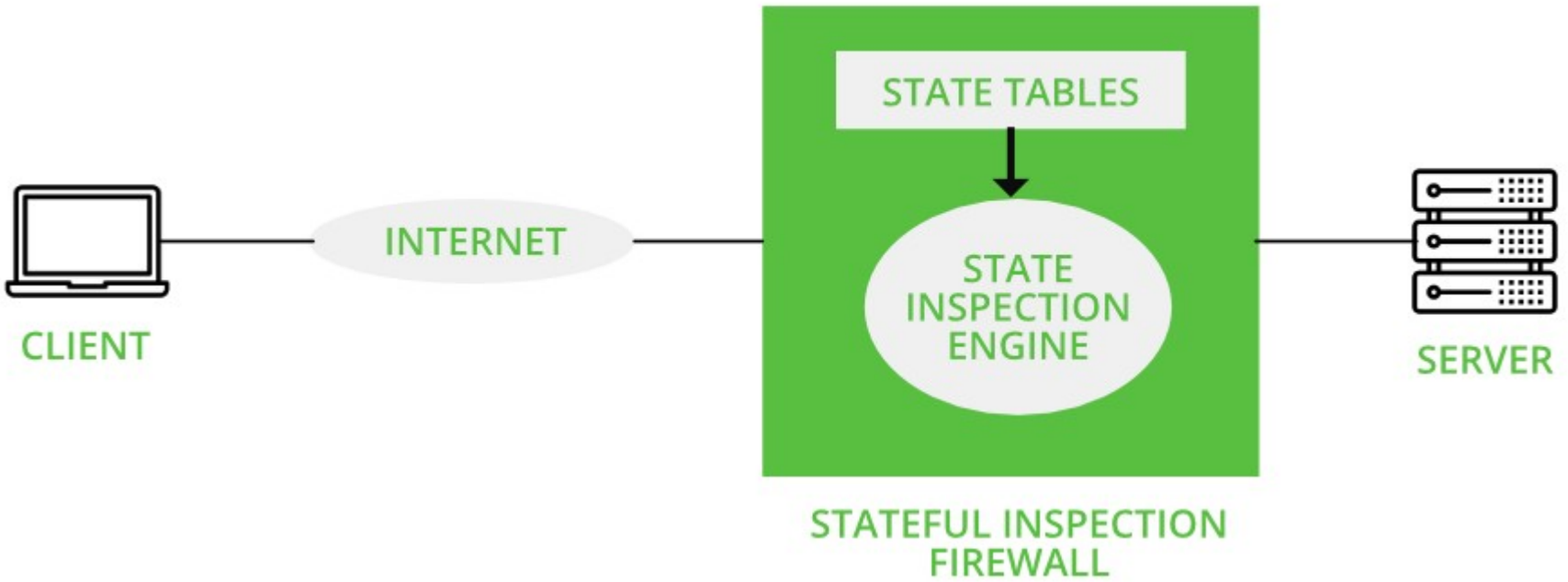
- packet filtering routers:
  - placed at the perimeter between the organization‘s internal networks and the internet service provider.
  - can be configured to accept or reject the packets as per the rule of the organization.
  - one of the simple and effective ways to lower down the organization‘s risk from the internet.

- Drawbacks
  - ✓The length and the complexity of the rule sets implemented to filter the packets can grow and degrade network performance.
  - ✓suffers from a lack of auditing and strong authentication mechanisms.

Stateful Inspection Firewalls

- also referred to as *dynamic packet filtering.*
- monitors the state of active connections and uses the information to permit the network packets.
- filters data packets supported by state and context.
- well suited with Transmission Control Protocol (TCP) and similar protocols. Also with UDP.

Stateful Inspection Firewalls



Stateful Inspection Firewalls

- Advantages:
  - ✓aware of the state of a connection.
  - ✓can detect when illicit data is being used to infiltrate the network
  - ✓has the power to log and store important aspects of network connections
  - ✓do not have to open up a large range of ports to allow communication
- Disadvantages:
  - ✓can be complex to configure.
  - ✓cannot prevent application-layer attacks
  - ✓do not carry user authentication of connections
  - ✓Not all protocols contain state information

Application-proxy Gateway Firewalls

- contain a proxy agent that acts as an intermediary between two hosts that wish to communicate with each other.
- never allows a direct connection between two hosts.
- Each successful connection attempt actually results in the creation of two separate connections.
- external hosts only communicate with the proxy agent, internal IP addresses are not visible to the outside world.
- proxy agent interfaces directly with the firewall ruleset.

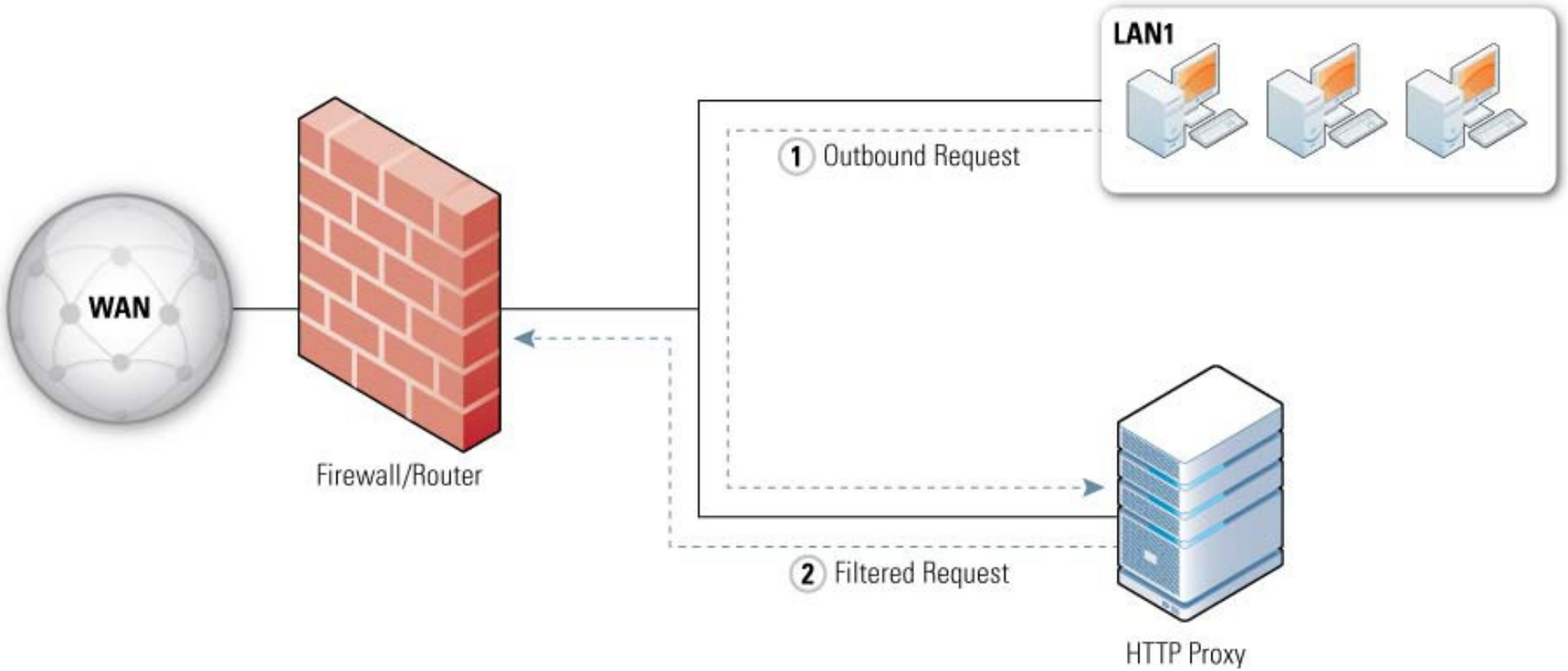
Application-proxy Gateway Firewalls

- Advantages:
  - ✓ can offer a higher level of security for some applications.
  - ✓ inspects traffic content to identify policy violations.
  - ✓ some have the ability to decrypt packets (e.g., SSL-protected payloads), examine them, and re-encrypt them before sending them on to the destination host.
- Disadvantages:
  - ✓ spends much more time reading and interpreting each packet.
  - ✓ poorly suited to high-bandwidth or real-time applications.
  - ✓ tend to be limited in terms of support for new network applications and protocols.

Dedicated Proxy Servers

- usually have much more limited firewalling capabilities.
- differ from application-proxy gateways.
- application-specific, some actually perform analysis and validation of common application protocols such as HTTP.

Dedicated Proxy Servers



- ✓ Many organizations enable caching of frequently used web pages on the proxy to reduce network traffic and improve response times.

Hybrid Firewall

- consist of multiple firewalls, each providing a specified set of functions.
- For instance, you can use one firewall to execute packet filtering while another firewall acts as a proxy.
- can tweak the performance of the security system, taking advantage of the diverse range of capabilities the different firewalls offer.

Hybrid Firewall

- Advantages:
  - ✓ can add a new firewall to an existing security system without having to remove or replace your current firewall.
  - ✓ can allow you to set up a distributed firewall which enables to establish security rules that control access between two separate networks.
  - ✓ Granular control of the protection of the network.
  - ✓ Easier threat isolation.
- Disadvantages:
  - ✓ may unnecessarily complicate your network without providing much tangible benefit.
  - ✓ Some organizations have neither the extra time nor people power to deal with multiple firewall setups.
  - ✓ A relatively simple configuration error can result in a costly breach.

Hybrid Firewall

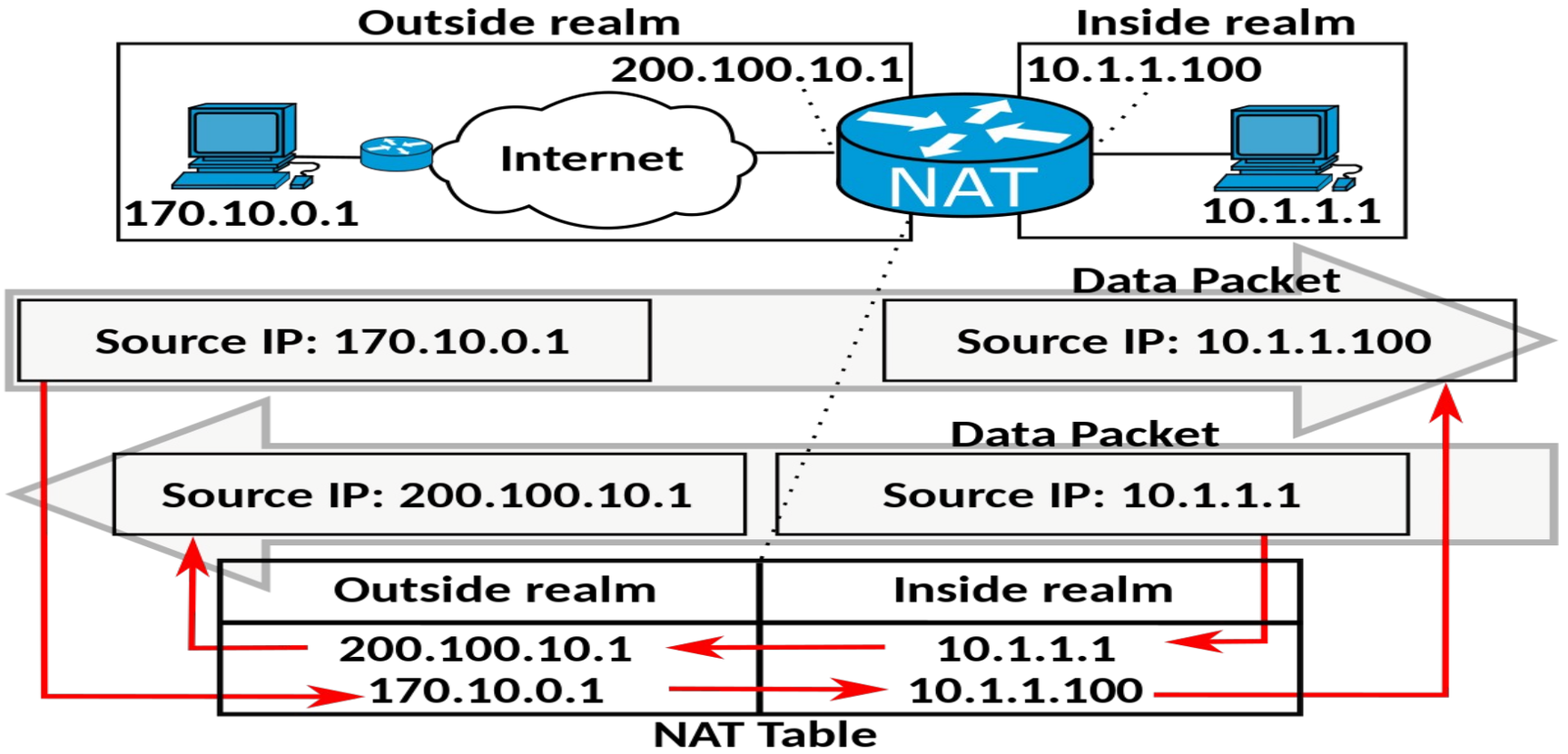
- Before committing to a hybrid firewall solution, it is best to keep the following considerations in mind:
  - ✓ Can you accomplish what you aim to do with a hybrid solution using just a single firewall?
  - ✓ How will a hybrid firewall setup impact network throughput?
  - ✓ How can you get the most from your hybrid firewall?

Network address translation (NAT)

- a method of mapping an IP address space into another.
- A unique public IP address was assigned to each device on the network. These all are at the LAN(Local Area Network) side of the default gateway.
- Assign the IP address of the ISP(Internet Service Provider) to the WAN(Wide Area Network) side of the default gateway.
- Every computer knows the default gateway’s IP address.

Network address translation (NAT)

- Establishing two-way communication:



Network address translation (NAT)

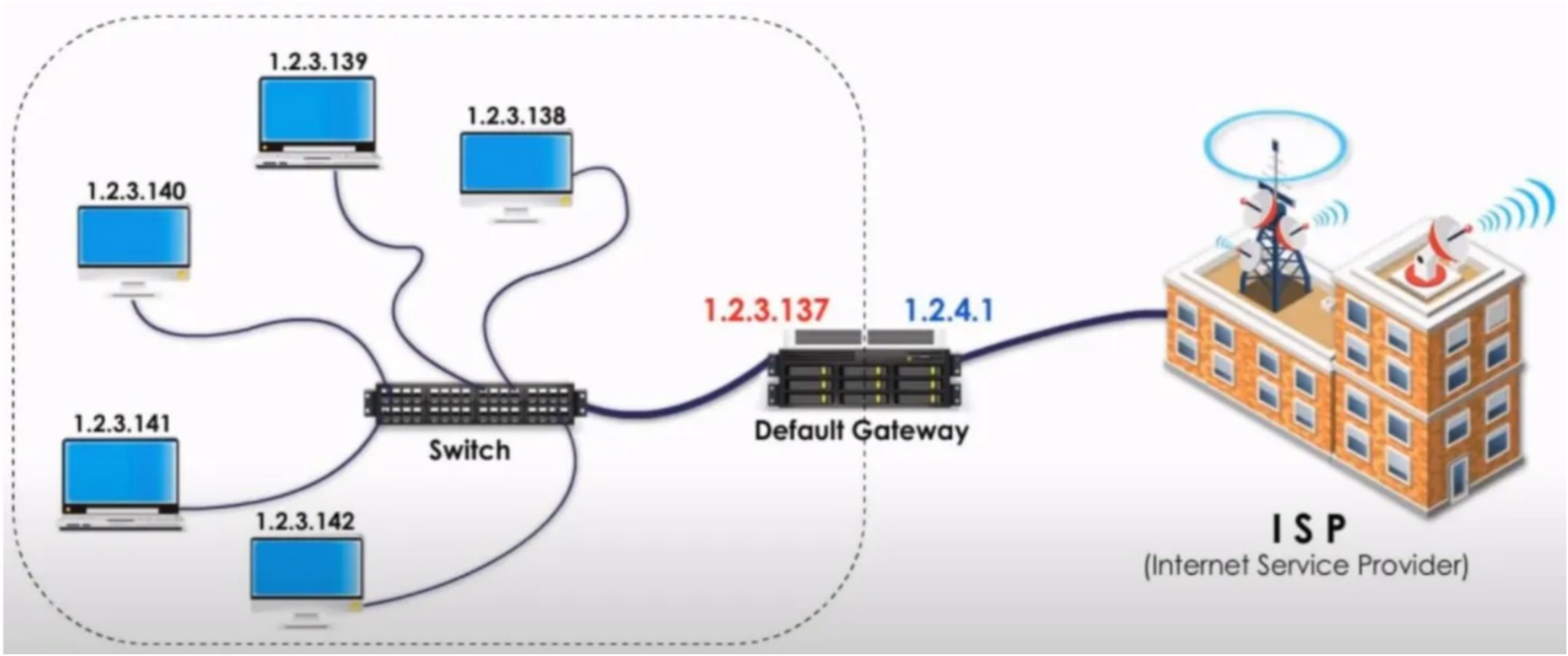
- Applications:
  - ✓Routing
  - ✓Load balancing

Network address translation (NAT)

- Dynamic NAT (DNAT):
  - ✓ mapping of an internal private IP address into the router’s public IP is dynamic
  - ✓ also called IP masquerading
  - ✓ outgoing traffic carries “**a”** public IP of the router
  - ✓ When data comes in, it is looked upon in the NAT table for its destination address.
  - ✓ more secure as hackers can’t get the host address straight away
  - ✓ But still expensive.

Network address translation (NAT)

- DNAT:

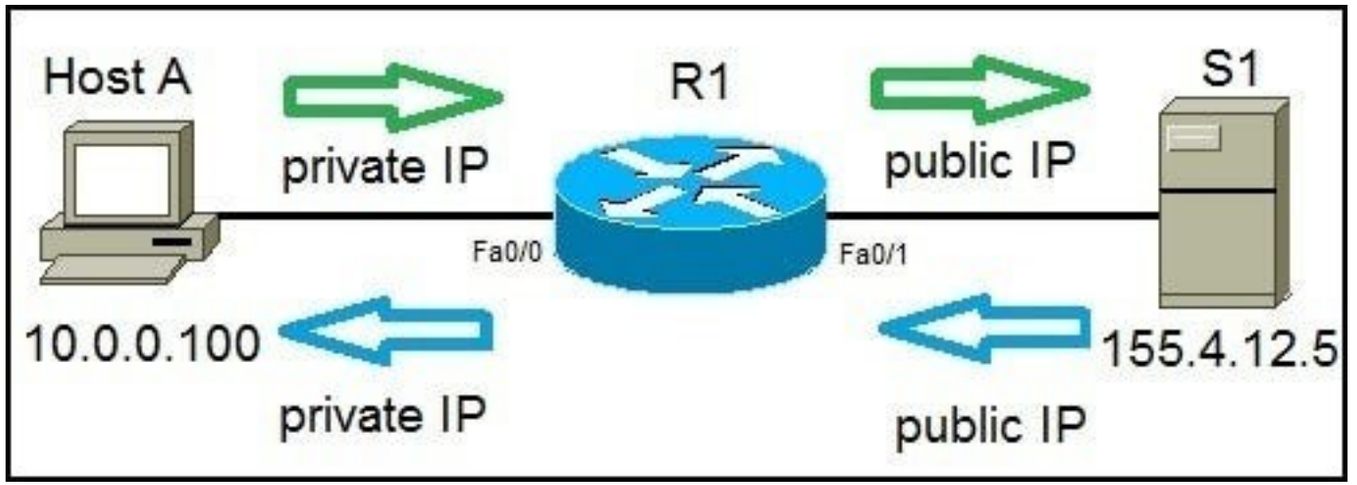


Network address translation (NAT)

- Static NAT (SNAT):
  - ✓ one private address is mapped with one public IP address
  - ✓ These IP addresses never change

Network address translation (NAT)

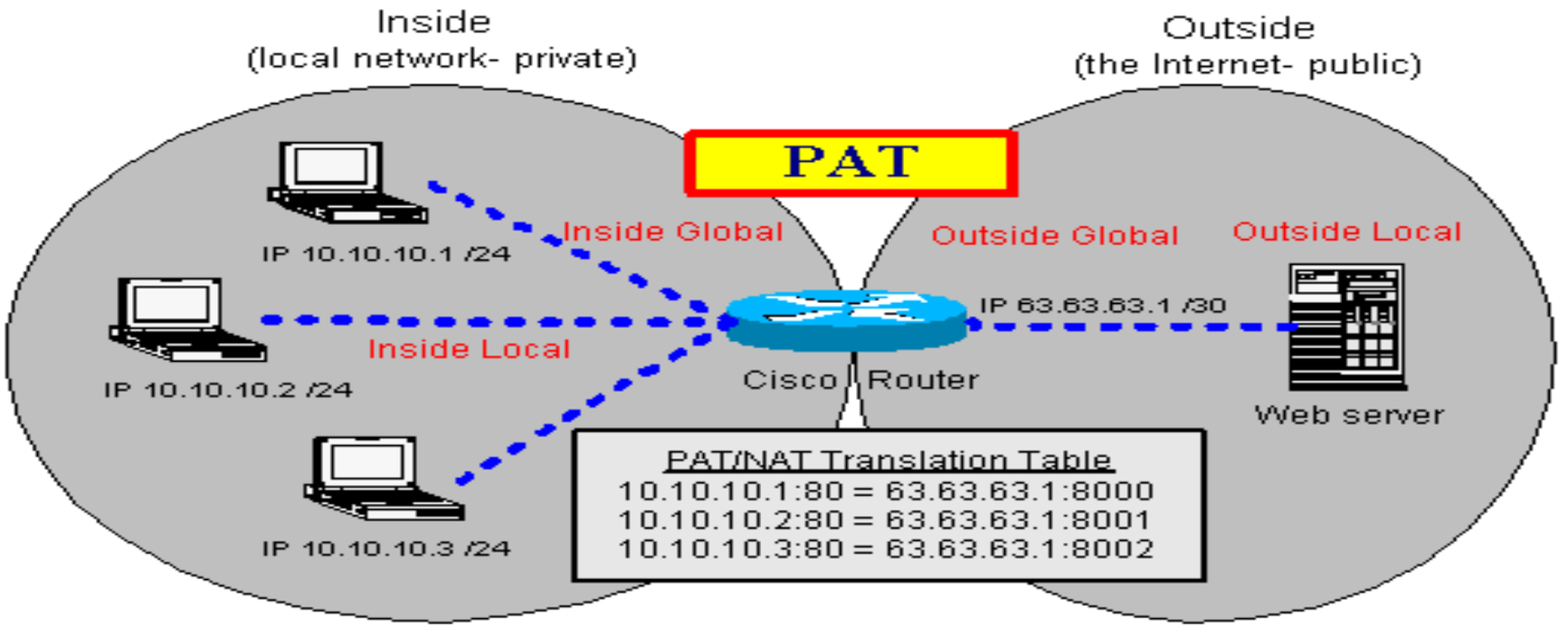
- SNAT:



Port address translation (PAT)

- most common form of NAT.
- Each host on a LAN has its IP address translated into the router’s WAN side IP with a different port number.
- makes each session unique.

Port address translation (PAT)



Demilitarized Zone(DMZ)

- a perimeter network.
- protects and adds an extra layer of security to an organization’s internal LAN from untrusted traffic.
- goal is to allow an organization to access untrusted networks.
- store external-facing services and resources, as well as servers.
- makes it more difficult for a hacker to gain direct access.
- A company can minimize the vulnerabilities of its Local Area Network.
- can communicate efficiently and share information directly via a safe connection.

How Does a DMZ Network Work?

- DMZ network provides a buffer between the internet and an organization’s private network
- DMZ is isolated by a security gateway, such as a firewall
- The default DMZ server is protected by another security gateway that filters traffic coming in from external networks.
- It is ideally located between two firewalls
- even if a sophisticated attacker is able to get past the first firewall, they must also access the hardened services in the DMZ before they can do damage to a business.

Benefits of Using a DMZ

- Enabling access control
- Preventing network reconnaissance
- Blocking Internet Protocol (IP) spoofing

- **DMZ Design and Architecture**
  - ✓Single firewall
  - ✓Dual firewall

Virtual Private Networking (VPN)

- to encrypt and decrypt specific network traffic flows between the protected network and external networks.
- use additional protocols to encrypt traffic and provide user authentication and integrity checking.
- are most often used to provide secure network communications across untrusted networks.

Virtual Private Networking

- two most common VPN architectures are:
  - ✓gateway-to-gateway- connect multiple fixed sites over public lines through the use of VPN gateways.
  - ✓host-to-gateway- provides a secure connection to the network for individual users, usually called *remote users.*

Intrusion Detection System (IDS)

- monitors network traffic and searches for known threats and suspicious or malicious activity.
- sends alerts to IT and security teams
- some can block malicious or suspicious traffic.
- are software applications.
- runs on organizations’ hardware or as a network security solution.

Intrusion in Cybersecurity

- Address spoofing
- Fragmentation
- Pattern evasion
- Coordinated attack

IDS Types

- Network intrusion detection system (NIDS)
- Host intrusion detection system (HIDS)
- Signature-based intrusion detection system (SIDS)
- Anomaly-based intrusion detection system (AIDS)
- Perimeter intrusion detection system (PIDS)
- Virtual machine-based intrusion detection system (VMIDS)
- Stack-based intrusion detection system (SBIDS)

So, What is the Use and benifits of an Intrusion Detection System (IDS)?

Intrusion Detection System (IDS) Challenges

- False alarms
- False negatives

Intrusion Prevention Systems (IPS)

- is also an intrusion detection prevention system (IDPS)
- proactively detects and prevents harm from malicious traffic
- protection identifies potential threats by monitoring network traffic in real time by using network behavior analysis
- includes anti-virus/anti-malware software, firewall, anti-spoofing software, and network traffic monitoring

How does IPS Work?

- Signature-based Detection
- Statistical Anomaly-based Detection

Potential Attacks Detected and Prevented By IPS

- Address Resolution Protocol (ARP) Spoofing
- Buffer Overflow
- Distributed Denial of Service (DDoS)
- Operating System (OS) Fingerprinting
- Etc.

Types of IPS

- Network-based intrusion prevention system (NIPS)
- Wireless intrusion prevention system (WIPS)
- Host-based intrusion prevention system (HIPS)
- Network behavior analysis (NBA)

IDS vs. IPS

- An IDS solution is typically limited to the monitoring and detection of known attacks and activity that deviates from a baseline normal prescribed by an organization.

- An intrusion prevention system (IPS) goes beyond this by blocking or preventing security risks. An IPS can both monitor for malicious events and take action to prevent an attack from taking place.

Firewall Administration

- Administrators can configure, manage, and monitor networks firewalls
- One can customize role-based administrative access to the management interfaces to delegate specific tasks or permissions to certain administrators
- Improper firewall configuration can result in attackers gaining unauthorized access

Firewall Administration

- Secure the Firewall
- Establish Firewall Zones and an IP Address Structure
- Configure Access Control Lists (ACLs)
- Test the Firewall Configuration
- etc.

Firewall Positioning Policies

- A firewall policy dictates how firewalls should handle network traffic for specific IP addresses and address ranges, protocols, applications, and content types based on the organization’s information security policies.

- **Types:**
  - ✓ Policies Based on IP Addresses and Protocols
  - ✓ Policies Based on Applications
  - ✓ Policies Based on User Identity
  - ✓ Policies Based on Network Activity

References

- https://resources.infosecinstitute.com/topic/firewall-types-and-architecture/
- https://www.geeksforgeeks.org/what-is-stateful-inspection/
- https://www.fortinet.com/resources/cyberglossary/hybrid-firewall-advantages-disadvantages#:~:text=What%20Is%20a%20Hybrid%20Firewall%3F%20Hybrid%20firewalls%20consist,filtering%20while%20another%20firewall%20acts%20as%20a%20proxy.
- https://medium.com/networks-security/nat-snat-dnat-pat-port-forwarding-b7982fab02cd
- https://www.fortinet.com/resources/cyberglossary/what-is-dmz
- https://www.fortinet.com/resources/cyberglossary/intrusion-detection-system#:~:text=What%20is%20an%20Intrusion%20Detection%20System%20%28IDS%29%3F%20An,when%20it%20detects%20any%20security%20risks%20and%20threats.
- https://www.fortinet.com/resources/cyberglossary/what-is-an-ips
- https://www.fortinet.com/resources/cyberglossary/firewall-configuration

Thank you